UiA University of Agder

Master

# Fantasy Premier League: A Data-Driven Approach

Exploring LSTM-networks for performance forecasting

Andersen, E.B. // Molberg, P.A.

SUPERVISOR

Jochen Jungeilges

**University of Agder, 2025**

Faculty of Business and Law

UiA Business School

# 1 Acknowledgment

This thesis has been written as part of the Master's programme in Analytical Finance at the University of Agder.

We would like to extend our sincere gratitude to our supervisor, Professor Jochen Jungeilges, for his unwavering support, constructive feedback, and encouragement throughout this journey. From the very beginning, he embraced our choice of topic—despite its departure from conventional finance research—with enthusiasm and open-mindedness.

His guidance has been instrumental in shaping both the direction and the quality of this thesis, and we are deeply appreciative of the freedom he granted us to explore our ideas independently.

This work is the result of both academic curiosity and a shared passion for applying analytical tools to unconventional problems, and we are grateful for the opportunity to pursue it under his mentorship.

# 2 Abstract

This thesis explores a data-driven approach to the cultural phenomenon Fantasy Premier League, investigating whether a purely data-driven method relying on historical in-game data and optimisation can replicate or beat human performance. The forecasting method Long-Short term memory neural networks is employed, opting to produce four models for each on-field football position. The sample period of the data-set is seasons 22/23,23/24 and 24/25 (round 1 to 31) of the English premier league. The results show that the models rapidly over-fit on its training data. The results lead to the conclusion that, the models fail to reproduce the forecasting ability of human players, and obtain score a lower than that of human participants.

# Contents


**1 Acknowledgment** **i**

**2 Abstract** **ii**

**3 Introduction** **1**

3.1 Brief introduction to the Fantasy Premier League Game . . . . . 1

3.2 Motivation . . . . . 2

3.3 Research problem . . . . . 2

3.4 Thesis outline . . . . . 3

**4 Literature Review** **3**

**5 Data** **6**

5.1 Data Processing . . . . . 7

5.2 Descriptive Statistics . . . . . 8

**6 Methodology** **12**

6.1 Rolling window . . . . . 12

6.2 Z-scaling . . . . . 12

6.3 In-sample and out-of-sample . . . . . 13

6.4 Lasso regression . . . . . 13

6.5 K-fold cross validation . . . . . 14

6.6 Recurrent Neural Networks Overview . . . . . 15

6.7 Long Short-Term Memory . . . . . 16

6.8 Optimisation Method . . . . . 18

6.9 Digital Tools . . . . . 19

**7 Results** **20**

7.1 Feature Selection . . . . . 20

7.2 Keras LSTM Model structure . . . . . 22

7.3 LSTM Model performance . . . . . 22

7.3.1 Training and validation . . . . . 22

7.4 Squad Optimisation Results . . . . . 25

**8 Discussion** **27**

8.1 Feature set . . . . . 27

8.2 Data split, Callbacks and No Holdout set . . . . . 28

8.3 The LSTM Models . . . . . 28

8.4 MILP . . . 29
8.4.1 Gamechips . . . 30

**9 Conclusion** **32**
9.1 Suggestion for further research . . . 33

**Appendices** **36**

**A Points System** **36**

**B Descriptive Statistics** **38**
B.1 Tables . . . 38
B.1.1 Position Statistics Tables . . . 40
B.2 Figures . . . 43
B.2.1 Box Plots for Starting Players . . . 43
B.2.2 Box Plots for All Players . . . 44
B.3 Box Plots for Players With 1+ Minutes Played . . . 45
B.4 Additional Plots . . . 46
B.5 QQ Plots . . . 48
B.6 Model Structures . . . 50

**C Squad optimisation model constraints** **51**
C.1 Model Constraints . . . 51
C.1.1 Full Squad Constraints . . . 51
C.1.2 Starting Squad Constraints . . . 53
C.1.3 Captaincy constraints . . . 53
C.1.4 Substitution constraints . . . 54
C.1.5 Budget Constraints . . . 54
C.1.6 Gamechips . . . 55
C.1.7 Transfer Constraints . . . 55

**D Python Script: Mixed Integer Linear Program** **56**

**E Squad Results R Plots** **90**

**F R-Script 1: Fetching Data** **107**

**G R-Script 2: Exploratory Data Analysis** **116**

**H R-Script 3: Keras FPL LSTM Model** **148**

**I Discussion paper: Responsibility - Emil B. Andersen** **186**
I.1 Responsibility considerations . . . 187
I.1.1 Gambling concerns . . . 187
I.1.2 Internet addiction . . . 187
I.1.3 Mental health implications . . . 188
I.2 Paths for mitigation . . . 189
I.3 Concluding remarks . . . 189

**J Discussion Paper International - Per Anders Molberg** **192**
J.1 About the thesis . . . 192
J.1.1 Fantasy Premier League . . . 192
J.1.2 The Master Thesis . . . 193
J.2 Defining International . . . 194
J.3 Discussion of International Trends and Internationalisation of Fantasy Premier League . . . 195
J.4 Conclusion . . . 196

# 3 Introduction

The English Premier League (EPL) is the most lucrative and influential elite football league in the world (Deloitte, 2024). Its cultural reach spans continents, with millions of viewers tuning in weekly - one might say the sun never sets for the English Premier League.

Alongside it, the Fantasy Premier League (FPL) has grown into a digital phenomenon since its launch in 2002, engaging over 11 million players in a season-long competition of strategy and prediction. Participants select athletes they expect to perform well and earn points based on actual match outcomes. What began as a niche hobby has evolved into a competitive, data-driven pursuit, with several non-academic books being released on the topic in various languages.

The popularity of both the EPL and FPL has given rise to a wealth of accessible, high-quality data on player performance, team dynamics, and gameweek outcomes. This abundance of structured and timely information makes the FPL an ideal setting for quantitative analysis and predictive modelling—an aspect that aligns closely with interests in analytical finance and decision science.

## 3.1 Brief introduction to the Fantasy Premier League Game

The FPL is a season-long online game based on the real-life performances of players in the EPL. At the start of each season, participants assemble a squad of 15 EPL players, comprised of 2 goalkeepers (GK), 5 defenders (DEF), 5 midfielders (MID), and 3 forwards (FWD). Importantly, no more than three players can be selected from any single EPL team, and the total squad value must not exceed £100 million. Each player is assigned a cost that reflects their expected ability to earn points, which are awarded weekly based on real-life performance metrics such as goals, assists, clean sheets, and more.

The main objective is to maximise the total number of points accumulated throughout the season. Each gameweek (GW), participants select a starting line-up of 11 players from their squad. This line-up must include exactly 1 GK, at least 3 DEF, and at least 1 FWD. The remaining 4 players are placed on the bench in a prioritized order and do not contribute to the gameweek score unless they are automatically substituted in to replace starting players who did not feature in their real-life matches.

Participants also designate one starting player as captain and another as vice-captain. The captain's points are doubled, while the vice-captain takes over this role if the captain does not play during the gameweek.

To manage their team throughout the season, participants are granted one free transfer each gameweek, allowing them to swap out one player without penalty. Unused free transfers

can be rolled over, up to a maximum of five. Additional transfers beyond the free quota incur a penalty of four points per transfer, which are deducted from the gameweek's total score.

To add strategic depth, the FPL provides four gamechips that can be used to enhance a team's performance. The Triple Captain chip increases the captain's score threefold instead of doubling. The Bench Boost chip allows all 15 players, including substitutes, to contribute points in a single gameweek. The Wildcard chip permits unlimited transfers for one gameweek, with changes becoming permanent. The Free Hit chip also allows unlimited transfers, but the team reverts to its original state after the gameweek. Each chip, except the Wildcard - which can be played twice - can be used only once per season.

## 3.2 Motivation

As students of analytical finance, we find the sequential and constrained decision-making process embedded in the FPL highly intriguing. Each gameweek presents a new set of interdependent choices: selecting a team, managing transfers, and allocating scarce resources—all under uncertainty. In addition, the FPL's data-rich environment makes it particularly compelling for a quantitative approach. With detailed player statistics available across numerous metrics and time horizons, the game naturally lends itself to a quantitative, model-driven approach. This abundance of real-time and historical data enables the application of predictive models, optimisation techniques, and risk-adjusted decision-making frameworks commonly used in finance. Furthermore, the quantitative approach removes supporter bias, wherein participants select players from their favourite teams.

In fact, the challenge of optimal team selection in FPL closely resembles that of constructing a financial portfolio. Participants must allocate capital across multiple assets (players), within a strict budget, in pursuit of maximising returns (fantasy points). In both settings, expectations about future performance guide allocation decisions, and success depends on balancing short-term opportunities with long-term planning.

Our motivation is to test the versatility of financial methodologies outside traditional markets, in a culturally significant and analytically demanding setting. FPL serves as a unique and engaging test-bed for this purpose—offering a rich, dynamic, and widely recognized platform for applying the tools of finance in a new domain.

## 3.3 Research problem

The aim of this thesis is to address the question of whether or not a purely data driven approach, relying exclusively on historical in-game data and forecast-driven optimisation can replicate or beat human performance at a complex and information driven game like the FPL.

Other papers have employed different forecasting techniques, such as regression, Bayesian Q-learning and modified averages. To the authors' knowledge however, the deployment of neural networks is yet to be tested academically.

In this thesis, the research problem is two-fold. To optimise the team selection, we first need to have an idea of which players are likely to perform well. To that end, this thesis employs Long Short-Term Memory (LSTM, see 6.7) networks. Once obtained, the forecasts are translated into an optimal 15-player squad which satisfies all FPL constraints. This is done through a Mixed Integer Linear Program (MILP). The MILP also takes care of selecting a starting line-up, substitute priority, selecting a captain and vice-captain, as well as deployment of all gamechips.

### 3.4 Thesis outline

The remainder of the thesis is organized as follows. Chapter 4 gives an overview of previous work in the genre, mainly focusing on quantitative approaches to the FPL. In chapter 5, a thorough description of how we collected the data and how we processed it is given, as well as a brief introduction to our feature set. Chapter 6 gives a comprehensive explanation on our methodology and the justification behind it. Chapter 7 presents the results of our thesis, from the training of the LSTM-networks to the final ranking achieved when put through the MILP. In chapter 8, the implications of the results are discussed. Finally, chapter 9 contains our concluding remarks.

## 4 Literature Review

Setting out to give future researchers in the growing field of Fantasy Sports Research (FSR) an overview of the current research body, Wilkins (2024) conducted a bibliometric analysis with 230 publications reviewed. The author chose articles dependent on several criteria, for example results from literature search need to be articles from peer-reviewed journals rather than book chapters and it must be in English. They find that the field gained traction around the years of the financial crisis. Overwhelmingly, articles are written and published in the United States of America. The most cited papers seem to focus on the behavioural aspect of Fantasy Sports(FS), that is researchers are interested in game participants and the relationship between consumption of game and sport. The top three journals are in order: *Sport Marketing Quarterly, Gaming Law Review and Economics, Journal of Gambling Studies and Journal of Sport Management (tie)* and the most common journal domain is Business and Economics. The most explored themes relate to the externalities from fantasy sports, like addictive behaviour or understanding how and why participants experienced

the games. Furthermore, most FSR is original research based on surveys and observational data. Lastly, they find that many papers have no key words (24.8 percent), and the most frequented key word was " Fantasy Sport ".

The work of Wilkins (2024) highlights the scarcity of quantitative approaches in the fantasy sports literature. As this thesis adopts a data-driven perspective, the remainder of the review focuses on similar contributions. First out, **Sargent and Bedford (2010)** sought to improve player performance forecasts in the Australian Football League (AFL) by employing optimised non-linear smoothing. They argue that, since AFL performance data exhibits non-Gaussian properties, traditional linear smoothing methods may not be optimal. Instead, they propose a two-step approach, where a Tukey non-linear smoother is applied to the raw performance data before exponential smoothing is used to further pre-process the data. To generate forecasts, Monte-Carlo simulations were conducted on each player, with confidence intervals constructed from the results of the simulations. Their findings indicate that their combined smoothing approach can significantly reduce error in predictive accuracy compared to mean projection and simple exponential smoothing.

**Bonomo et al. (2014)** studied mathematical programming as a tool for points optimisation in Gran DT, an Argentinian fantasy soccer game. Two different models were applied, namely *a posteriori* and *a priori*. The former optimised team selection based on realized points, i.e. after data is known, representing the best possible team configuration. The latter optimised team selection with respect to estimated points returns. To obtain estimates, a modified average of player points, including home or away status, league position of the opposing team and if the player was on a scoring/winning streak was computed. The *a priori* model consistently achieved a top 10% rank over numerous seasons. For further research, feature engineering and portfolio optimisation methodologies from finance are suggested to improve their model.

**Matthews et al. (2012)** addressed the challenge of automating sequential team selection over an entire FPL season. They modelled the problem as a Bayesian reinforcement learning task, where decisions had to be made under uncertainty regarding player performance, fixture difficulty, and injuries. The manager's decision-making process was formulated as a belief-state Markov Decision Process (MDP), enabling the agent to update its beliefs about player abilities as new data became available. To handle the computational complexity of selecting from over 500 players, team selection was framed as a multi-dimensional knapsack problem (MKP). Various techniques were tested to solve the belief-state MDP under different computational constraints. The top-performing model, a Bayesian Q-learning approach with a three-minute convergence limit, achieved a rank of 26,065 out of 2.5 million participants, placing it in the top 1.1% of managers.

**Eilertsen et al. (2018)** set out to develop a forecast-based optimisation model for the FPL. Four different forecasting approaches were tested. The first approach built on the work of Bonomo et al. (2014), computing a modified average of realised points in prior gameweeks. The second approach was rooted in linear regression, with players being categorized by position before feature selection through lasso regression was performed. A linear regression model was then fitted to estimate points returns. The third approach converted bookmakers odds of individual player and team performances into probability distributions, before computing point estimates rooted in the distributions. Due to the computational demands of stochastic solutions, the optimisation model was solved with a deterministic approach, using a rolling horizon heuristic with expected player points as inputs. Two different models were tested, with and without constraints akin to portfolio optimisation, optimising over starting XI variance in points returns. A qualitative gamechip strategy was also implemented, targetting blank- and double gameweeks. Their results reveal that the modified average was indeed an improvement over Bonomo et al. (2014), ranking in the top 8% of players and being their top performer. Interestingly, the model performed better when excluding gamechips for the modified average approach as well as the bookmakers odds approach. However, when including portfolio optimisation constraints in their model, and testing for the modified average method with gamechips included, a top 1.5% rank was achieved. They conclude that outright winning the FPL appears to be unachievable with the applied methods, pointing to stochastic solutions, machine learning and better gamechip strategy implementation as further branches of research.

A more recent paper exploring player performance and squad optimisation is by **Venter and Van Vuuren (2024)**. The paper approaches this challenge as a closing-window combinatorial optimisation problem. Using multiple different statistical learning techniques for forecasting, and a mixed binary programming model for squad selection. The statistical techniques and learning techniques employed include: naïve forecasts, different moving averages(MA), Exponential smoothing, regression models, Autoregressive Integrated Moving Average (ARIMA) models, Decision trees, k-Nearest Neighbour(kNN), Croston's method, and Ensembles of models. The mixed binary programming model relies on both, inputs of historical data and forecasts, i.e. it utilizes past performance along with forecasts of expected player performance to make teams sequentially. It works in a rolling window fashion and is solved iteratively. As in our thesis, Venter and Van Vuuren procure data from the GitHub repository made by Anand for past player performance data and to predict future performance. With data spanning the 2016/2017 season to 2020/2021. With the first four seasons being used to train and validate the models and the last season being used as the test and comparison set. Opting for Mean Absolute Scaled Error to measure the relative

performance of the forecasting methods for automating the ensemble building process. Unfortunately since the automation algorithm creates the ensembles that are best at predicting each individual player it is not put forward which models performs the best across the board. However, the study finds that the ensemble models outperform their single model counter parts, with EXP techniques and Croston performing well. Their best result lands in the top 4% of players worldwide for the 2020/21 season.

Lastly, a qualitative contribution to the FPL library with regards to strategy. **O'Brien et al. (2021)** investigate the presence of skill versus luck in FPL. Using a large dataset of participants' historical performances, they examine whether successful managers exhibit consistency across multiple seasons. Their findings indicate that skill plays a significant role, as top managers tend to perform well consistently, and the average participant improves over time. When analysing a single season gameweek by gameweek, they identify key differences in decision making. Specifically, successful managers make better preparations for Gameweek 1, ensuring a strong start, and adapt more effectively to blank gameweeks (where some teams do not play) and double gameweeks (where some teams play twice). These insights suggest that long-term planning and informed decision-making are crucial factors distinguishing top-performing managers from the rest.

# 5 Data

The data for our thesis, made by Anand (2025), were retrieved from the GitHub repository "Fantasy-Premier-League". The repository has collected and stored data from the official FPL API stretching back to the 2016/2017 season. However, in its longest form, the dataset has a lower grade of information and lacks some features which were only available from the 2022/2023 season onwards. Thus, only data from the 2022/23 season onwards has been used in this paper. The rationale behind using the Vaastav repository as a middleman instead of alternatives or the official FPL API are: 1. It is free of charge, 2. The Official API does not allow for detailed player data for non-active seasons, 3. Fetching to R via GitHub as RAW .CSV files is simple, and finally, 4. It is maintained on GitHub by multiple contributors. Additionally, we wanted to ensure some baseline continuity in the dataset, which would be impossible if we were to use some obscure .CSV save-files of the FPL Seasons on different websites. To the best our knowledge, this repository had the richest feature set and quality for previous seasons, at the available price of zero NOK. Attempts were made to locate other sources as in Eilertsen et al. (2018), however, most of the available data was of poor quality, behind paywalls or deprecated.

## 5.1 Data Processing

The data is fetched from RAW files available from the repository on a per season basis, except the current season (2024-25), where it is fetched weekly. The introduction of the gamechip Assistant Manager, in where the participant hires a real life manager on their fantasy squad for 3 weeks as their "Assistant Manager", made season-wise fetching impossible for the current season. Therefore, we aggregate the weeks ourselves and remove the unrelated information. Additionally, the 24/25 season introduced some name changes, because of this we had to spend time manually finding and renaming players who had their name changed. We also converted all names to conform with the Latin alphabet. The dataset spans from the first gameweek Friday 5 August in Season 22/23 to the cut-off time at gameweek 31 in the 24/25 season, which had its last game Monday, 7th of April 2025. We refer to appendix F for more on the practical implementation.

The dataset has a total of 78,025 observations on 41 different variables. The table 1 below shows a sample of the dataset for Chelsea FC midfielder Cole Palmer in tabular form. Mind, the table is split in half and transposed for displaying purposes.

| **Variable** | **Value** | **Variable** | **Value** |
|---|---|---|---|
| 1. name | Cole Palmer | 2. position | MID |
| 3. team | Chelsea | 4. xP | 9.5 |
| 5. assists | 3 | 6. bonus | 2 |
| 7. bps | 61 | 8. clean sheets | 0 |
| 9. creativity | 43.8 | 10. Exp. assists | 0.15 |
| 11. Exp. goal involvements | 0.82 | 12. Exp. goals | 0.67 |
| 13. Exp. goals conceded | 1.8 | 14. fixture | 20 |
| 15. goals conceded | 2 | 16. goals scored | 1 |
| 17. ict index | 14.8 | 18. influence | 82.8 |
| 19. kickoff time | 2024-08-25 13:00 | 20. minutes | 82 |
| 21. opponent team | Wolves | 22. own goals | 0 |
| 23. penalties missed | 0 | 24. penalties saved | 0 |
| 25. red cards | 0 | 26. Round | 2 |
| 27. saves | 0 | 28. selected | 3258422 |
| 29. starts | 1 | 30. Team a score | 6 |
| 31. team h score | 2 | 32. threat | 21 |
| 33. total points | 17 | 34. transfers balance | -189274 |
| 35. transfers in | 66076 | 36. transfers out | 255350 |
| 37. value | 105 | 38. was home | FALSE |
| 39. yellow cards | 1 | 40. GW | 78 |

Table 1: Example Row from Dataset (Cole Palmer), Transposed

We will briefly explain the non-obvious variables in table 1. Note that all variables are given after-match: Variable **4. xP** represents what the FPL expects in total points for each match. **6. Bonus** represents the awarded bonus points given to the three/four(3, 2 or 1

point(s), draws are allowed) best players for each game, **7. bps** is the ranking score for Bonus for each match, i.e. 61 bps and 2 bonus points was awarded to Cole Palmer . In the sample match this implies someone else had a greater bps score than Cole Palmer. **8. Clean Sheets** tracks whether or not the team of the player conceded a goal or not. **9. Creativity, 17. ICT Index, 18. Influence, 32. Threat** are variables from the game which models various metrics of player performance, such as impact on match, ability to produce goal scoring opportunities, likelihood to score, on-field ability to position themselves amongst others (Premier League, 2023). ICT is the aggregation of Influence, Creativity and Threat into one score. **11.-13, Expected (Exp.) goal involvements, goals, goals conceded** contain information about how many goals would be expected on average based on where players took their shots from. For example, based on the shooting positions of Cole Palmer in gameweek 2, he was expected to score 0.67 goals. **26. Round, 40. GW(Gameweek)** Indicates which round of Premier League we are in, in this case it was the second round of the current Premier League Season, we record this as the 78th gameweek because we add the seasons together. **28. Selected** this shows how many game participants (managers) have the football player Cole Palmer on their fantasy team (squad), for this specific gameweek. For round 2 there were 3'258'422 duplicates of him on the same amount of unique squads.**29. Starts** indicates whether or not the player is starting on-field or not, 1 if yes otherwise 0. **30-31. team a/h score** How many goals the away/home team scored. **34.-36. Transfers Balance,In,Out** these variables shows player $p$ transfers, net, in and out. For the example in table 1 more managers transferred Cole Palmer out (255'350) of their squad before the Gameweek started than in (66'076). **37. Value** refers to the players in-game marketplace value, the game interface divides the dataset value by ten. I.E. in this case Cole Palmer's in-game value would be 10.5 Million Pound Sterling.

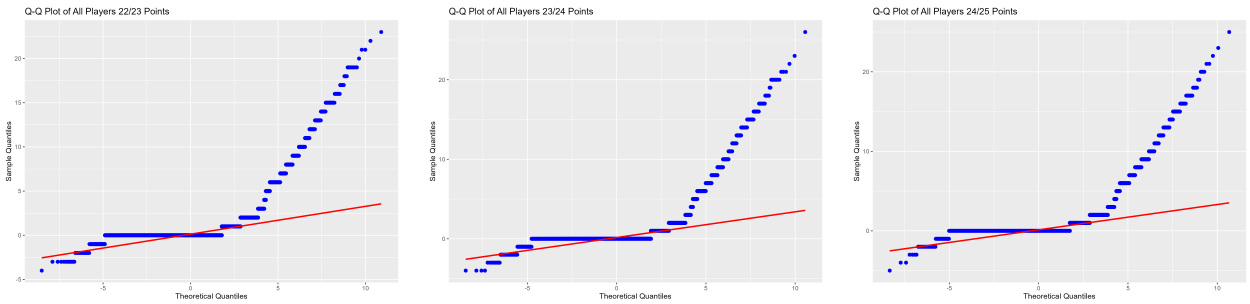
## 5.2 Descriptive Statistics

The distribution of the total points appears to be non-normal (see figures 1,4, 3), and differs depending on which position the players have. The total-points series appears to be non-stationary throughout our sample period. Inspecting the left-hand side of figure 3, where each player has been subjected to the Augmented Dickey-Fuller (ADF) test for stationarity, 302 players fail to reject the null hypothesis. Season-specific descriptive statistics are presented in table 2 for all players (AP). In appendix B there are additional descriptive statistics and plots with different data subsets. In the distribution plots in figure 4 below, the different shapes that the realisations take on based on the footballers position are presented. Where red represents GK, blue represents DEF, green represents MID and FWD is represented in purple.

| Metric | All Players (AP) 22/23 | AP 23/24 | AP 24/25 GW[1-31] |
|---|---|---|---|
| n - Observations | 26505.00 | 29725.00 | 21750.00 |
| n - Null Observations | 15744.00 | 19072.00 | 12925.00 |
| n - NA Observations | 0.00 | 0.00 | 0.00 |
| n - distinct players | 778 | 867 | 777 |
| Min value observed | -4.00 | -4.00 | -5.00 |
| Max value observed | 23.00 | 26.00 | 25.00 |
| Value range | 27.00 | 30.00 | 30.00 |
| Sum observed values | 31724.00 | 31271.00 | 25374.00 |
| Median value | 0.00 | 0.00 | 0.00 |
| Mean value | 1.20 | 1.05 | 1.17 |
| Variance | 5.55 | 5.26 | 5.44 |
| Standard deviation | 2.36 | 2.29 | 2.33 |
| Skewness | 2.96 | 3.36 | 3.13 |
| Kurtosis | 13.59 | 17.01 | 15.17 |

Table 2: Total Points Descriptive Statistics for All Players in the dataset

Figure 1: Q-Q Plots for all players across seasons




(a) Q-Q Plot of all players for season 22/23

(b) Q-Q Plot of all players for season 23/24

(c) Q-Q Plot of all players for season 24/25

Figure 2: Q-Q Plot of all players for all seasons

Keep in mind when reading table 2, the cut-off date is gameweek 31 for the ongoing season. This means the most comparable seasons are 22/23 and 23/24 as these are fully recorded. From table 2 we observe that from the first season to the second season the number of observations increases, owing to the increase in players. There is no missing data in our dataset. However, slightly unique to FPL (sports) data is the high number of null observations owing to football teams' large set of non-playing players, i.e. reserves. The minimum and maximum vales observed for all seasons are -5 (24/25) total points and 26

(23/24) total points, the lowest total point range is 27 (22/23) and the highest is 30 (23/24, 24/25). Despite having a higher number of players, the second season achieves a lower total points sum of 31'271 than the first at 31'724. The median value for all seasons are 0, while the mean varies from season to season in order 1.2, 1.05, and 1.17 mean total points per player respectively. The variance and standard deviations are in order 5.55, 5.26, 5.44 and 2.36, 2.29 and 2.33 respectively. Our skewness (2.96, 3.36, 3.13) and kurtosis (13.59, 17.01, 15.17) tells a tale of a leptokurtic right skewed distribution with fatter tails and more extreme observations compared to a normal distribution. Through the Kolmogorov-Smirnov test (see right-hand side of figure 3, only 42 (3.1% of) players fail to reject the null hypothesis of normality. In the density plots in figure 4 the distributions of total points observations are given by position and by season. Visually inspecting the distributions we can see clear clusters forming around certain point scores (e.g. 0, 2) , for all positions, through all seasons. In other words, the distribution of observed points scores appears to be multimodal. These observations lay the foundation for our decision to train our LSTM models position-wise, instead of an all-encompassing model.

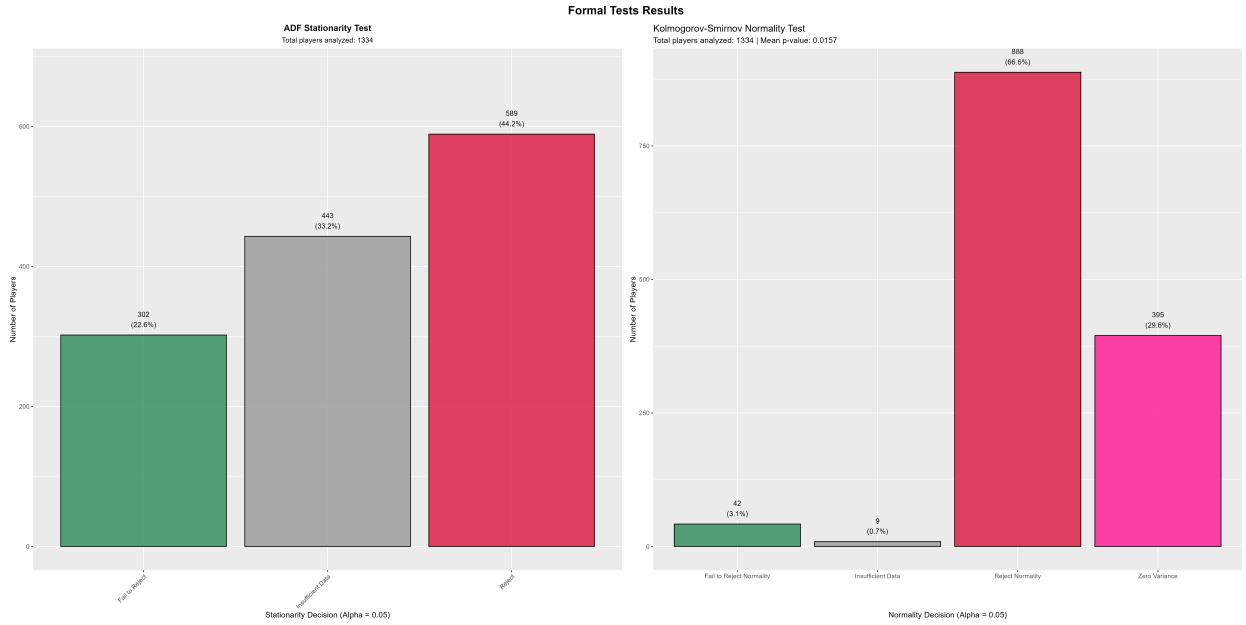
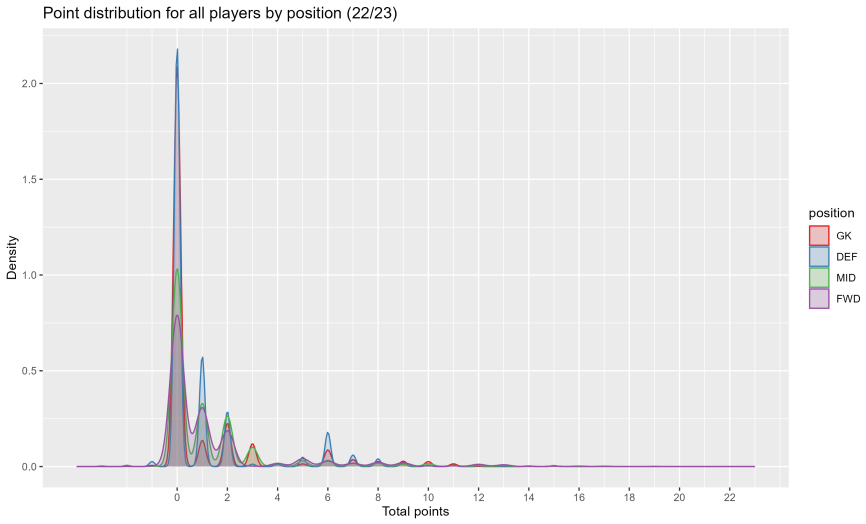
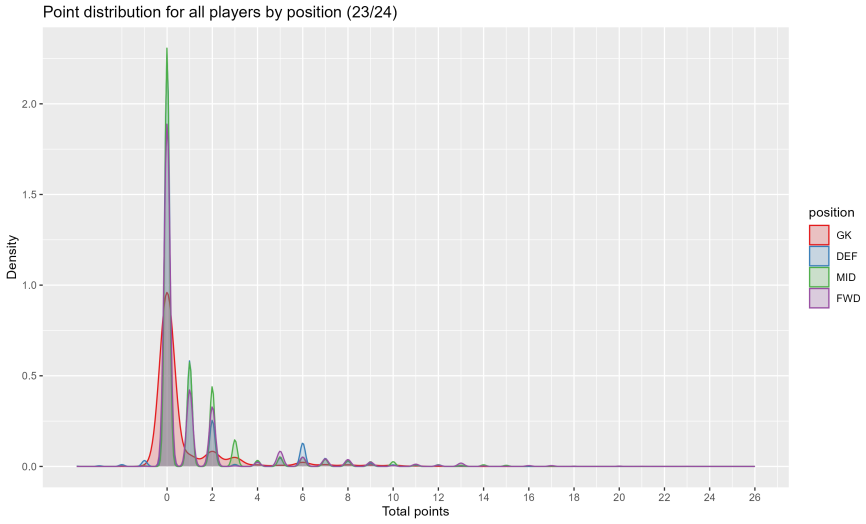


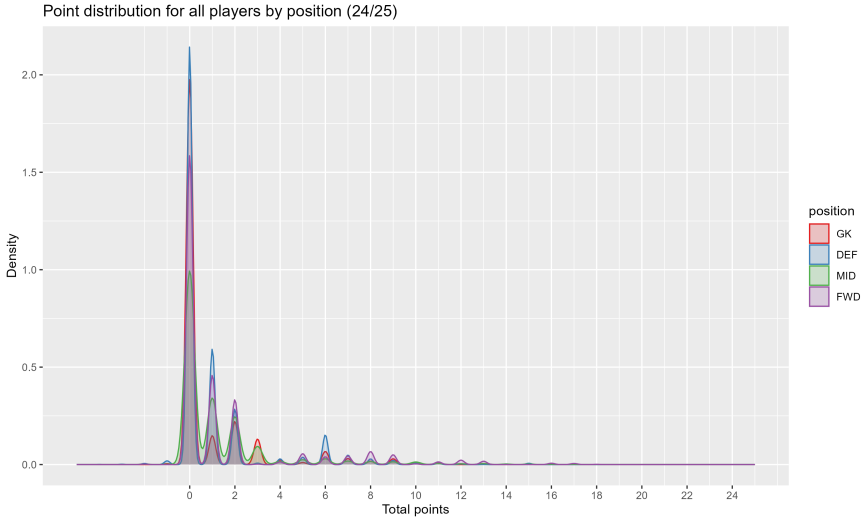

Figure 3: Player-wise ADF stationarity (left) and Kolmogorov-Smirnov normality (right) tests




(a) 2022–2023 Season




(b) 2023–2024 Season




(c) 2024–2025 Season

Figure 4: Point density distributions by player position across three seasons.

# 6 Methodology

## 6.1 Rolling window

The data has been structured in a rolling window format. The windows contain data for a chosen period $ws$, leading up to the gameweek it is predicting. Due to factors such as fatigue, injury and confidence, it is well known that player performance fluctuates throughout a season. By adopting a sequential rolling-window structure, we explicitly account for temporal dynamics, capturing recent fluctuations in player form and team performance.

Due to different timing of availability, features have been split into both numerical and categorical windows. In the numerical window, information is hidden from the model for the gameweek it is trying to predict. Specifically, when predicting $gw_t$, the numerical windows contain data up to and including $gw_{t-1}$. This reflects the real-life situation, where statistics such as goals scored are unknown for the upcoming gameweek $gw_t$.

In contrast, the categorical window allows the model to see data up to and including the target gameweek $gw_t$. This reflects the real world information human managers would have, as fixtures are known in advance. They know what teams are facing each other, what venue they are playing at and which players belong to which teams. Figure 5 serves as an illustration of what the rolling windows contain at time of predicting $gw_t$.

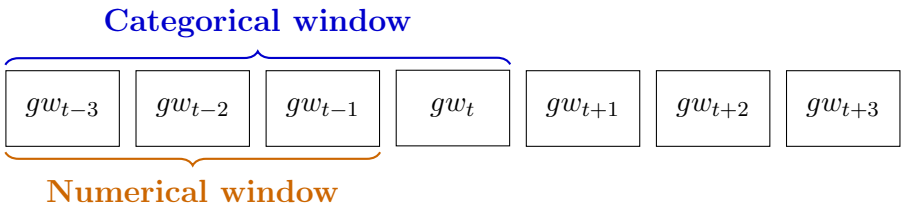



Figure 5: Rolling-window illustration.

## 6.2 Z-scaling

The FPL dataset contains a lot of features, many of which operate in completely different ranges. As per Chollet et al. (2022), "It would be problematic to feed into a neural network values that all take wildly different ranges.". There are a few reasons why this is considered problematic. Firstly, consider a feature operating in the range $[0, 1]$, and another feature operating in the range $[0, 10^5]$. When the model updates its ranges during training, the weight updates are proportional to the feature values. Thus, the $[0, 10^5]$ feature will typically dominate the model, overshooting its weight updates, while the weights of the $[0, 1]$ feature will barely budge. Another issue arising with different ranges is that the model takes longer to converge during training. Thus, centring and scaling the data improves the performance of our model while simultaneously reducing its computational demands. However, it is

important to note that not all features have been scaled. Identifiers, such as player id, an integer index with a unique value assigned to each player, were not subject to scaling, as it would have broken the indices. If $x$ denotes the input value, $\mu$ the mean of the input value, and $\sigma$ the corresponding standard deviation, the transformation is defined as in (1)

$$Z = \frac{x - \mu}{\sigma} \tag{1}$$

## 6.3 In-sample and out-of-sample

When training a neural network, one has to define in-sample (training) and out-of-sample (validation) subsets of the data. If the model generalises well to the training data, it is expected to perform well on the validation subset. Our optimisation model, which picks a team for each gameweek over the course of a season based on the outputs from our LSTM model, makes for some interesting considerations when selecting a split point for the data. First of all, a random sampling method, where the model sees for example 80% of the data during training, while utilizing the remainder for validation, was not feasible. Since we are simulating our model over the course of a complete season, such a split resulted in data leakage, as the model was exposed to data from gameweeks that were supposed to be unknown at the time. A walk-forward validation split was also considered. In such a setup, the training subset becomes increasingly larger, at the cost of the validation subset. Consider our model attempting to predict gameweek 1 of the 2024/2025 season. Then, the training set would be all gameweeks of the preceding seasons, while the validation set would be gameweek 1 onwards. When it is time to predict gameweek 2, the training set has been updated, now containing all gameweeks of the preceeding seasons plus gameweek 1 of the 2024/2025 season, while the updated validation set is from gameweek 2 onwards. Ideally, we would use this iterative setup as it closely represents the sequential nature of the game (resembling (Matthews et al., 2012)). However, the method is computationally expensive and unfeasible with the available resources. In this thesis, seasons 2022/2023 and 2023/2024 have been used as the training set, while the current season, 2024/2025, has been used as the validation set, meaning there is no true unseen set.

## 6.4 Lasso regression

In addition to the features taking on different ranges, some of them are also highly correlated with each other. Features such as expected goals, goals and shots often move in tandem. The Lasso regression is performed to counteract this aspect of the data. In short, the penalty term $\ell_1$ encourages the model to keep only one variable from each highly correlated group, shrinking the remaining coefficient estimates towards zero. Any coefficients that reach

exactly zero are effectively removed from the model, yielding a sparser model that retains only the most informative features. By comparison, Ordinary Least Squares (OLS) would assign unstable and non-unique coefficients to collinear predictors.

After careful consideration, we decided to perform Lasso regression on only the numerical windows. Since the categorical windows consist of identifiers, we want the model to capture their correlations with the numerical features, and have spared them from the feature reduction. To perform the Lasso regression, we used the R package `glmnet`.

With the dataset being constructed in a rolling window format, where each observation is a matrix of dimension (`window size`, `n features`), transformation was needed to fit the input requirement of a single 2D matrix. Each (`ws`, `numF`) window is reduced to a `numF`-dimensional vector by computing column means, thereby summarizing the average behaviour of each feature over the window. Thereafter, all vectors were aggregated into a final matrix, satisfying the requirement. After fitting, we map the nonzero coefficients back to their original features in each (`ws`, `numF`) window, effectively pruning the unselected variables from subsequent analyses. The mathematical presentation of the Lasso regression operation has been sourced from James et al. (2021).

If we by $y_i$ denote the centered and scaled response variable for observation $i$, and let $x_{ij}$ denote the standardized value of predictor $j$. Then, the Lasso coefficients, denoted as $\hat{\beta}_\lambda^L$, minimize

$$\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=1}^{p}|\beta_j| = \text{RSS} + \lambda\sum_{j=1}^{p}|\beta_j| \tag{2}$$

The right hand side of the equation is the Lasso $\ell_1$ penalty term, where $\lambda$ is a non-negative tuning parameter. For coefficient estimates to be exactly zero, $\lambda$ needs to be sufficiently large, as per James et al. (2021). However, as $\lambda \to \infty$, all coefficient estimates will be exactly zero, and thus, selecting a good $\lambda$ is critical.

## 6.5 K-fold cross validation

To ensure a principled choice of the regularization parameter $\lambda$, K-fold cross validation is employed. This technique tests out-of-sample prediction error. Firstly, the dataset is randomly partitioned into equally sized subsets (folds). Thereafter, $k-1$ folds are used as training data, with the remaining fold serving as the validation set. The process is iterated $k$ times to ensure that each subset is utilized for both training and testing. For each iteration, the Mean Square Error (MSE) is computed on the validation fold. Averaging these errors yields the cross-validation error for each $\lambda$, and the value that minimizes it is chosen.

## 6.6 Recurrent Neural Networks Overview

To solve the problem of forecasting player performance, this thesis employs a form of recurrent neural networks (RNN's). The technique is in the category of supervised machine learning, inspired by the functioning of the human brain. The similarities are drawn from the layers of neurons, which form the structure of the neural network. Typically, RNN's consist of an input layer, at least one hidden layer, and an output layer. Data is fed through the neurons in the input layer, where each feature is represented by one neuron. The data is then passed on to the neurons in the hidden layer, where each neuron computes a weighted sum of its inputs. The current value of a neuron is given by (3)

$$x_j = f\left(\sum_{i=1}^{n} w_{ji} x_i\right) \tag{3}$$

(Bishop & Bishop, 2024, ch.1, p.17) $w_{ij}$ denote the input weights, while $x_i$ represent the input from each neuron in the preceding layer. The weights decide how strongly each neuron in the hidden layer fires its signal to the output layer, which combines all signals and gives you a prediction.

Before the values are fed forward to the output layer, though, they are subjected to a non-linear activation function. The function allows the model to map non-linear patterns in the data. In this thesis, we use the popular activation function Rectified Linear Unit (ReLU), defined as

$$f(x) = \max(0, x) \tag{4}$$

(James et al., 2021). The ReLU allows positively valued neurons to pass through, while negative neurons are effectively set to zero, i.e. inactive. As a consequence, the ReLU advocates for a sparser network, yielding faster convergence for the model during training. Because our forecasting problem is one belonging to regression, ReLU is not utilised for the last output neuron, here it is exchanged for a linear activation function. Once the outputs are computed in the final layer, the predictions of the model are compared to the actual realizations. To quantify the performance of the model, Mean Squared Error (MSE) is computed. If $t_k^p$ represents the target value for output neuron $k$, $x_j^p$ the predicted value, $O$ the training set for the output layer, then the MSE for training pattern $p$ is given by

$$E_p(w) = \frac{1}{2} \sum_{k \in O} \left(t_k^p - x_j^p\right)^2 \tag{5}$$

The MSE serves two purposes. Firstly, it gives a meaningful metric to evaluate model performance. Secondly, the model actively incorporates MSE during training for weight readjustment. While the initial weights are chosen randomly, the model readjusts them with

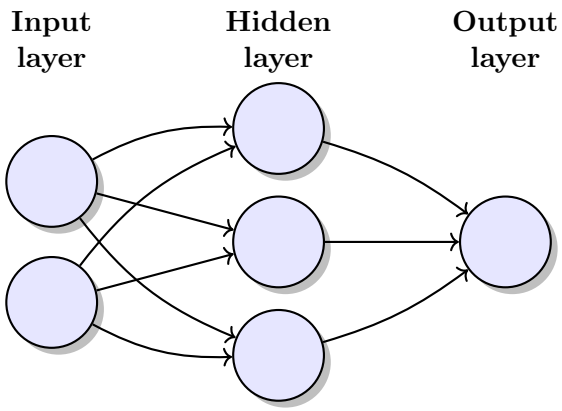



Figure 6: Example neural network architecture

the sole purpose of reducing the MSE.

Figure 2 illustrates the structure of a very simple neural network, containing an input layer with two neurons, a hidden layer with three neurons and an output layer with one neuron.

The core component of recurrent neural networks, however, is their hidden state $h_t$. It is computed on the current input $x_t$, and its previous hidden state $h_{t-1}$ by applying the weight matrices $W_x$ and $W_h$ respectively. The mathematical representation has been sourced from Dixon et al. (2020)

$$h_t = f(W_x x_t + W_h h_{t-1} + b) \tag{6}$$

The hidden state should in principle enable the RNN to store past inputs to generate the currently desired output, and have thus been widely used in sequential forecasting. However, when the temporal dependencies are separated by a sufficient number of timesteps, convergence slows down drastically. Hochreiter (1998) found the reason to be vanishing/exploding error backflow during backpropagation. That is, recent inputs will dominate non-recent inputs. Furthermore, it is stated that this will be the case no matter the learning rate of the neural network.

## 6.7 Long Short-Term Memory

On the basis of the vanishing error backflow problem, Hochreiter and Schmidhuber (1997) proposed a new, more sophisticated class of recurrent neural networks. The new technique, named Long Short-Term Memory (LSTM), ensures continuous and constant error backflow. Similarly to simpler RNN's, the LSTM also contains a hidden state $h_t$. Though, it is important to note that the computation of the hidden state $h_t$ in the LSTM differs from the hidden state $h_t$ in conventional RNN's

$$h_t = \hat{r}_t \circ tanh(c)_t \tag{7}$$

It is simply a Hadamard product of the reset gate $\hat{r}_t$, and the internal memory $c_t$, which has been squashed to a value in the range $[0, 1]$ by the tanh activation function. If the reset gate $\hat{r}_t = 0$, the memory is reset, while if $\hat{r}_t = 1$, the hidden state $h_t$ is determined solely by the memory cell.

In addition to the hidden state $h_t$, the LSTM also has a separate memory cell $C_t$. The memory cell $C_t$ is computed on a forget gate $\hat{\alpha}_t$, an input gate $\hat{z}_t$, as well as a cell gate $\hat{c}_t$ Dixon et al. (2020)

$$c_t = \hat{\alpha}_t \circ c_{t-1} + \hat{z}_t \circ \hat{c}_t. \tag{8}$$

If $U_r$, $U_\alpha$, $U_z$, and $U_c$ denote the weight matrices applied to the previous hidden state $h_{t-1}$, and $W_r$, $W_\alpha$, $W_z$, and $W_c$ represent the weights applied to the current input $x_t$, the gates of the LSTM cell are updated as shown in the equations below. Each gate also includes a corresponding bias term. The gate computations are defined as follows:

$$\text{Reset gate:} \quad \hat{r}_t = \sigma(U_r h_{t-1} + W_r x_t + b_r) \tag{9}$$

$$\text{Forget gate:} \quad \hat{\alpha}_t = \sigma(U_\alpha h_{t-1} + W_\alpha x_t + b_\alpha) \tag{10}$$

$$\text{Input gate:} \quad \hat{z}_t = \sigma(U_z h_{t-1} + W_z x_t + b_z) \tag{11}$$

$$\text{Cell memory gate:} \quad \hat{c}_t = \tanh(U_c h_{t-1} + W_c x_t + b_c) \tag{12}$$

The gate mechanics of LSTM allows it to directly control which information is forgotten, which is retained, and which is exposed through its gates. This allows it to learn both short-term and long-term dependencies in sequential data. We find this to be of critical importance in forecasting player performance, where underlying factors such as form, fatigue, injury etc. are constantly evolving, but not explicitly captured by the input features.

**Keras: Training Callback – Early Stopping** [1] To prevent the model from over-fitting, we use the `callback_early_stopping` function from Keras. This callback monitors the validation loss — in our case, the mean squared error (MSE) — and halts training if no improvement is observed after a specified number of consecutive epochs.

**Keras: Training Callback – Model Checkpoint** The `callback_model_checkpoint` saves the model weights whenever the validation MSE reaches a new minimum. This ensures

[1] Keras for R "is a high-level neural networks API" (Chollet, Allaire, et al., 2017). It is the layer which interfaces with Python and Tensorflow (Martín Abadi et al., 2015). It is the package which facilitates for producing advanced neural networks in the thesis.

that the best-performing model is preserved, even if performance deteriorates in later epochs due to over-fitting.

**Keras: Training Callback – Reduce Learning Rate on Plateau** To prevent stagnation during training, we apply the `callback_reduce_lr_on_plateau`. If the validation MSE fails to improve over a fixed number of epochs, the learning rate is reduced. This allows the optimiser to make finer weight adjustments and potentially converge more effectively toward a local or global minimum.

## 6.8 Optimisation Method

As in Sargent and Bedford (2010) each real-life player in the FPL obtains in-game points based on several metrics decided by the game-makers. Their total points score for any given match is the sum of a simple equation (13)

$$\text{Match points} = \text{Points Goals} + \text{Points Assists} + \cdots + \text{Bonus Points} \tag{13}$$

Owing to the authors not coming from an operational research background, we attempt to replicate (source code not provided) the extensive Fantasy Premier League Mixed Integer Linear Programming (MILP) model created by Eilertsen et al. (2018, Ch. 4-6) from their thesis *Developing a Forecast-Based Optimisation Model for Fantasy Premier League*. Chapters 4-6 of their study incorporate all the elements and formalisations of the Fantasy Premier League game and rules, less the Assistant Managers Chips from this season. Their detailed and explicit mathematical presentation for FPL specific mechanics makes it a preferred choice to reproduce over alternatives such as Venter and Van Vuuren (2024)) or a similar optimisation algorithm like that of Bonomo et al. (2014).

The squad optimisation model is theoretical in that, solving it is computationally infeasible, since it is stochastic and relies on realisations of match events in future gameweeks. The program maximises the expected point of a squad over a given number of gameweeks and is given in equation 14

$$\begin{aligned}\max z = \mathbb{E}\Bigg[\sum_{\substack{p\in P\\ t\in T}}\Big(&\rho_{pt}(\omega_t)y_{pt} + \rho_{pt}(\omega_t)f_{\omega_t} + \epsilon\rho_{pt}(\omega_t)h_{pt} + \\ &\sum_{l\in L} k_l\rho_{pt}(\omega_t)g_{ptl}) + 2\rho_{pt}(\omega_t)c_{pt}\Big)\Bigg] - R\sum_{t\in T}\alpha_t\end{aligned} \tag{14}$$

Where the first term $\rho_p(\omega_t)y_{pt}$ represents points for player p for random realisation $\omega_t$ in gameweek $t$. The second term decides captaincy, the third term vice-captaincy, the fourth

term substitution priority, fifth term if activated calculates the triple captain gamechip. The keen eye will notice that $\rho_p(\omega_t)$ repeats in all terms except the last, and the difference are the binary variables $y_{pt}, f_{wt}, h_{pt}, g_{ptl}, c_{pt}$ and the constants $\epsilon, k_l$, they are described in table 3 and all constraints are available in appendix C. The last term $R\sum_{t\in T}\alpha_t$ represents the transfer penalty term, currently the FPL game penalises squad transfers at negative R equal to 4 points per transfer, and the alpha term tracks the free transfer balance at time t. One small thing to keep in mind for later, are these small constants $\epsilon$ and $k_l$ which help model the vice captaincy and substitution priority, these will add tiny fractions of additional points to the objective function, which can add up over time. If the players of a team has played multiple matches in one gameweek (called double gameweeks), the scores from the individual matches are summed together for that specific week.

The practical application of the mixed integer linear program is found in Eilertsen et al. (2018, Ch. 5-6). Relying instead on forecast we remove the unknown omega for our estimated rho's. This is presented in equation(15)(Eilertsen et al., 2018, eq.5.1)

$$\textbf{max} \quad z = \sum_{\substack{p\in P\\ t\in T}} \left(\hat{\rho}_{pt}y_{pt} + \hat{\rho}_{pt}f_{pt} + \epsilon\hat{\rho}_{pt}h_{pt} + \sum_{l\in L} k_l\hat{\rho}_{pt}g_{ptl} + 2\hat{\rho}_{pt}c_{pt}\right) - R\sum_{t\in T}\alpha_t \tag{15}$$

We use the open-source solver COIN-OR branch and cut Mixed Integer Linear Programming solver (Forrest et al., 2024) through the `PuLP` interface in Python.

As in Bonomo et al. (2014) the optimisation model itself is not predictive. It can either be used after a season has been recorded or before with estimates of player performance. Both papers aims to improve model myopic-ness, in Eilertsen et al. (2018) and in our application of the model this is done by adding the option to solve for sub-horizons within the forecasting period.

## 6.9 Digital Tools

In our thesis, we have leveraged the following digital tools to enhance efficiency and collaboration: Git, GitHub, Microsoft(MS) OneDrive, MS Excel, Overleaf, R(Studio/VS Code), Python and GitHub Copilot Pro/OpenAI ChatGPT/Google Gemini Pro 2.5 for programming(GitHub, 2025; Google, 2025; OpenAI, 2025), assistance within the Overleaf TeX typesetting environment, and for linguistic polishing. For the LSTM models, we used Keras in R (Chollet, Allaire, et al., 2017) with TensorFlow as the back-end, the training and inference of the model training and squad optimisation is mainly run on a laptop ThinkPad E14 G5 AMD Ryzen 7530U with 16gb.

All scripts and datasets made by Molberg and Bøifot (2025a, 2025b) in the thesis are available on a GitHub repository and the LSTM script on Kaggle as Jupyter Notebooks.

| Sets | Description | |
|---|---|---|
| $\mathcal{T}$ | Set of premier league gameweeks | |
| $\mathcal{P}$ | Set of all fotball players | |
| $\mathcal{C}$ | Set of premier league teams | |
| $\mathcal{L}$ | Squad substitution priorities, where 1 is first priority | |
| **Subsets** | | **Parent set** |
| $\mathcal{P}^{GK}$ | Subset of goalkeepers | $\mathcal{P}$ |
| $\mathcal{P}^{DEF}$ | Subset of defenders | $\mathcal{P}$ |
| $\mathcal{P}^{MID}$ | Subset of midfielders | $\mathcal{P}$ |
| $\mathcal{P}^{FWD}$ | Subset of forwards | $\mathcal{P}$ |
| $\mathcal{P}_c$ | Fotball players in team c | $\mathcal{P}, \mathcal{C}$ |
| $\mathcal{T}_{FH}$ | Subset of gameweeks $\leq Season/2$ | $\mathcal{T}$ |
| $\mathcal{T}_{SH}$ | Subset of gameweeks $> Season/2$ | $\mathcal{T}$ |
| **Index** | | |
| $p$ | Players | |
| $t$ | Gameweeks | |
| $l$ | Substitution priority | |
| **Variables** | | |
| $y_{pt}$ | 1 if a player p is picked for starting line-up in a gameweek t, 0 otherwise. | |
| $f_{pt}$ | 1 if a player p is picked as captain in a gameweek t, 0 otherwise | |
| $h_{pt}$ | 1 if a player p is picked as vice-captain in a gameweek t, 0 otherwise | |
| $g_{ptl}$ | 1 if a player p is picked as substitution priority l in a gameweek t, 0 otherwise | |
| $c_{pt}$ | 1 if the Triple Captain is used on a player p in a gameweek t, 0 otherwise | |
| $\alpha_t$ | Number of penalized transfers in a gameweek t | $\alpha_t \in \mathbb{Z}^+$ |

Table 3: Sets, indices and variables

# 7 Results

This chapter presents the empirical findings resulting from the application of our methodology. The structure follows the three main components of our approach: feature selection, the architecture and performance of the LSTM model, and the final results from the squad optimisation using the MILP. The results of our thesis are not entirely reproducible. The code in the appendix H and notebook on Kaggle yields slightly different results for each run, despite setting a seed in both overlay(R) and underlay(Python).

## 7.1 Feature Selection

The initial numerical feature set is presented in the first column of table 4. On the right, the table shows the reduced feature set, post lasso regression, partitioned by on-field position. Notice the feature set have some variations for GK compared to outfield positions. Represented as follows; if exclusive to GK append (GK), or exclusive to out-field positions append ($\neg GK$), otherwise the variable is non-exclusive. The lasso regression resulted in a reduction in all feature sets. The initial size of the numerical feature set for GK were 25 and for out-field players 24. Post reduction, 14 predictors remained for GK, 12 for DEF, 16

for MID and 11 for FWD. The variables Minutes, BPS, Value, and Selected remained in all reduced feature sets. On the other hand, ICT Index, Influence, and Saves was pruned from all sets regardless of position. Furthermore, Assists was chosen for all positions excluding DEF, while Creativity was kept for all out-field positions. Expected Goals Conceded and Bonus was kept for all positions excluding FWD. Expected Assists remained for GK and MID. Additionally, Expected Goal Involvement stayed relevant exclusively for DEF, and Expected Goals exclusive to MID. Clean Sheets was kept as a predictor for GK and MID. Own goals was kept for GK and FWD, and the GK exclusive variable Penalties saved was carried forward. Red cards was passed on to the reduced set for all out-field positions and the threat variable in all but forwards. The transfer in variables for DEF and FWD, whilst Transfers out excluded DEF, the net Transfer Balance variable was included for GK and DEF. Lastly, Yellow cards was retained for FWD only.

| **Input Variables** | **GK** | **DEF** | **MID** | **FWD** |
|---|---|---|---|---|
| Goals Scored ($\neg GK$) | | | G.Scored | |
| Assists | Assists | | Assists | Assists |
| Creativity | | Creativity | Creativity | Creativity |
| Minutes | Minutes | Minutes | Minutes | Minutes |
| xGoals Conceded(xGC) | xGC | xGC | xGC | |
| Saves (GK) | – | – | – | – |
| Bonus | Bonus | Bonus | Bonus | |
| BPS | BPS | BPS | BPS | BPS |
| xAssist | xA | | xA | |
| xGoal Involvement | | xGI | | |
| xGoals | | | xG | |
| Ict Index – | – | – | – | – |
| Influence | – | – | – | – |
| Clean Sheets | Clean Sheets | | Clean Sheets | |
| Own Goals | Own Goals | | | Own Goals |
| Penalties Saved (GK) | Pen. Saved | | | |
| Red Cards | | Red Cards | Red Cards | Red Cards |
| Threat | Threat | Threat | Threat | |
| Transfers In | | Transfers In | | Transfers In |
| Transfers Out | Transfers Out | | Transfers Out | Transfers Out |
| Value | Value | Value | Value | Value |
| Selected | Selected | Selected | Selected | Selected |
| Starts | | | Starts | |
| Transfers Balance | T. Balance | T. Balance | | |
| Yellow Cards | | | | Yellow Cards |
| **Sum Variables** | | | | |
| 25 for GK, 24 for ($\neg$GK) | 14 | 12 | 16 | 11 |

Table 4: Numerical input variables by player position prior to and after Lasso feature selection process

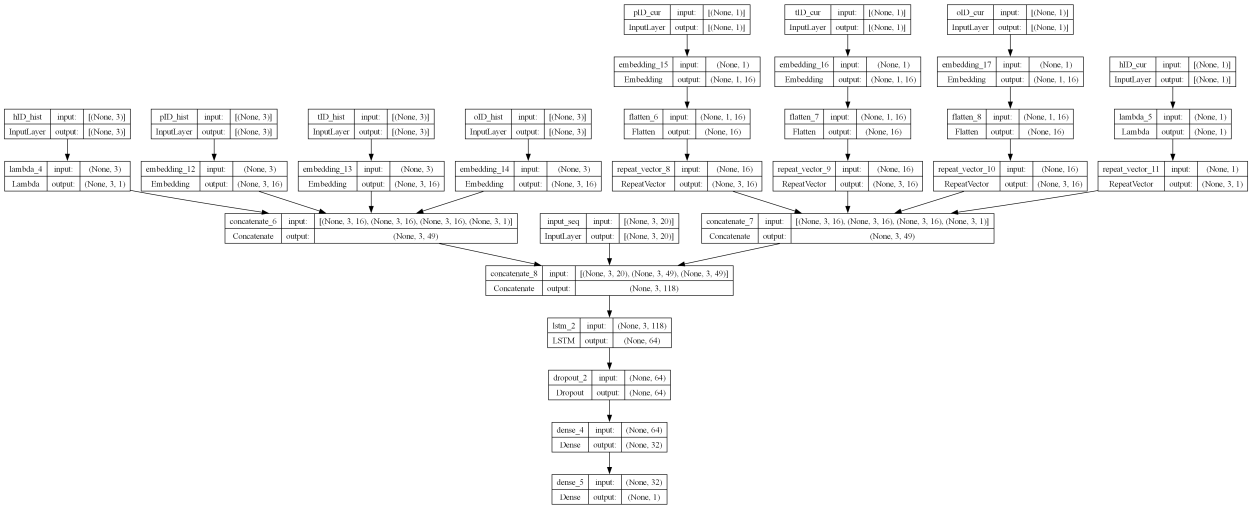



Figure 7: Keras Model for MID

## 7.2 Keras LSTM Model structure

Figure 7 plots the structure of our LSTM model for MID. Since the rolling windows for categorical and numerical inputs are not of equal dimensions, categorical inputs concerning the upcoming gameweek $gw_t$ has been extracted and fed in separately. The features for the upcoming gameweek $gw_t$ can be seen in the top right corner of the model plot, namely $pID_{\text{cur}}$ (player ID), $tID_{\text{cur}}$ (team ID), $oID_{\text{cur}}$ (opponent ID) and $hID_{\text{cur}}$ (home/away ID). With the home/away ID being a binary variable, it did not have to go through the same embedding layers as the others, which are integer indices.

In the top left corner, the remainder of the categorical rolling windows are fed through in similar fashion. The numerical windows did not have to go through the same embedding process, and can be seen in the middle of the plot. All inputs were merged into a tensor compatible with `keras`, and fed through an LSTM-layer.

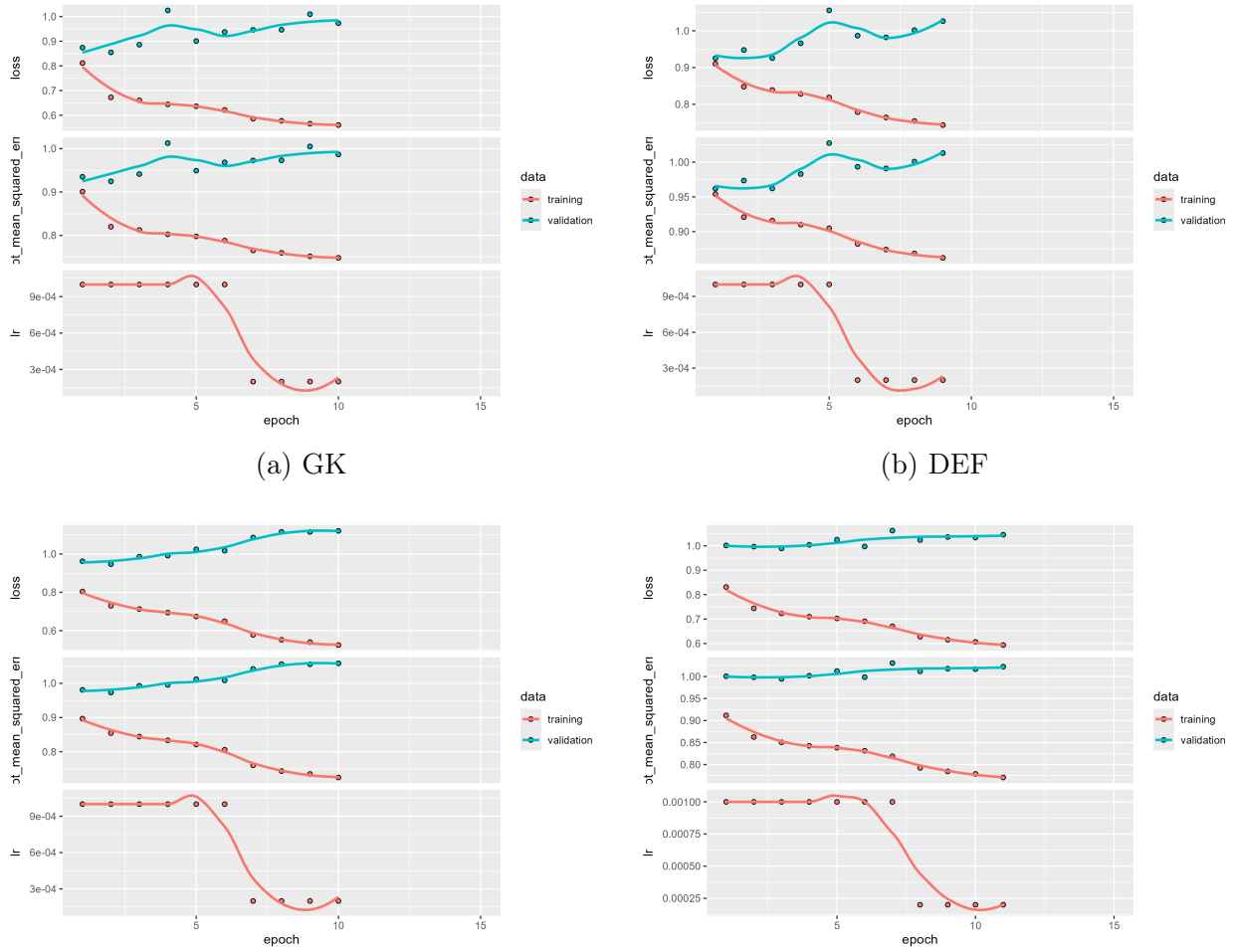
The model structure is identical for all positions, with the input data being the differentiator. Nonetheless, model plots for the remaining positions can be found in the appendix (B.6).

## 7.3 LSTM Model performance

### 7.3.1 Training and validation

Figure 8 exhibits the training history of the LSTM-models for GK (8a), DEF (8b), MID (8c) and FWD (8d). The callback checkpoint with the lowest MSE was found in the second epoch for GK and MID, in the first epoch for DEF and in the third epoch for FWD. For all positions, the model struggles to improve its performance on the validation set while continuously improving on the training data, as measured by MSE (loss) and RMSE. All LSTM models had a limit of fifteen training epochs. Even so, all stopped early by the callback

function - Early Stopping. The callback was activated after not seeing an improvement in the loss function for 8 consecutive epochs, i.e. the integer rounded half of the maximum number of training epochs. The learning rate (lr) remained constant until the sixth training epoch for the GK and MID models, fifth epoch for the DEF model, and seventh epoch for the FWD model. For the subsequent epochs, the learning rate was reduced by a factor of 20 percent, where it remained throughout. The callback for learning rate reduction activated after 4 epochs of no improvement to the loss function (Floor value of half the patience for early stopping). Accompanying the training plots is table 5, exhibiting the lowest values obtained from training on scaled values, and the subsequent rescaling of these values. For the z-scaled validation errors the MSE(RMSE) was 0.85(0.92) for GK, 0.93(0.96) for DEF, 0.95(0.97) for MID, and 0.99(0.99) for FWD. Further, the rescaled validation errors all have negative residual means, in the same order these were -0.33, -0.22, -0.38, -0.08. The Rescaled MSE(RMSE) was 3.63(1.91) for GK, 4.45(2.11) for DEF, 5.54(2.35) for MID, and 6.95(2.64) for FWD. The overall rescaled residual mean is -0.29, with the overall rescaled validation mean squared error being 5.13 and, the overall root mean squared error at 2.27 total points.




(a) GK

(b) DEF

(c) MID

(d) FWD

Figure 8: LSTM training and validation history for different positions: (a) GK, (b) DEF, (c) MID, and (d) FWD. Computed on z-scaled values.

| Metric | GK | DEF | MID | FWD |
|---|---|---|---|---|
| **Validation errors on z-scaled values** | | | | |
| Loss Function (MSE) | 0.85 | 0.93 | 0.95 | 0.99 |
| Metric RMSE | 0.92 | 0.96 | 0.97 | 0.99 |
| **Validation Errors on rescaled values** | | | | |
| Residual Mean | -0.33 | -0.22 | -0.38 | -0.08 |
| Rescaled MSE | 3.63 | 4.45 | 5.54 | 6.95 |
| Rescaled RMSE | 1.91 | 2.11 | 2.35 | 2.64 |
| **Overall Validation Residual Mean** | | -0.29 | | |
| **Overall Validation MSE** | | 5.13 | | |
| **Overall Validation RMSE** | | 2.27 | | |

Table 5: Training loss values (MSE) and root mean squared error (RMSE) across player positions, along with prediction errors on the validation set by position and overall average. All values are rounded to two decimal places.

Figure 9 exhibits the four positions residuals distribution plots. The distribution colours are red for GK, blue for DEF, green for MID, and purple for FWD. The red dotted vertical line indicates x equals zero. The distributions appear to have most of its mass at their mean or below zero, additionally for position GK, DEF and MID there are two pronounced peaks. For GK and MID the first around their mean (-0.33,-0.38) and a second lower peak at approximately negative 1.8 to 2.8, furthermore for DEF the lowest peak is approximately its mean value (-0.22) and with a second taller peak around negative 2.5. For all plots the densities appear to take on a rights-skewed shape.

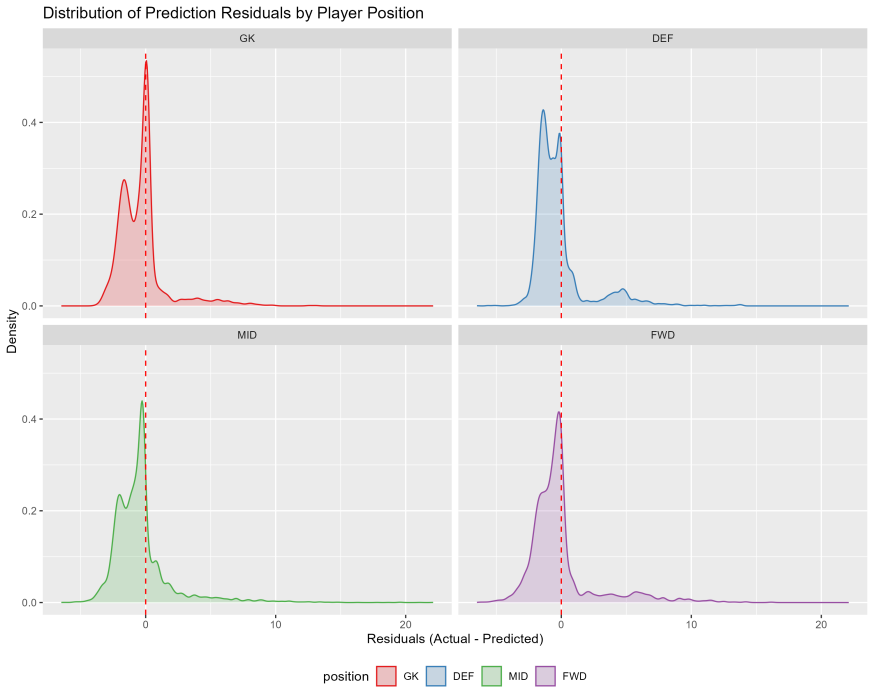



Figure 9: Distribution of the reverse scaled residuals by position

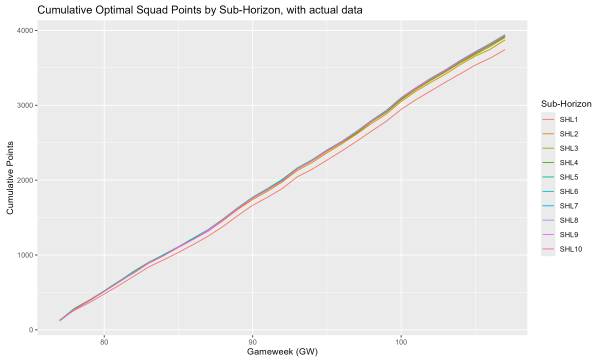
## 7.4 Squad Optimisation Results

The following results come from executing the MILP adapted from Eilertsen et al. (2018) detailed in Section 6.8, on realised (actual) historical total points and forecasted total points. Since the objective function returns decimal numbers this is rounded to integer values to conform with FPL norm. The squad optimisation results presented in this section do not include the usage of the FPL specific gamechips. Figure 10 visually presents the computations from the mixed integer linear program. The two upper sub-figures 10a and 10b are cumulative plots, and the two lower sub-figures 10c are weekly score plots. Solid plot lines indicate the use of realised total points, whilst the dotted lines indicate the use of estimated total points. In the case of the plots 10b and 10d that means the optimisation has been executed on the estimated points, which constructed a squad, then the forecasted points have been exchanged for the true total points to see the actual performance of the squad. The left side plots are optimised exclusively on realised total points. Each different line represents a squad optimisation run with a unique sub-horizon length (range 1-10). From the weekly points plots (10d,10c), the weekly forecast lines appear visually smoother than their actual counterparts. All four plots show the different "paths" for different sub-horizons lengths in the range 1 to 10 and the end values for the cumulative plots are shown in table 6. The highest obtained total score was found to be 3944 total points for the realised points called the Optimal Squad Cumulative Actual Points(Squad* C.A. Points) for sub-horizon length equal to 5 (in blue). The optimal forecasted total points squad(F*.C.Squad Points) was for sub-horizon 4 (in red) at 1644 total points, however this sub-horizon did not achieve the highest actual points (F*.C.A Squad Points), this happened for sub-horizon 8 (in green) with an actual total points of 1534. Note that because of the great distance between actual and forecasted values, the y-axes are not scaled to fit the largest observed values for either cumulative or weekly plots, to avoid understating the variance in point returns.

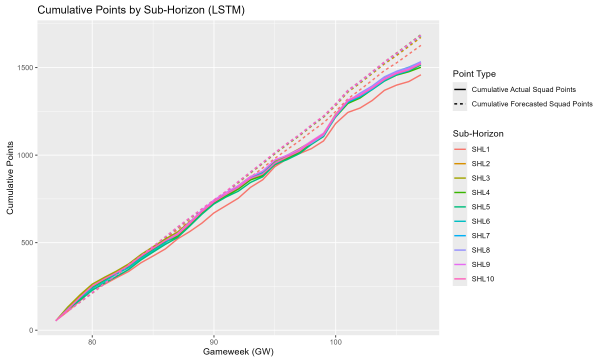
Summarising the results in a non-technical manner, four LSTM models were trained, one for each position. Overall, all four LSTM models tended to miss actual player scores by just over two points on average, slightly overestimating performance. Goalkeepers were predicted most accurately and forwards least, but all positions showed similar error levels. The highest score for the forecast-based solution was obtained when optimising over a sub-horizon length of 8, which yielded 1,534 total points. The best possible team configuration, based on realized points, was found for a sub-horizon length of 5, achieved a total point score of 3944.

| Sub horizon length | F*.C Squad Points | F*.C.A Squad Points | Squad* C.A.Points |
|---|---|---|---|
| 1 | 1627 | 1459 | 3745 |
| 2 | 1673 | 1516 | 3874 |
| 3 | 1683 | 1526 | 3907 |
| 4 | 1688 | 1502 | 3921 |
| 5 | 1685 | 1515 | 3944 |
| 6 | 1682 | 1533 | 3938 |
| 7 | 1686 | 1525 | 3936 |
| 8 | 1684 | 1534 | 3934 |
| 9 | 1686 | 1526 | 3939 |
| 10 | 1686 | 1521 | 3936 |

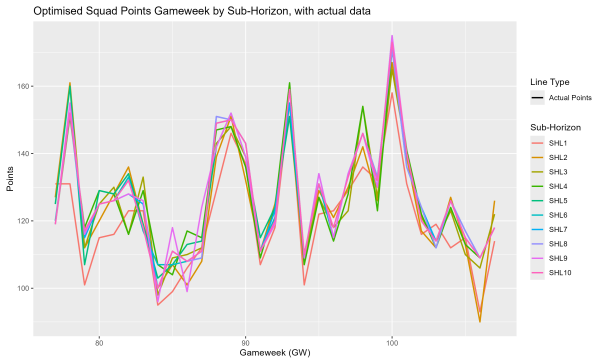
Table 6: Forecast Optimised Cumulative (F*.C) points by sub-horizon, with forecast based squad optimisation and same squad actual (A) points vs Actual/Real points based optimisation. Green indicates best value for F*.C.A, blue the best Squad* and purple the best F*.C.



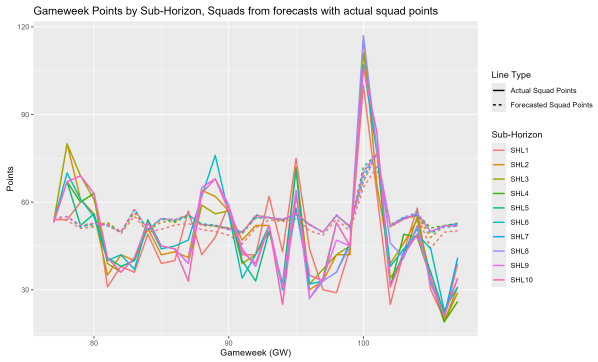

(a) Cumulative plot for optimal squad, actual points




(b) Cumulative plot for optimal squad, forecasted total points




(c) Weekly score for optimal squad with actual points




(d) Weekly score for optimal squad with forecasted points, with same squad actual points

Figure 10: Comparison of cumulative fantasy points over season 24/25(a,b) and Weekly score for optimal squad (c,d)

# 8 Discussion

The findings from the previous section (7) suggest some challenges in the suitability and application of the tools deployed to attack the thesis' research problem and dataset. Even if the initial methodological approach of analytical finance was sound. Based on the available body of literature presented in section 4, and the exploration of the FPL data in section 5.2, the complex decision making problem appears to be broken down into a two part problem, close to asset forecasting and optimisation problem, with data which are characterised by non-Gaussianity, natural noise and trend formations. However, indicated by our findings, our forecasting tool (LSTM) tends to over-fit rapidly on our training data, leading to lesser generalisation and lower performance levels than those reported in previous research with other techniques(e.g. Matthews et al. (2012)). The decision to use LSTM above other NN techniques or other ML techniques, was based on its efficacy in forecasting financial time-series and volatility, especially in "non-inherent value" financial assets like cryptocurrencies or "MEME" stocks(Jaquart et al., 2022; Nazareth & Ramana Reddy, 2023). The findings from utilising the optimisation model adapted from Eilertsen et al. (2018) also give some interesting results, some of which slightly contradict the original findings. The latter subsections will further discuss our findings individually.

## 8.1 Feature set

As presented earlier in table 4 different numerical features were drawn for different positions. The feature set was reduced to decrease multicollinearity and training time. Other variable selection methods were contemplated, such as variable importance by permutation (first introduced in Breiman (2001)). However, feature selection by permutation was deemed to be a too computationally and time-wise costly approach. Owing to the need to train a full mode for each position, and because the training is difficult to reproduce and introduces some randomness, this would have been needed to be done multiple times to create a robust permutation result. The resulting reduced feature sets for the position yielded unintuitive results, for example in FWD the usual main key performance indicators for players in a striker position are goals scored, expected assist, expected goal involvement and expected goals. Neither of which are included by the lasso regression. Interestingly, Own Goals, which is quite a rare event, was kept for FWD. This may have been an event of significant impact the few times it occurred. Highly speculative, it would appear as if some of the variables that are selected often such as Value, Transfers In/Out, Selected, and T.Balance are the ones which rely on crowd wisdom. These four variables are directly related to the lagged opinion of other managers on the future of expected "return" for specific players. The inclusion of

Minutes, BPS, Value and Selected for all positions suggest that these features are the most informative, regardless of position. In contrast, the exclusion of ICT Index and Influence for all positions suggest that these are highly correlated with other, more informative features. The differentiation highlights the varying determinants of performance across positions, and supports our rationale for feature engineering each position uniquely.

## 8.2 Data split, Callbacks and No Holdout set

In this study, the dataset is organised chronologically and split into training and validation/test sets, with no true test set (true unseen set). Since the validation set is employed to surveil and adjust the hyper-parameters under training, it is no longer truly unseen as the specific weights saved for the models come from the weights with the lowest MSE on the validation set. That is, the model reaches its most optimal generalisation, based on feedback from prediction errors of the validation set. Opposed to other machine-learning techniques like Random Forests (tree-based ensemble) LSTM networks do not rely on bagging or some other technique when training, which effectively creates a partition within the training set for validation. Recall, that when training the LSTM models, the three callback functions were employed to monitor training. The callbacks: early stopping, model checkpoint, and reduce lr on plateau all rely on the validation set. The implications for the studies results is that the prediction errors can be "overly optimistic", and in fact could perform worse on a true unseen test set. However, the decision was based on a lack of appropriate size of a FPL dataset. Since the underlying data for each player position were so different, four models were trained, further reducing the dataset. Owing to a fear of either under-fitting the models, or over-fitting on temporal trends and noise in player performance data, the choice to only make a two part chronological split was made.

## 8.3 The LSTM Models

The LSTM model appears to over-fit rapidly to the training data, as suggested by the steadily increasing validation error from the outset (see Figure 8). Upon inspection of its predictions, the model seems to exhibit a form of bias toward players who performed well during the training period. These players are frequently forecast a high performance in the validation period, even when their recent form or playing time suggests otherwise. Similarly, players who performed poorly in the training data are consistently forecast poor performance in the validation period, despite delivering stellar campaigns. One potential explanation is that the model overweights historical performance, failing to sufficiently adapt to recent changes in player status, and learning the characteristics of good player performance. This is particularly concerning given that the model receives up-to-date indicators through rolling-

window numerical features, such as recent minutes played. The failure to adjust predictions in light of such features suggests that either these inputs are not weighted appropriately, or the model struggles to interpret them meaningfully in context. Looking at the residual figure 9 and table 5, all positions have negative mean. This supports the notion of the models failing to adapt to changes in player status and the characteristics of player performance. Recall that from 5.2, that we have a lot of players in our dataset which are not actively playing. When inspecting samples of the model prediction output, we noticed that the model consistently predict point returns for non-starters. This aligns with the residual distributions and their slight negative means. That is, the models give some evidence of generally over-predicting non-playing players' total points, whilst under-predicting playing players. The choice of LSTM networks may have been suboptimal and not ideally suited to the task at hand. The primary advantage of LSTMs over standard RNN's lies in their ability to capture long-term temporal dependencies within sequential data. However, if such dependencies are absent, the added complexity of LSTMs can become a liability rather than a strength, potentially leading to over-fitting and unnecessary computational overhead. A further complication may arise from the model architecture itself. The numerical features are fed into the LSTM as a collective window, while categorical features are passed individually. This architectural distinction may cause the model to treat all numerical inputs as equally informative or dilute the influence of critical signals (such as recent playing time) within the larger input vector. Consequently, the model's weighting of input signals may be poorly calibrated, leading to overly optimistic predictions for historically strong players regardless of current performance.

It should be noted the LSTM Models have several disadvantages compared to a real player: It cannot think, it is static (pre-trained) and cannot improve over time, the forecast performance gets poorer the further out the predictions get, it only has access to FPL performance data (i.e. no expert wisdom /crowd wisdom available) and does not benefit from interacting with other players.

## 8.4 MILP

The MILP solved the optimal team for different sub-horizons. While the intuitive train of thought would suggest there is only one solution constituting the best team, finding it would demand solving for a sub-horizon of a complete season. With the available resources, this was computationally infeasible. The interesting finding is that a longer sub-horizon yields better performance with regards to point accumulation for all models. This is especially prevalent when increasing the sub-horizon from 1 to 2, and from 2 to 3.

Comparing the optimal solution 10a to figure 10b, which plots the point accumulation

of the LSTM-based model, it is clear that the room for improvement is substantial. While the best solution teams accumulate just short of 4000 points, the best LSTM-based models accumulate a meagre points total just below 1700. Furthermore, the forecasted points of the LSTM-based team is consistently and significantly above its actual score, as seen in table 6. This may be attributed to the LSTM-networks overestimation of the players in the selected team. Inspecting the team selections, it is found that injured or otherwise unavailable players are consistently picked, some even throughout the whole season. While their unavailability should be picked up by the LSTM-models through the Minutes and Red Cards variables, it seems these features are not weighted appropriately. As a consequence, the MILP is reluctant to incur penalties for non-free transfers, as the forecasted points of injured players are not affected by their unavailability. This is further amplified by the LSTM-models' underestimation of players whom recorded weak campaigns in the training data, ensuring the 4 point penalty for a non-free transfer rarely representing good value.

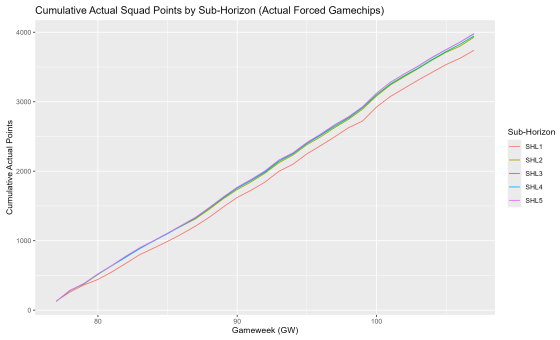
### 8.4.1 Gamechips

In our results section several plots (10), and table (6) are presented from the squad optimisation results. These do not include the deployment of FPL gamechips. The reason was to create an environment which is easily comparable to the work of other researchers and practitioners.. The thought was that squad scores would better reflect "pure" forecasting performance. However, the main reason for implementing the model of Eilertsen et al. (2018), was to be able to fully model all the quirks of FPL. Therefore, to explore this in a reproducible way, "skills" identified in O'Brien et al. (2021) are employed to "play" the chips in a consistent manner. The "skillful" FPL managers consistently planned for double gameweek events, and also more effectively repositioned their squads when fixtures became challenging for their current team. In our dataset, we find that there were two double gameweeks, specifically in round 24 and round 25. These gameweeks were targeted with regards to game chips that amplify squad total points return. Triple captain was played in round 24, while bench boost was employed in round 25. The Wildcard chips can be played once every half season, and were forced to be played on the fourth round of every half season, in gameweek 19. Usually there are a greater number of back-to-back Premier League rounds around Christmas, therefore in round 18 we allow the MILP model to use the free-hit chip. The deployment of all gamechips is exhibited in table 7.

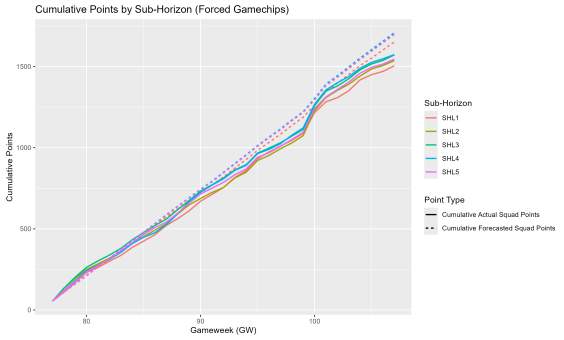
From figure 11 and table 8 we see the results of playing the forced gamechips. When running the models with a sub-horizon length of 10, the time cost was at most around 36 hours for one optimisation loop. Which was mostly due to the last 5 sub-horizon lengths. Therefore we ran the forced gamechip models with sub-horizons in range (1,5), the shortened

| Gamechips | Dataset GW (in Season 24/25) |
|---|---|
| Wildcard 1st Half Season | GW 80 (Round 4) |
| Free Hit | GW 95 (Round 18) |
| Wildcard 2st Half Season | GW 99 (Round 22) |
| Triple Captain | GW 100 (Round 24) |
| Bench Boost | GW 101 (Round 25) |

Table 7: Forced Gameweek utilisation




(a) Cumulative Actual Squad Points by Sub-Horizon with forced gamechips




(b) Cumulative Forecasted Squad Points by Sub-Horizon with forced gamechips, Actual in Solid, LSTM in dotted

Figure 11: Comparison of actual and forecasted squad points with forced gamechips

range was supported by the actual optimal squad result being achieved in this sub-horizon length. With the FPL gamechips a higher score is achieved than without the gamechips. The highest cumulative actual score based on forecasted points (F*.C.A Squad Points) was 1574 and happened with sub-horizon equal to five. Surprisingly, the highest score for actual points occured with a sub-horiozon of four, returning 3976 points. Comparatively, with gamechips, the squad returned 44 points more for the squad based on forecasted points and 42 points more for the optimal squad based.

| **Sub horizon length** | **F*.C Squad Points** | **F*.C.A Squad Points** | **Squad* C.A.Points** |
|---|---|---|---|
| 1 | 1652 | 1505 | 3740 |
| 2 | 1698 | 1537 | 3929 |
| 3 | 1707 | 1572 | 3946 |
| 4 | 1709 | 1574 | 3976 |
| 5 | 1700 | 1546 | 3982 |

Table 8: Final Forecasted Cumulative (F*.C) points by sub horizon, and the same squads actual points. With rounded results and forced Game Chips. In green is the best solution for F*.C.A Squad Points, and blue is the best actual optimal squad

Unsurprisingly, using the in-game point boosting aids, even in a fixed manner and sub-optimally for Wildcard and free-hit, yields a higher point result. More importantly, the MILP optimisation runs unexpectedly doesn't perform better after sub-horizon 5 (blue) as presented in 6. Although highly speculative, this gives some interesting insights. Firstly,

at some point the efficiency gains of long-term thinking in a highly dynamic and unpredictable environment like the FPL, yields none to minimal gains. Conversely, in the same environment, identifying high-performers are essential and in doing so, changing between different high-performers will yield very little change in output. This can have implications for other fields, like management who need to effectively choose employees. That is, the objective should become to identify high performers, not wasting resources identifying the highest high-performers, or separating between layers of performance. Looking before sub-horizon 5 in the same table, the MILP model attempts to fit in players with abnormal high performance in an attempt to maximise its objective value, this also happens afterwards at the end of the sub-horizons where they add "fringe" players to boost end of period returns, which leads to a slightly worse performance in the end.

# 9 Conclusion

This thesis set out to explore whether forecasting and optimisation techniques that have been shown to be effective tools in empirical finance could be effectively applied to the FPL. We employed LSTM-networks to predict player performance, and translated these forecasts into team selections using a MILP that adhered strictly to the rules of the FPL.

Our results indicate that the LSTM models struggled with generalisation. Despite performing well on training data, they consistently over-fit and produced weak forecasts for the validation set. The MILP, while robust in its optimisation logic, was limited by the quality of its inputs—the forecasted points. Consequently, the forecasting errors propagated through the optimisation process, leading to suboptimal squad performance relative to human managers.

Nonetheless, the study demonstrates how the FPL can serve as a meaningful proxy for financial portfolio optimisation problems, especially in the context of constrained, sequential decision-making under uncertainty. The challenges encountered underscore the importance of model calibration, data richness, and the interplay between forecasting quality and optimisation outcomes.

While the performance of our approach fell short of surpassing top human players, this thesis contributes to the literature in testing LSTM-networks for FPL-forecasts. With improved forecasting models and enriched data sources, future research may bring data-driven FPL agents closer to the top performing human players.

## 9.1 Suggestion for further research

The proposed method in Venter and Van Vuuren (2024) of creating hybrid-models (referred to as ensembles) seems to be a reasonable approach, however, this has be done. We propose that a next look at research, given the appropriate amount of data, should attempt to integrate additional sources of data so as to most closely replicate the information available to real human players as possible. For example, features regarding player availability could have significantly enhanced our model, but were not present in the data at hand. With the ever-increasing noise and engagement surrounding FPL, usage of crowd wisdom through social media could be used to improve forecasts.

# References


Anand, V. (2025). FPL Historical Dataset.

Bishop, C. M., & Bishop, H. (2024). *Deep learning: Foundations and concepts*. Springer. https://doi.org/10.1007/978-3-031-45468-4

Bonomo, F., Durán, G., & Marenco, J. (2014). Mathematical programming as a tool for virtual soccer coaches: A case study of a fantasy sport game. *International Transactions in Operational Research*, *21*(3), 399–414. https://doi.org/10.1111/itor.12068

Breiman, L. (2001). Random Forests. *Machine Learning*, *45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Chollet, F., Allaire, J., et al. (2017). R interface to keras.

Chollet, F., Kalinowski, T., & Allaire, J. J. (2022). *Deep learning with R* (Second edition). Manning.

Deloitte. (2024). Annual review of football finance 2024 [Accessed: 3 April 2025].

Dixon, M. F., Halperin, I., & Bilokon, P. (2020). *Machine Learning in Finance: From Theory to Practice*. Springer International Publishing. https://doi.org/10.1007/978-3-030-41068-1

Eilertsen, W., Gupta, A., & Kristiansen, B. K. (2018). *Developing a Forecast-Based Optimization Model for Fantasy Premier League* [Master's thesis, NTNU]. Retrieved February 3, 2025, from https://ntnuopen.ntnu.no/ntnu-xmlui/handle/11250/2577003

Forrest, J., Ralphs, T., Vigerske, S., Santos, H. G., Hafer, L., Kristjansson, B., jpfasano, EdwinStraver, Jan-Willem, Lubin, M., rlougee, a-andre, jpgoncal1, Brito, S., h-i-gassmann, Cristina, Saltzman, M., tosttost, Pitrus, B., ... to-st. (2024, August). Coin-or/Cbc: Release releases/2.10.12. https://doi.org/10.5281/ZENODO.2720283

GitHub. (2025). Github copilot pro [(Pro version) [AI pair programmer].].

Google. (2025). Gemini [(2.5 Pro/Flash version) [Large language model].].

Hochreiter, S. (1998). The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, *06*(02), 107–116. https://doi.org/10.1142/S0218488598000094

Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, *9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning: With Applications in R*. Springer US. https://doi.org/10.1007/978-1-0716-1418-1

Jaquart, P., Köpke, S., & Weinhardt, C. (2022). Machine learning for cryptocurrency market prediction and trading. *The Journal of Finance and Data Science*, *8*, 331–352. https://doi.org/10.1016/j.jfds.2022.12.001

Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Jia, Y., Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, ... Xiaoqiang Zheng. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems [Software available from tensorflow.org]. https://www.tensorflow.org/

Matthews, T., Ramchurn, S., & Chalkiadakis, G. (2012). Competing with Humans at Fantasy Football: Team Formation in Large Partially-Observable Domains [Number: 1]. *Proceedings of the AAAI Conference on Artificial Intelligence*, *26*(1), 1394–1400. https://doi.org/10.1609/aaai.v26i1.8259

Molberg, P. A., & Bøifot, E. A. (2025a). Msc-busadm-25-eba-pam. www.github.com/Nysgjerrigper/MSc-BusAdm-25-EBA-PAM

Molberg, P. A., & Bøifot, E. A. (2025b). Msc-busadm-25-eba-pam-kaggle. www.kaggle.com/code/peramolb/msc-busadm-25-eba-pam-kaggle

Nazareth, N., & Ramana Reddy, Y. V. (2023). Financial applications of machine learning: A literature review. *Expert Systems with Applications*, *219*, 119640. https://doi.org/10.1016/j.eswa.2023.119640

O'Brien, J. D., Gleeson, J. P., & O'Sullivan, D. J. P. (2021). Identification of skill in an online game: The case of Fantasy Premier League [Publisher: Public Library of Science]. *PLOS ONE*, *16*(3), e0246698. https://doi.org/10.1371/journal.pone.0246698

OpenAI. (2025). Chatgpt [(May 15th version) [Large language model].].

Premier League. (2023, July). How the ICT Index in Fantasy works. Retrieved May 6, 2025, from https://www.premierleague.com/news/65567

Sargent, J., & Bedford, A. (2010). Improving Australian Football League player performance forecasts using optimized nonlinear smoothing. *International Journal of Forecasting*, *26*(3), 489–497. https://doi.org/10.1016/j.ijforecast.2009.10.003

Venter, V. Z., & Van Vuuren, J. H. (2024). An optimisation approach towards soccer Fantasy Premiere League team selection. *ORION*, *40*(1). https://doi.org/10.5784/40-1-753

Wilkins, L. (2024). A bibliometric analysis of fantasy sports research. *Entertainment Computing*, *48*, 100613. https://doi.org/10.1016/j.entcom.2023.100613

# Appendices

## A Points System

Table 9: Point System

| Action | Points |
|---|---|
| Playing up to 60 minutes | 1 |
| For playing 60 minutes or more (excluding stoppage time) | 2 |
| For each goal scored by a goalkeeper | 10 |
| For each goal scored by a defender | 6 |
| For each goal scored by a midfielder | 5 |
| For each goal scored by a forward | 4 |
| For each goal assist | 3 |
| For a clean sheet by a goalkeeper or defender | 4 |
| For a clean sheet by a midfielder | 1 |
| For every 3 shot saves by a goalkeeper | 1 |
| For each penalty save | 5 |
| For each penalty miss | -2 |
| Bonus points for the best players in a match | 1-3 |
| For every 2 goals conceded by a goalkeeper or defender | -1 |
| For each yellow card | -1 |
| For each red card | -3 |
| For each own goal | -2 |

| Action | Bonus Points System |
|---|---|
| Playing 1 to 60 minutes | 3 |
| Playing over 60 minutes | 6 |
| Goalkeepers and defenders scoring a goal | 12 |
| Midfielders scoring a goal | 18 |
| Forwards scoring a goal | 24 |
| Assists | 9 |
| Goalkeepers and defenders keeping a clean sheet | 12 |
| Saving a penalty | 9 |
| Save | 2 |
| Successful open play cross | 1 |
| Creating a big chance (a chance where the receiving player should score) | 3 |
| For every 2 clearances, blocks and interceptions (total) | 1 |
| For every 3 recoveries | 1 |
| Key pass | 1 |
| Successful tackle (net*) | 2 |
| Successful dribble | 1 |
| Scoring the goal that wins a match | 3 |
| Goal line clearance | 3 |
| Foul won | 1 |
| Shot on target | 2 |
| 70 to 79% pass completion (at least 30 passes attempted) | 2 |
| 80 to 89% pass completion (at least 30 passes attempted) | 4 |
| 90%+ pass completion (at least 30 passes attempted) | 6 |
| Goalkeepers and defenders conceding a goal | -4 |
| Conceding a penalty | -3 |
| Missing a penalty | -6 |
| Yellow card | -3 |
| Red card | -9 |
| Own goal | -6 |
| Missing a big chance | -3 |
| Making an error which leads to a goal | -3 |
| Being tackled | -1 |
| Conceding a foul | -1 |
| Being caught offside | -1 |
| Shot off target | -1 |

Table 10: Bonus Point System (BPS)

# B Descriptive Statistics

## B.1 Tables

| Metric | All Players (AP) 22/23 | AP 23/24 | AP 24/25 |
|---|---|---|---|
| n - Observations | 26505.00 | 29725.00 | 21750.00 |
| n - Null Observations | 15744.00 | 19072.00 | 12925.00 |
| n - NA Observations | 0.00 | 0.00 | 0.00 |
| Min value observed | -4.00 | -4.00 | -5.00 |
| Max value observed | 23.00 | 26.00 | 25.00 |
| Value range | 27.00 | 30.00 | 30.00 |
| Sum observed values | 31724.00 | 31271.00 | 25374.00 |
| Median value | 0.00 | 0.00 | 0.00 |
| Mean value | 1.20 | 1.05 | 1.17 |
| Variance | 5.55 | 5.26 | 5.44 |
| Standard deviation | 2.36 | 2.29 | 2.33 |
| Skewness | 2.96 | 3.36 | 3.13 |
| Kurtosis | 13.59 | 17.01 | 15.17 |

Table 11: Descriptive Statistics for All Players in the dataset

| Metric | $\geq 1$ played minutes 22/23 | 23/24 | 24/25 |
|---|---|---|---|
| n - Observations | 11345.00 | 11384.00 | 9383.00 |
| n - Null Observations | 587.00 | 735.00 | 559.00 |
| n - NA Observations | 0.00 | 0.00 | 0.00 |
| n - distinct players | 554 | 571 | 547 |
| Min value observed | -4.00 | -4.00 | -5.00 |
| Max value observed | 23.00 | 26.00 | 25.00 |
| Value range | 27.00 | 30.00 | 30.00 |
| Sum observed values | 31729.00 | 31275.00 | 25375.00 |
| Median value | 2.00 | 2.00 | 2.00 |
| Mean value | 2.80 | 2.75 | 2.70 |
| Variance | 8.48 | 9.08 | 8.44 |
| Standard deviation | 2.91 | 3.01 | 2.91 |
| Skewness | 1.94 | 2.08 | 2.10 |
| Kurtosis | 7.37 | 8.23 | 8.34 |

Table 12: Descriptive Statistics for Players with At Least 1 Minute of play time

| Metric | Starting players (SP) 22/23 | SP 23/24 | SP 24/25 |
|---|---|---|---|
| n - Observations | 5368.00 | 8360.00 | 6798.00 |
| n - Null Observations | 265.00 | 483.00 | 372.00 |
| n - NA Observations | 0.00 | 0.00 | 0.00 |
| n - distinct players | 456 | 488 | 471 |
| Min value observed | -3.00 | -4.00 | -5.00 |
| Max value observed | 21.00 | 26.00 | 25.00 |
| Value range | 24.00 | 30.00 | 30.00 |
| Sum observed values | 17814.00 | 27206.00 | 22025.00 |
| Median value | 2.00 | 2.00 | 2.00 |
| Mean value | 3.32 | 3.25 | 3.24 |
| Variance | 9.54 | 1width=0.51 | 9.83 |
| Standard deviation | 3.09 | 3.24 | 3.13 |
| Skewness | 1.61 | 1.79 | 1.79 |
| Kurtosis | 5.77 | 6.86 | 6.89 |

Table 13: Descriptive Statistics for Starting Players in the dataset

### B.1.1 Position Statistics Tables

| position | Count | Mean | Median | SD | Min | Max | Skewness | Kurtosis |
|---|---|---|---|---|---|---|---|---|
| GK | 770 | 3.61 | 2.00 | 3.00 | -3.00 | 15.00 | 1.07 | 3.53 |
| DEF | 3776 | 2.63 | 1.00 | 2.86 | -4.00 | 18.00 | 1.45 | 5.11 |
| MID | 5322 | 2.74 | 2.00 | 2.79 | -2.00 | 21.00 | 2.39 | 9.57 |
| FWD | 1477 | 3.00 | 2.00 | 3.31 | -2.00 | 23.00 | 2.08 | 7.63 |

Table 14: Position Statistics for Players with 1+ Min 22/23

| position | Count | Mean | Median | SD | Min | Max | Skewness | Kurtosis |
|---|---|---|---|---|---|---|---|---|
| GK | 776 | 3.19 | 2.00 | 2.64 | -2.00 | 13.00 | 1.26 | 3.84 |
| DEF | 3843 | 2.42 | 1.00 | 2.94 | -4.00 | 21.00 | 1.83 | 7.09 |
| MID | 5320 | 2.81 | 2.00 | 3.00 | -2.00 | 26.00 | 2.36 | 9.45 |
| FWD | 1445 | 3.14 | 2.00 | 3.34 | -2.00 | 23.00 | 2.03 | 7.68 |

Table 15: Position Statistics for Players with 1+ Min 23/24

| position | Count | Mean | Median | SD | Min | Max | Skewness | Kurtosis |
|---|---|---|---|---|---|---|---|---|
| GK | 625 | 3.29 | 2.00 | 2.71 | -1.00 | 15.00 | 1.35 | 4.42 |
| DEF | 3077 | 2.41 | 1.00 | 2.69 | -5.00 | 15.00 | 1.61 | 5.77 |
| MID | 4645 | 2.69 | 2.00 | 2.89 | -2.00 | 25.00 | 2.58 | 11.10 |
| FWD | 1036 | 3.31 | 2.00 | 3.50 | -2.00 | 18.00 | 1.61 | 5.01 |

Table 16: Position Statistics for Players with 1+ Min 24/25

| position | Count | Mean | Median | SD | Min | Max | Skewness | Kurtosis |
|---|---|---|---|---|---|---|---|---|
| GK | 2791 | 1.00 | 0.00 | 2.26 | -3.00 | 15.00 | 2.76 | 10.74 |
| DEF | 9183 | 1.08 | 0.00 | 2.25 | -4.00 | 18.00 | 2.68 | 10.96 |
| MID | 11418 | 1.28 | 0.00 | 2.35 | -3.00 | 21.00 | 3.15 | 15.32 |
| FWD | 3113 | 1.42 | 0.00 | 2.73 | -2.00 | 23.00 | 2.99 | 13.40 |

Table 17: Position Statistics for All Players 22/23

| position | Count | Mean | Median | SD | Min | Max | Skewness | Kurtosis |
|---|---|---|---|---|---|---|---|---|
| GK | 3413 | 0.73 | 0.00 | 1.84 | -2.00 | 13.00 | 3.24 | 14.09 |
| DEF | 9611 | 0.97 | 0.00 | 2.21 | -4.00 | 21.00 | 3.20 | 15.48 |
| MID | 12847 | 1.16 | 0.00 | 2.37 | -2.00 | 26.00 | 3.41 | 17.41 |
| FWD | 3854 | 1.18 | 0.00 | 2.55 | -2.00 | 23.00 | 3.31 | 16.23 |

Table 18: Position Statistics for All Players 23/24

| position | Count | Mean | Median | SD | Min | Max | Skewness | Kurtosis |
|---|---|---|---|---|---|---|---|---|
| GK | 2288 | 0.90 | 0.00 | 2.04 | -1.00 | 15.00 | 2.94 | 12.38 |
| DEF | 7262 | 1.02 | 0.00 | 2.12 | -5.00 | 15.00 | 2.80 | 11.91 |
| MID | 9798 | 1.27 | 0.00 | 2.40 | -2.00 | 25.00 | 3.37 | 17.62 |
| FWD | 2402 | 1.43 | 0.00 | 2.82 | -2.00 | 18.00 | 2.70 | 1width=0.53 |

Table 19: Position Statistics for All Players 24/25

| position | Count | Mean | Median | SD | Min | Max | Skewness | Kurtosis |
|---|---|---|---|---|---|---|---|---|
| GK | 488 | 3.57 | 2.00 | 2.89 | -3.00 | 11.00 | 0.89 | 2.90 |
| DEF | 1990 | 2.98 | 2.00 | 3.00 | -3.00 | 17.00 | 1.23 | 4.39 |
| MID | 2342 | 3.38 | 2.00 | 3.01 | -2.00 | 21.00 | 2.10 | 7.76 |
| FWD | 548 | 4.09 | 2.00 | 3.70 | -2.00 | 17.00 | 1.39 | 4.08 |

Table 20: Position Statistics for Starters 22/23

| position | Count | Mean | Median | SD | Min | Max | Skewness | Kurtosis |
|---|---|---|---|---|---|---|---|---|
| GK | 760 | 3.23 | 2.00 | 2.64 | -2.00 | 13.00 | 1.25 | 3.81 |
| DEF | 3150 | 2.73 | 2.00 | 3.14 | -4.00 | 21.00 | 1.57 | 6.00 |
| MID | 3577 | 3.51 | 2.00 | 3.26 | -2.00 | 26.00 | 2.05 | 7.65 |
| FWD | 873 | 4.12 | 2.00 | 3.68 | -2.00 | 23.00 | 1.70 | 6.04 |

Table 21: Position Statistics for Starters 23/24

| position | Count | Mean | Median | SD | Min | Max | Skewness | Kurtosis |
|---|---|---|---|---|---|---|---|---|
| GK | 618 | 3.31 | 2.00 | 2.72 | -1.00 | 15.00 | 1.34 | 4.39 |
| DEF | 2513 | 2.72 | 2.00 | 2.86 | -5.00 | 15.00 | 1.35 | 4.87 |
| MID | 3033 | 3.40 | 2.00 | 3.19 | -2.00 | 25.00 | 2.24 | 8.88 |
| FWD | 634 | 4.48 | 2.00 | 3.81 | -2.00 | 18.00 | 1.17 | 3.60 |

Table 22: Position Statistics for Starters 24/25

## B.2 Figures

### B.2.1 Box Plots for Starting Players

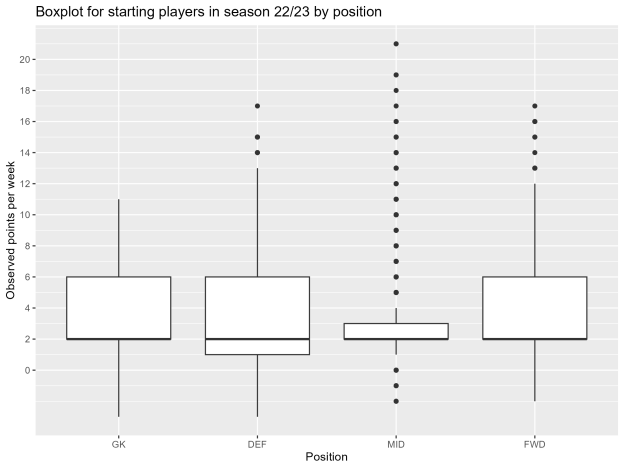



Figure 12: Box plot of starting players by position for the 2022/23 season

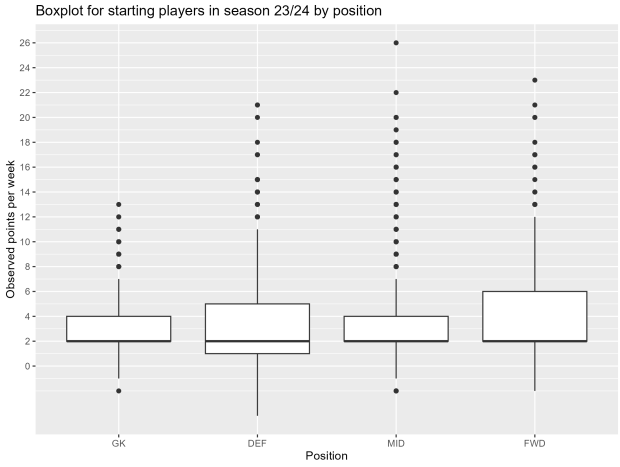



Figure 13: Box plot of starting players by position for the 2023/24 season

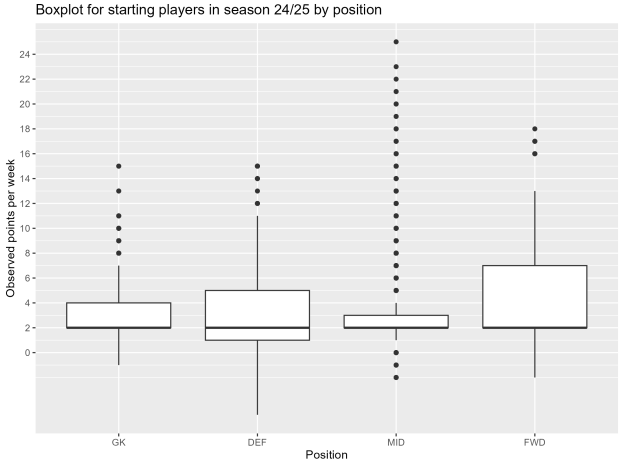



Figure 14: Box plot of starting players by position for the 2024/25 season

### B.2.2 Box Plots for All Players

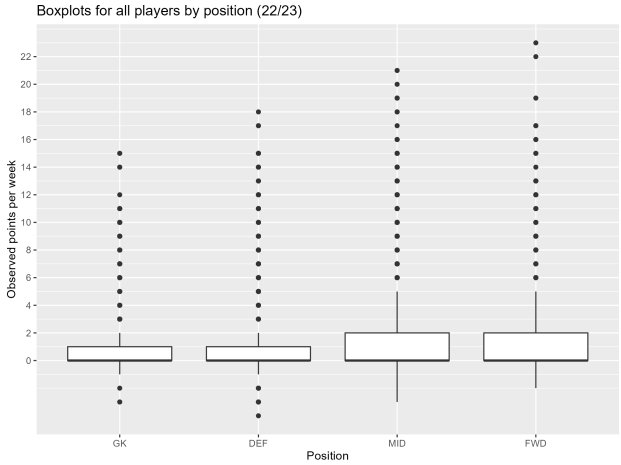



Figure 15: Box plots for all players by position in the 2022/23 season

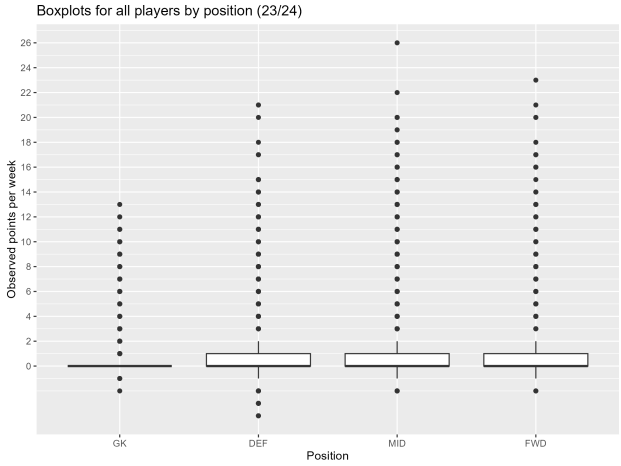



Figure 16: Box plots for all players by position in the 2023/24 season

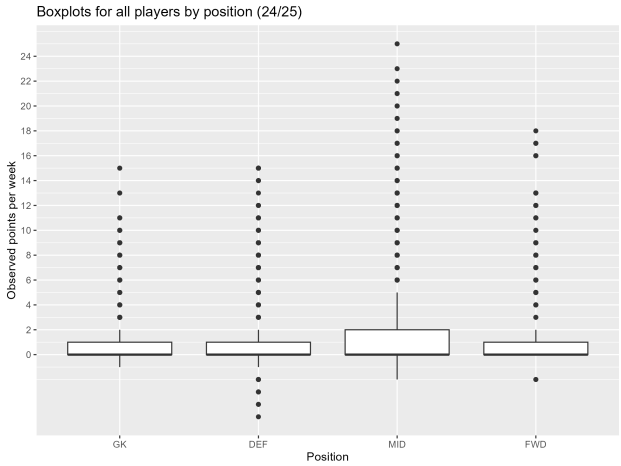



Figure 17: Box plots for all players by position in the 2024/25 season

## B.3 Box Plots for Players With 1+ Minutes Played

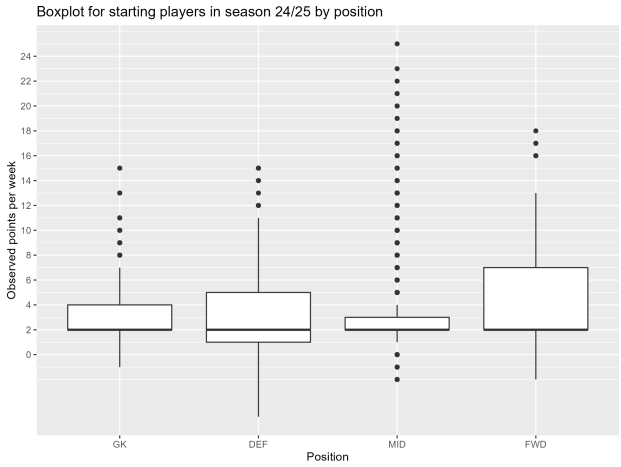



Figure 18: Box plots by position for players with at least 1 minute played in the 2022/23 season

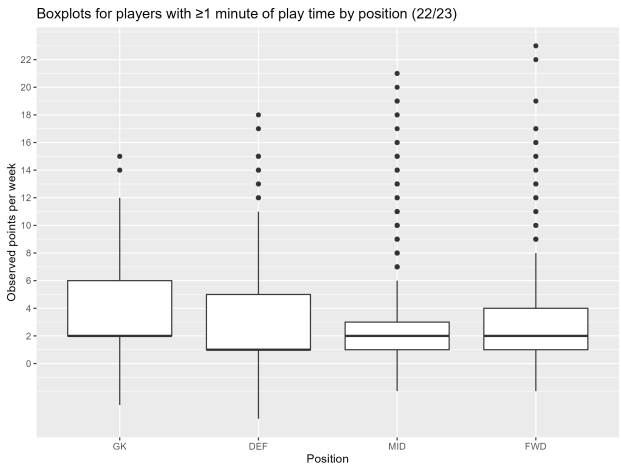



Figure 19: Box plots by position for players with at least 1 minute played in the 2023/24 season

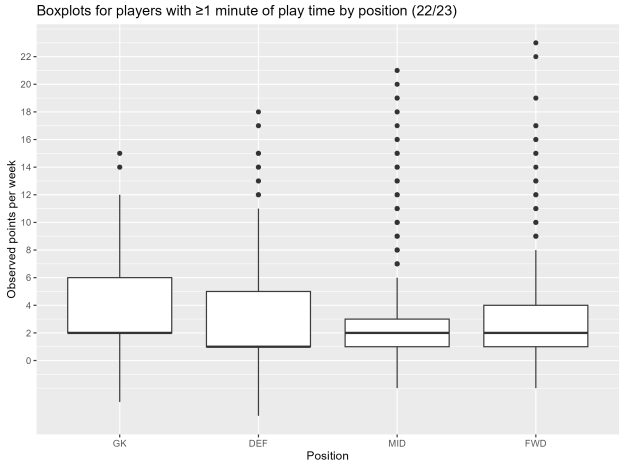



Figure 20: Box plots by position for players with at least 1 minute played in the 2024/25 season

## B.4 Additional Plots

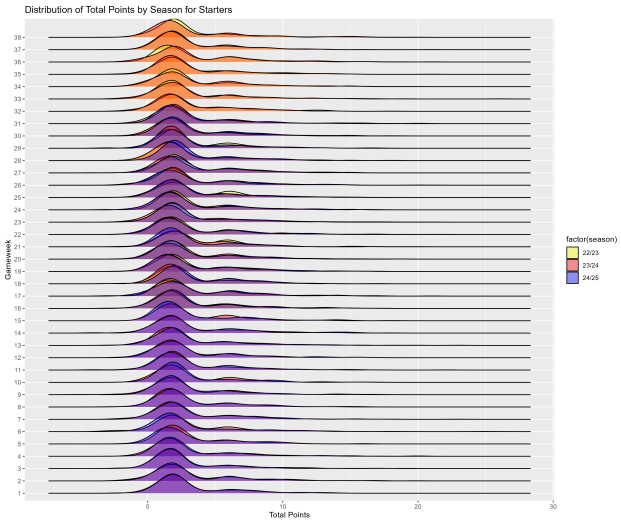



Figure 21: Gameweek ridgeline plot by position

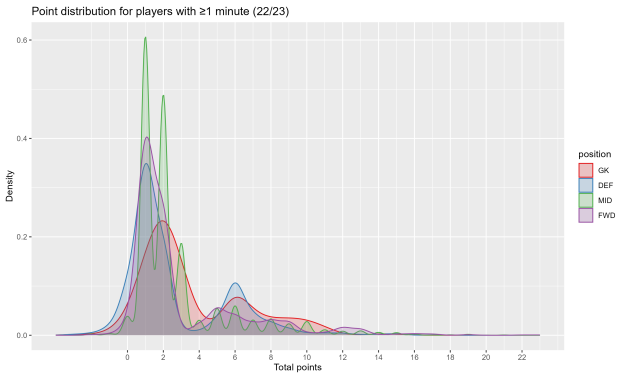



Figure 22: Point density distribution by position for players with 1+ minute played in the 2022/23 season

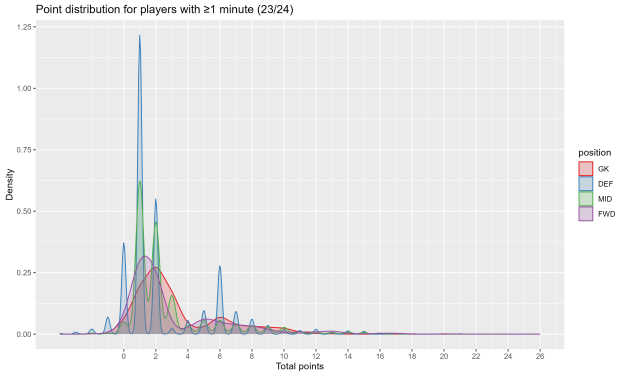



Figure 23: Point density distribution by position for players with 1+ minute played in the 2023/24 season

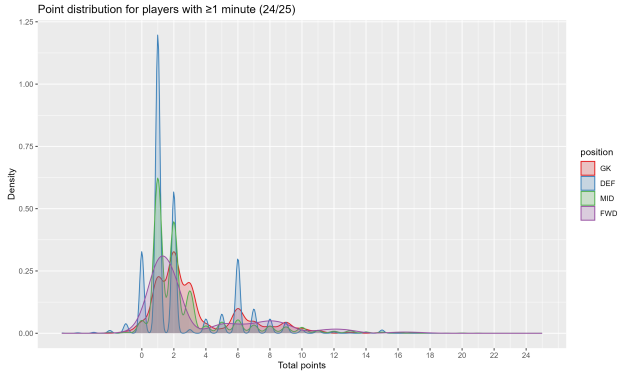



Figure 24: Point density distribution by position for players with 1+ minute played in the 2024/25 season

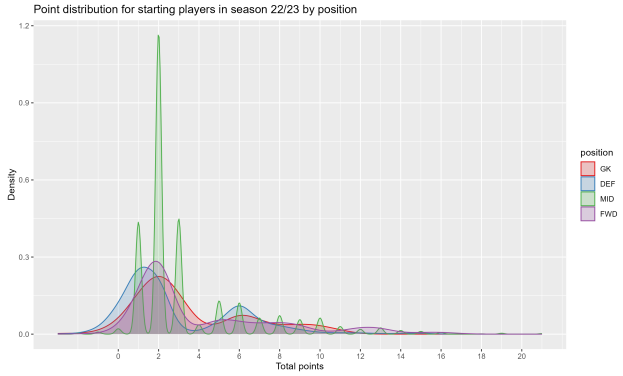



Figure 25: Point density distribution for starting players in the 2022/23 season

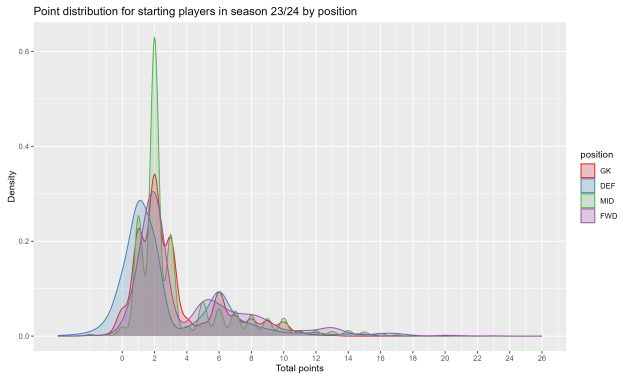



Figure 26: Point density distribution for starting players in the 2023/24 season

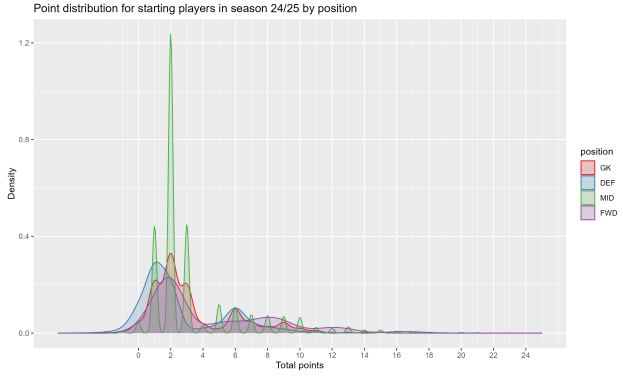



Figure 27: Point density distribution for starting players in the 2024/25 season

## B.5 QQ Plots

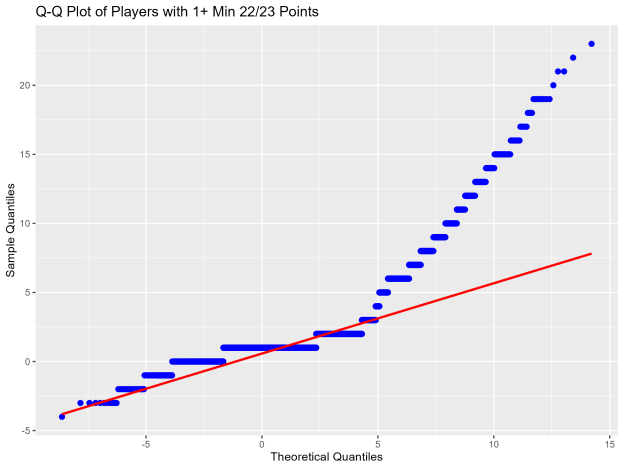



Figure 28: QQ Plot of players with 1+ minutes per gameweek in the 22/23 season

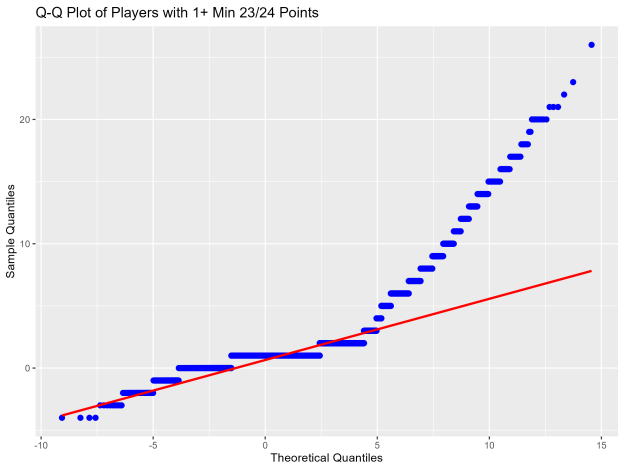



Figure 29: QQ Plot of players with 1+ minutes per gameweek in the 23/24 season

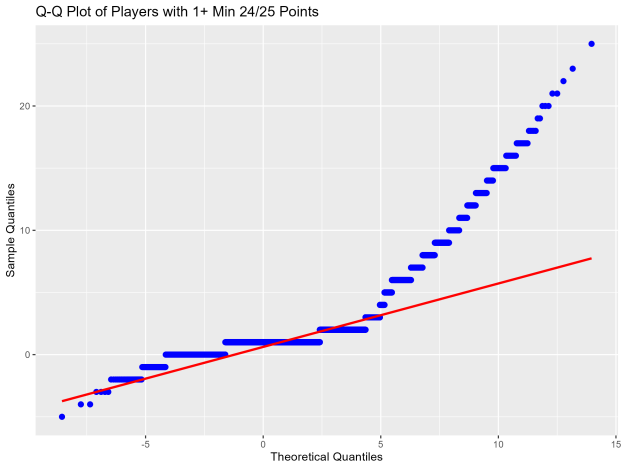



Figure 30: QQ Plot of players with 1+ minutes per gameweek in the 24/25 season

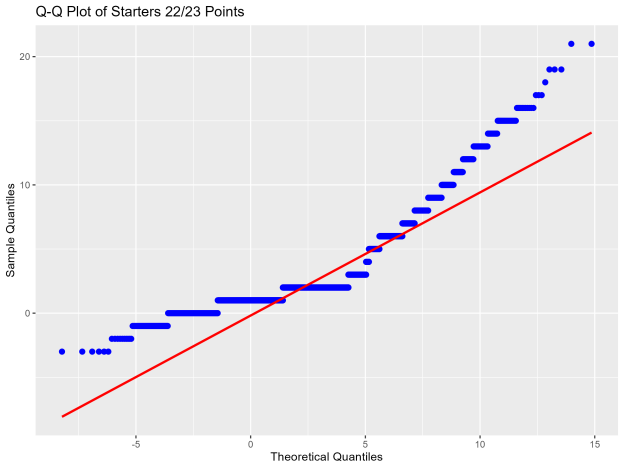



Figure 31: QQ Plot of starting players in the 22/23 season

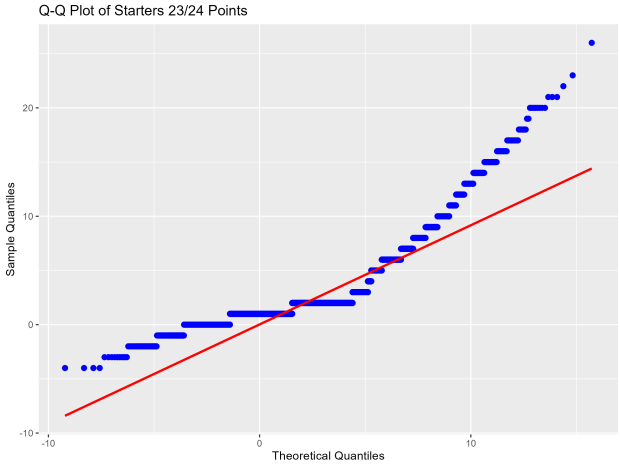



Figure 32: QQ Plot of starting players in the 23/24 season

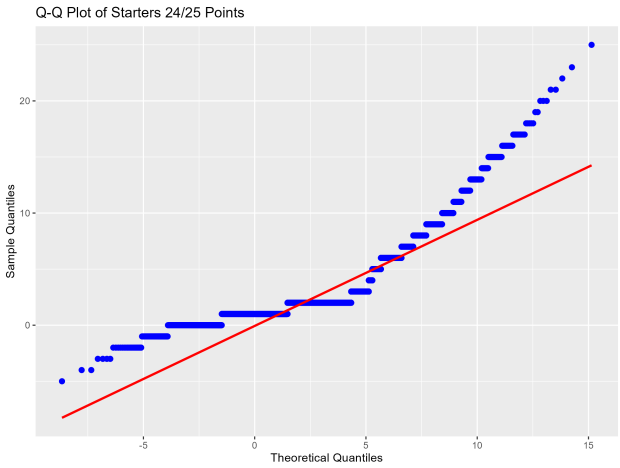



Figure 33: QQ Plot of starting players in the 24/25 season

## B.6 Model Structures

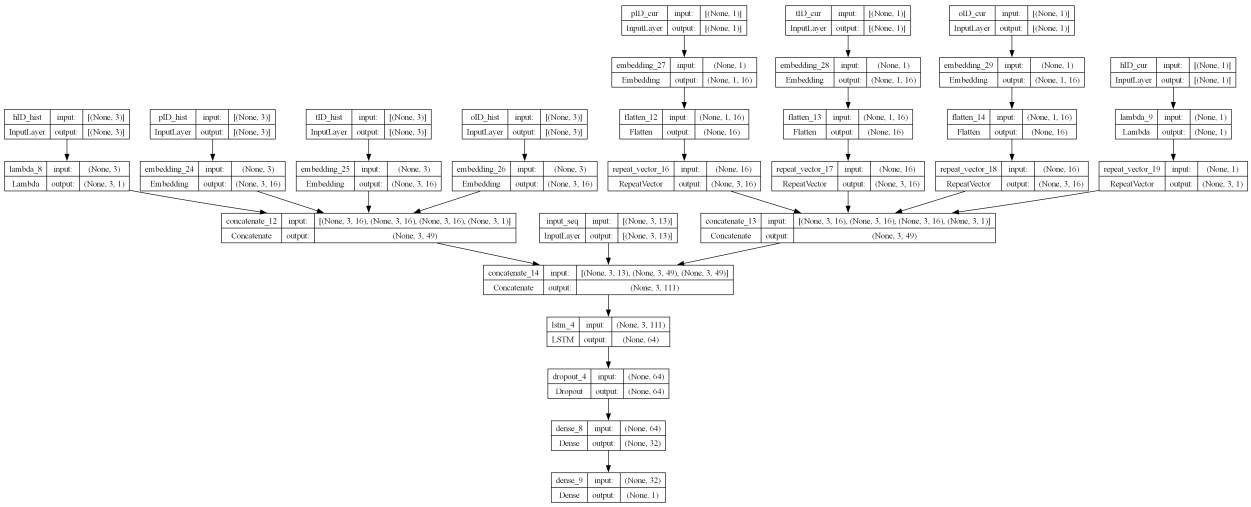



Figure 34: Keras model structure for GK

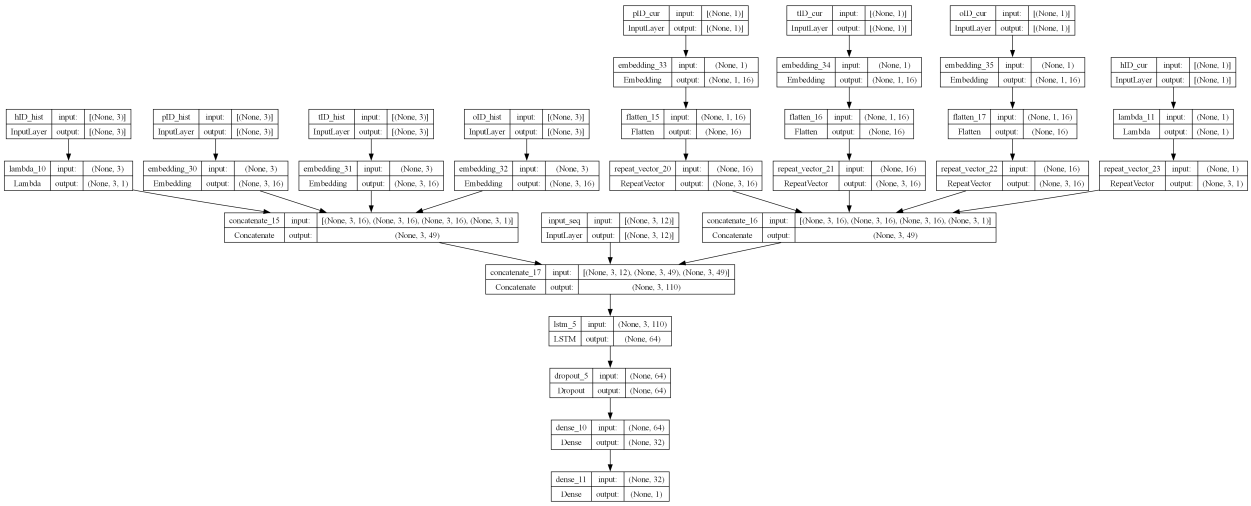



Figure 35: Keras model structure for DEF

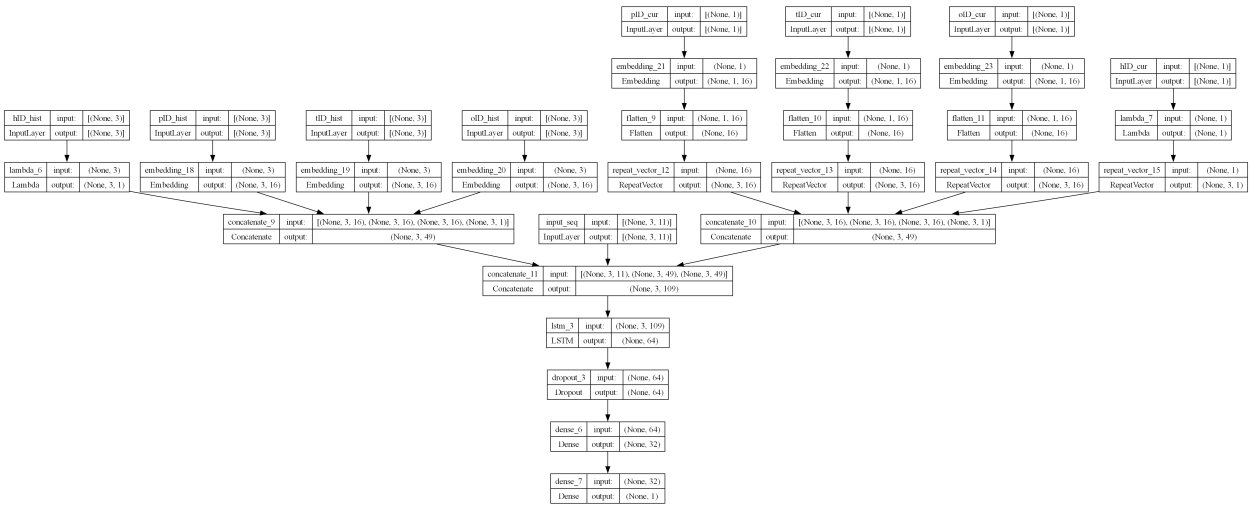



Figure 36: Keras model structure for FWD

# C Squad optimisation model constraints

The following material is sourced from Eilertsen et al. (2018).

| Sets | Description | |
|---|---|---|
| $\mathcal{T}$ | Set of premier league gameweeks | |
| $\mathcal{P}$ | Set of all fotball players | |
| $\mathcal{C}$ | Set of premier league teams | |
| $\mathcal{L}$ | Squad substitution priorities, where 1 is first priority | |
| **Subsets** | | **Parent set** |
| $\mathcal{P}^{GK}$ | Subset of goalkeepers | $\mathcal{P}$ |
| $\mathcal{P}^{DEF}$ | Subset of defenders | $\mathcal{P}$ |
| $\mathcal{P}^{MID}$ | Subset of midfielders | $\mathcal{P}$ |
| $\mathcal{P}^{FWD}$ | Subset of forwards | $\mathcal{P}$ |
| $\mathcal{P}_c$ | Fotball players in team c | $\mathcal{P}, \mathcal{C}$ |
| $\mathcal{T}_{FH}$ | Subset of gameweeks $\leq Season/2$ | $\mathcal{T}$ |
| $\mathcal{T}_{SH}$ | Subset of gameweeks $> Season/2$ | $\mathcal{T}$ |
| **Index** | | |
| $p$ | Players | |
| $t$ | Gameweeks | |
| $l$ | Substitution priority | |

## C.1 Model Constraints

This sections presents all the model constraints from Eilertsen et al. (2018, ch.4-ch.6).

### C.1.1 Full Squad Constraints

$$\sum_{p \in \mathcal{P}^{GK}} x_{pt} = M^{GK} \quad t \in \mathcal{T} \tag{16}$$

$$\sum_{p \in \mathcal{P}^{DEF}} x_{pt} = M^{DEF} \quad t \in \mathcal{T} \tag{17}$$

$$\sum_{p \in \mathcal{P}^{MID}} x_{pt} = M^{MID} \quad t \in \mathcal{T} \tag{18}$$

$$\sum_{p \in \mathcal{P}^{FWD}} x_{pt} = M^{FWD} \quad t \in \mathcal{T} \tag{19}$$

$$\sum_{p \in \mathcal{P}_c} x_{pt} \leq M^{C} \quad t \in \mathcal{T}, c \in \mathcal{C} \tag{20}$$

| **Parameter** | Description |
|---|---|
| $\rho_{pt}(\omega_t)$ | Points for a player $p$ dependent on the realization of a random event, $\omega_t$, in gameweek $t$. |
| $\epsilon$ | Constant such that $\epsilon << 1$ |
| $k_l$ | Constants such that $k_l << \epsilon, \forall l \in \mathcal{L}, \wedge k_1 > \cdots > k_{\mathcal{L}}$ |
| $C_{PT}^{Sell}$ | Sell price of player $p$ in a gameweek $t$ |
| $C_{pt}^{Buy}$ | Value of a player $p$ in a gameweek $t$ |
| $R$ | Number of points deducted for each penalized transfer |
| $M^{GK}$ | Number of goalkeepers required in the selected squad |
| $M^{DEF}$ | Number of defender required in the selected squad |
| $M^{MID}$ | Number of midfielders required in the selected squad |
| $M^{FWD}$ | Number of forwards required in the selected squad |
| $M^{C}$ | Maximum number of players allowed in the selected squad from the same team |
| $E$ | Number of players required in the starting line-up |
| $E^{GK}$ | Number of Goalkeepers required in the starting line-up |
| $E^{DEF}$ | Number of Defenders required in the starting line-up |
| $E^{MID}$ | Number of Midielders required in the starting line-up |
| $E^{FWD}$ | Number of Forwards required in the starting line-up |
| $\beta$ | Sufficiently high constant |
| $\bar{\alpha}$ | Sufficiently high constant |
| $\phi$ | Number of players which are substitutes |
| $\phi^{GK}$ | Number of goalkeepers which are substitutes |
| $\overline{Q}$ | Maximum number of free transfers possible to accumulate over gameweeks |
| $\underline{Q}$ | Number of free transfers every gameweek |
| **Binary Variable** | Description |
| $x_{pt}$ | 1 if a player p is picked for the selected squad in a gameweek t, 0 otherwise |
| $x_{pt}^{\text{freehit}}$ | 1 if a player p is picked for the selected squad in a gameweek t where the Free Hit is used, 0 otherwise. |
| $y_{pt}$ | 1 if a player p is picked for starting line-up in a gameweek t, 0 otherwise. |
| $f_{pt}$ | 1 if a player p is picked as captain in a gameweek t, 0 otherwise |
| $h_{pt}$ | 1 if a player p is picked as vice-captain in a gameweek t, 0 otherwise |
| $g_{ptl}$ | 1 if a player p is picked as substitution priority l in a gameweek t, 0 otherwise |
| $u_{pt}$ | 1 if a player p is transferred out of the selected squad in a gameweek t, 0 otherwise. |
| $e_{pt}$ | 1 if a player p is transferred in to the selected squad in a gameweek t, 0 otherwise |
| $w_t$ | 1 if a Wildcard is used in a gameweek t , 0 otherwise |
| $c_{pt}$ | 1 if the Triple Captain is used on a player p in a gameweek t, 0 otherwise |
| $b_t$ | 1 if the Bench Boost is used in a gameweek t, 0 otherwise |
| $r_t$ | 1 if the Free Hit is used in a gameweek t, 0 otherwise. |
| $\lambda_{pt}$ | Binary auxiliary variable |
| Other Variables | | In |
| $v_t$ | Remaining budget in a gameweek t | $v_t \geq 0$ |
| $q_t$ | Number of free transfers available in a gameweek t | $q_t \in \mathbb{Z}^+$ |
| $\alpha_t$ | Number of penalized transfers in a gameweek t | $\alpha_t \in \mathbb{Z}^+$ |

Table 23: Model Parameters and Binary Variables

$$\sum_{p \in \mathcal{P}^{GK}} x_{pt}^{freehit} = M^{GK} r_t \quad t \in \mathcal{T} \tag{21}$$

$$\sum_{p \in \mathcal{P}^{DEF}} x_{pt}^{freehit} = M^{DEF} r_t \quad t \in \mathcal{T} \tag{22}$$

$$\sum_{p \in \mathcal{P}^{MID}} x_{pt}^{freehit} = M^{MID} r_t \quad t \in \mathcal{T} \tag{23}$$

$$\sum_{p \in \mathcal{P}^{FWD}} x_{pt}^{freehit} = M^{FWD} r_t \quad t \in \mathcal{T} \tag{24}$$

$$\sum_{p \in \mathcal{P}_c} x_{pt} \leq M^{C} \quad t \in \mathcal{T}, c \in \mathcal{C} \tag{25}$$

The above constraints (16)-(20) make sure the total number of players is equal to $M$ and

only has amount $M^C$ players from the same team. Equations (21)-(25) are the freehit squad.

### C.1.2 Starting Squad Constraints

$$\sum_{p \in \mathcal{P}} y_{pt} = E + \phi b_t \quad t \in T \tag{26}$$

$$\sum_{p \in \mathcal{P}^{GK}} y_{pt} = E^{GK} + \phi^{GK} b_t \quad t \in T \tag{27}$$

$$\sum_{p \in \mathcal{P}^{DEF}} y_{pt} \geq E^{DEF} \quad t \in T \tag{28}$$

$$\sum_{p \in \mathcal{P}^{MID}} y_{pt} \geq E^{MID} \quad t \in T \tag{29}$$

$$\sum_{p \in \mathcal{P}^{FWD}} y_{pt} \geq E^{FWD} \quad t \in T \tag{30}$$

$$y_{pt} \leq x_{pt}^{\text{freehit}} + x_{pt}(1 - r_t) \quad p \in \mathcal{P}, t \in \mathcal{T} \tag{31}$$

(26)-(31) models the starting eleven of a selected squad, the last equation makes sure the starting eleven comes from the chosen squad. Since this equation is non-linear Eilertsen et al. (2018) has implemented a linearisation (32),(33)(34) with $\lambda_{pt}$ as an helper variable:

$$y_{pt} \leq x_{pt}^{freehit} + \lambda_{pt} \quad p \in \mathcal{P}, t \in \mathcal{T} \tag{32}$$

$$\lambda_{pt} \leq x_{pt} \quad \mathcal{P}, t \in \mathcal{T} \tag{33}$$

$$\lambda_{pt} \leq (1 - r_t) \quad \forall p \in P, t \in T \tag{34}$$

### C.1.3 Captaincy constraints

Equations (35),(36), and (37) makes sure only one player can be captain and vice-captain, it makes sure the triple captain chip replaces the normal captain in the model and enforces only role at a time.

Either Captain or Triple Captain:

$$\sum_{p \in P} f_{pt} + \sum_{p \in P} c_{pt} = 1 \quad t \in \mathcal{T} \tag{35}$$

Vise-Captain:

$$\sum_{p\in\mathcal{P}} h_{pt} = 1 \quad t \in \mathcal{T} \tag{36}$$

Only one of three at any time:

$$f_{pt} + c_{pt} + h_{pt} \leq y_{pt} \quad p \in \mathcal{P}, t \in \mathcal{T} \tag{37}$$

### C.1.4 Substitution constraints

The equations below (38),(39), and (40) imposes the substitution constraints:

$$y_{pt} + \sum_{l\in\mathcal{L}} g_{ptl} \leq x_{pt} + \beta r_t \quad p \in \mathcal{P} \setminus \mathcal{P}^{GK}, t \in \mathcal{T} \tag{38}$$

$$y_{pt} + \sum_{l\in\mathcal{L}} g_{ptl} \leq x_{pt}^{\text{freehit}} + \beta(1 - r_t) \quad p \in \mathcal{P} \setminus \mathcal{P}^{GK}, t \in \mathcal{T} \tag{39}$$

$$\sum_{p\in\mathcal{P}\setminus\mathcal{P}^{GK}} g_{ptl} \leq 1 \quad t \in \mathcal{T}, l \in \mathcal{L} \tag{40}$$

### C.1.5 Budget Constraints

The equations below (Eilertsen et al., 2018, eq.4.33-4.39) (41),(42),(43), (44),(45),(46), (47) imposes the budget constraints. Equations (41)-(44), keeps track and imposes limitations of budget, equations (45) - (47) keeps the budget constraint while the free hit chip is active.

$$B^S - \sum_{p\in\mathcal{P}} C_{p1}^{Buy} x_{p1} = v_1 \tag{41}$$

Starting budget minus value of player p in gameweek one equals remaining budget t 1.

$$v_{t-1} + \sum_{p\in P} C_{pt}^{Sell} u_{pt} - \sum_{p\in P} C_{pt}^{Buy} e_{pt} = v_t \quad t \in \mathcal{T} \setminus \{1\} \tag{42}$$

$$x_{p,(t-1)} + e_{pt} - u_{pt} = x_{pt} \quad p \in \mathcal{P}, t \in \mathcal{T} \setminus \{1\} \tag{43}$$

$$e_{pt} + u_{pt} \leq 1 \quad p \in \mathcal{P}, t \in \mathcal{T} \tag{44}$$

$$\sum_{p\in\mathcal{P}} C_{pt}^{Sell} x_{p,(t-1)} + v_{t-1} \geq \sum_{p\in\mathcal{P}} C_{pt}^{Buy} x_{pt}^{\text{freehit}} \quad t \in \mathcal{T} \setminus \{1\} \tag{45}$$

$$\sum_{p \in \mathcal{P}} u_{pt} \leq E(1 - r_t) \quad t \in \mathcal{T} \setminus \{1\} \tag{46}$$

$$\sum_{p \in \mathcal{P}} e_{pt} \leq E(1 - r_t) \quad t \in \mathcal{T} \setminus \{1\} \tag{47}$$

### C.1.6 Gamechips

$$\sum_{t \in T_{FH}} w_t \leq 1 \tag{48}$$

$$\sum_{t \in T_{SH}} w_t \leq 1 \tag{49}$$

$$\sum_{\substack{p \in \mathcal{P} \\ t \in \mathcal{T}}} c_{pt} \leq 1 \tag{50}$$

$$\sum_{t \in \mathcal{T}} b_t \leq 1 \tag{51}$$

$$\sum_{t \in \mathcal{T}} r_t \leq 1 \tag{52}$$

$$w_t + \sum_{p \in \mathcal{P}} c_{pt} + b_t + r_t \leq 1 \quad t \in \mathcal{T} \tag{53}$$

$$y_{pt} \leq x_{pt}^{\text{freehit}} + \lambda_{pt} \quad p \in P, t \in T \tag{54}$$

These constraints (48)-(53) enforce activation of the gamechips and makes sure only one is available at a time.

### C.1.7 Transfer Constraints

Equations (55)-(59) implements the transfer mechanics of the game, transfer balance and number of free transfers given each week, including free-hit and wildcard. Importantly. E(Starting XI) is used and not M(XV).

$$q_2 = \underline{Q} \tag{55}$$

$$Ew_t + q_t - \sum_{p \in \mathcal{P}} e_{pt} + \underline{Q} + \alpha_t \geq q_{t+1} \quad t \in \mathcal{T} \setminus \{1\} \tag{56}$$

$$\bar{\alpha}(\overline{Q} - q_{t+1}) \geq \alpha_t \quad t \in \mathcal{T} \tag{57}$$

$$q_{t+1} \geq \underline{Q} \quad t \in \mathcal{T} \tag{58}$$

$$q_{t+1} \leq \overline{Q} + (\underline{Q} - \overline{Q})w_t + (\underline{Q} - \overline{Q})r_t \quad t \in \mathcal{T} \tag{59}$$

# D Python Script: Mixed Integer Linear Program

```
    import pandas as pd
import pulp
import numpy as np
import time
import sys
import os
import math # Used for rounding FT
import argparse # Add this import

# Dette skriptet er forskjellig fra t ved at den bruker .bat
   filen
# Denne filen har ikke valgfritt starter lag
# DVS. den kjøres i en løkke uten å måtte åpne vscode
# Går sekvensielt
# Forskjellen er at denne gir faktisk poeng til laget også.

# --- Command Line Arguments ---
parser = argparse.ArgumentParser(description='FPL Optimisation
   with MILP')
parser.add_argument('--horizon', type=int, default=3, help='Sub
   -horizon length (weeks to look ahead)')
parser.add_argument('--start_gw', type=int, default=78, help='
   Starting gameweek')
parser.add_argument('--max_gw', type=int, default=78+29, help='
   Maximum gameweek')
args = parser.parse_args()

# --- Input Parameters ---
```

```
CSV_FILE_PATH = "C:/Users/peram/Documents/test/
   Validation_Predictions_Clean_v2.csv"

START_GAMEWEEK = args.start_gw # Now uses command line argument
MAX_GAMEWEEK = args.max_gw # Now uses command line argument
SUB_HORIZON_LENGTH = args.horizon # Now uses command line
   argument
# --- SET TIMELIMIT HERE ---
SOLVER_TIME_LIMIT = None  # Seconds (e.g., 600 for 10 mins) or
   None for no limit


# ---  Gamechip manual usage or None ----
SEASON_START_GW = START_GAMEWEEK
TARGET_GW_WC1_OPPORTUNITY = 0 #SEASON_START_GW + 1
TARGET_GW_TC = 0 #None#SEASON_START_GW + 2
TARGET_GW_FH = 0 #None#SEASON_START_GW + 3
TARGET_GW_WC2_OPPORTUNITY = 0#None#SEASON_START_GW + 4
TARGET_GW_BB = 0 #None#SEASON_START_GW + 5
print(f"Chip Opportunity GWs (Absolute): WC1={
   TARGET_GW_WC1_OPPORTUNITY}, TC={TARGET_GW_TC}, FH={
   TARGET_GW_FH}, WC2={TARGET_GW_WC2_OPPORTUNITY}, BB={
   TARGET_GW_BB}")


# --- Configure Solver (Default Threads) ---
solver_to_use = pulp.PULP_CBC_CMD(
    msg=True, # Show solver output
    timeLimit=SOLVER_TIME_LIMIT # Pass timeLimit if set
)
print(f"--- Using Solver: {solver_to_use.name} (Default Threads
   ) ---")
if SOLVER_TIME_LIMIT:
    print(f"--- Solver Time Limit per Subproblem: {
       SOLVER_TIME_LIMIT}s ---")
else:
```

```
    print("--- Solver Time Limit per Subproblem: None ---")


# --- Check Versions ---
print(f"--- Running on Python {sys.version} ---")
print(f"--- PuLP Version: {pulp.__version__} ---")

# --- 0. Setup & Data Loading ---
print("\n--- 0. Setup & Data Loading ---")
try:
    allesesonger = pd.read_csv(CSV_FILE_PATH)
    print(f"Raw CSV '{CSV_FILE_PATH}' loaded successfully.")
except FileNotFoundError:
    print(f"ERROR: Raw CSV file '{CSV_FILE_PATH}' not found.")
    sys.exit()
except Exception as e:
    print(f"ERROR: Failed to load raw CSV file. Reason: {e}")
    sys.exit()

print("Initial raw data shape:", allesesonger.shape)

# --- Data Pre-processing and Filtering ---
print("\n--- Pre-processing Raw Data ---")
essential_input_cols = ['player_id', 'GW', 'name', 'position',
   'team', 'predicted_total_points', 'value'] # Adjusted
missing_cols = [col for col in essential_input_cols if col not
   in allesesonger.columns]
if missing_cols:
    print(f"ERROR: Missing essential columns in the raw CSV: {
       missing_cols}")
    sys.exit()

# Ensure GW and value are numeric
try:
    allesesonger['GW'] = pd.to_numeric(allesesonger['GW'])
```

```
    allesesonger['value'] = pd.to_numeric(allesesonger['value
        '])
    allesesonger['value'] = allesesonger['value'].fillna(50.0)
        # Fill NA before potential scaling
    # --- SCALING --- Uncomment if your value needs scaling
    # allesesonger['value'] = allesesonger['value'] * 10
    # print("Value scaled by 10")
except ValueError as e:
    print(f"ERROR: Could not convert 'GW' or 'value' column to
        numeric: {e}")
    sys.exit()

# Add after line 73 (after filling NA values)
# --- DOUBLE GAMEWEEK HANDLING ---
print("Checking for double gameweeks and resolving...")
try:
    # Find players with multiple appearances in the same
        gameweek
    player_gw_counts = allesesonger.groupby(['player_id', 'GW
        ']).size().reset_index(name='appearances')
    double_gw_entries = player_gw_counts[player_gw_counts['
        appearances'] > 1]

    if not double_gw_entries.empty:
        print(f"Found {len(double_gw_entries)} player-gameweek
            combinations with multiple appearances")

        # Check for potential inconsistencies in double
            gameweeks
        inconsistencies = []
        for _, row in double_gw_entries.iterrows():
            player_id, gw = row['player_id'], row['GW']
            player_gw_data = allesesonger[(allesesonger['
                player_id'] == player_id) & (allesesonger['GW']
                == gw)]
```

```
        # Check if position or name is inconsistent
        if player_gw_data['position'].nunique() > 1:
            inconsistencies.append(f"Player ID {player_id}
               has different positions in GW {gw}")

        # Check if team is different (this could be valid
           for mid-season transfers)
        if player_gw_data['team'].nunique() > 1:
            teams = ", ".join(player_gw_data['team'].unique
               ())
            player_name = player_gw_data['name'].iloc[0]
            print(f"NOTE: Player {player_name} (ID {
               player_id}) appears for multiple teams in GW
                {gw}: {teams}")

if inconsistencies:
    print("WARNING: Found inconsistencies in double
       gameweek data:")
    for issue in inconsistencies:
        print(f"  - {issue}")

# Perform the aggregation with necessary handling
print("Aggregating double gameweeks...")
# Define which columns to sum and which to keep first
   value
sum_cols = ['predicted_total_points', '
   actual_total_points'] if 'actual_total_points' in
   allesesonger.columns else ['predicted_total_points']
first_cols = ['name', 'position', 'team', 'value']

# Create aggregation dictionary
agg_dict = {col: 'sum' for col in sum_cols}
agg_dict.update({col: 'first' for col in first_cols if
   col in allesesonger.columns})

# Group and aggregate
```

```
            allesesonger = allesesonger.groupby(['player_id', 'GW
                '], as_index=False).agg(agg_dict)
            print(f"Data shape after double gameweek aggregation: {
                allesesonger.shape}")
        else:
            print("No double gameweeks found in the data")

except Exception as e:
    print(f"WARNING: Error during double gameweek processing: {
        e}")
    print("Continuing with original data without double
        gameweek aggregation")

# Filter Gameweeks for the entire run horizon
data_load_start_gw = START_GAMEWEEK - 1 if START_GAMEWEEK > 1
    else START_GAMEWEEK
data_full_range_raw = allesesonger[(allesesonger['GW'] >=
    data_load_start_gw) & (allesesonger['GW'] <= MAX_GAMEWEEK)].
    copy()
if data_full_range_raw.empty:
    print(f"ERROR: No raw data found for GW range {
        data_load_start_gw}-{MAX_GAMEWEEK}.")
    sys.exit()
print(f"Filtered raw data for GW {data_load_start_gw}-{
    MAX_GAMEWEEK}. Shape: {data_full_range_raw.shape}")

# Clean categorical columns
for col in ['team', 'position', 'name']:
    if col in data_full_range_raw.columns:
        data_full_range_raw[col] = data_full_range_raw[col].
            astype(str).str.strip()


# --- 1. Data Cleaning, Aggregation & FULL Set Definition ---
print("\n--- 1. Data Cleaning, Aggregation & FULL Set
    Definition ---")
```

```
# Define initial FULL sets from raw filtered data
T_setofgameweeks_full = sorted(data_full_range_raw['GW'].unique
   ())
P_setofplayers_initial = sorted(data_full_range_raw['player_id
   '].unique())
C_setofteams_initial = sorted(data_full_range_raw['team'].
   dropna().unique())

n_T_full = len(T_setofgameweeks_full)
n_P_initial = len(P_setofplayers_initial)
if n_T_full == 0 or n_P_initial == 0: print("ERROR: Initial GW
   or Player set empty."); sys.exit()
print(f"Initial FULL sets: {n_T_full} GWs, {n_P_initial}
   Players, {len(C_setofteams_initial)} Teams.")

# Create all player-GW combinations for merging
player_gw_combos_full = pd.MultiIndex.from_product([
   P_setofplayers_initial, T_setofgameweeks_full],
                                                    names=['
                                                       player_id
                                                       ', 'GW
                                                       '])
data_complete_full = pd.DataFrame(index=player_gw_combos_full).
   reset_index()

# Merge and fill missing data
print("Merging and filling missing data...")
data_merged_full = pd.merge(data_complete_full,
   data_full_range_raw, on=['player_id', 'GW'], how='left',
   suffixes=('', '_discard'))
data_merged_full = data_merged_full[[col for col in
   data_merged_full.columns if not col.endswith('_discard')]]
data_merged_full.sort_values(by=['player_id', 'GW'], inplace=
   True)
essential_info_cols = ['name', 'position', 'team']
```

```
for col in essential_info_cols: data_merged_full[col] =
   data_merged_full.groupby('player_id')[col].transform(lambda
   x: x.ffill().bfill())
data_merged_full['predicted_total_points'] = data_merged_full['
   predicted_total_points'].fillna(0)
data_merged_full['value'] = data_merged_full.groupby('player_id
   ')['value'].transform(lambda x: x.ffill().bfill())
data_merged_full['value'] = data_merged_full['value'].fillna
   (50.0)

# --- AGGREGATION FOR DGWs --- # <<<< THIS BLOCK IS KEPT
print("Aggregating data for Double Gameweeks...")
sum_cols = ['predicted_total_points', 'minutes', 'goals_scored
   ', 'assists', 'bonus', 'bps', 'saves', 'penalties_saved'] #
   Add/remove as needed
first_cols = ['value', 'name', 'position', 'team']
group_cols = ['player_id', 'GW']
agg_funcs = {}
for col in sum_cols:
    if col in data_merged_full.columns: agg_funcs[col] = 'sum'
for col in first_cols:
    if col in data_merged_full.columns: agg_funcs[col] = 'first
       '
data_aggregated = data_merged_full.groupby(group_cols, as_index
   =False).agg(agg_funcs)
print(f"Data shape after aggregation: {data_aggregated.shape}")

# --- Post-Aggregation Cleaning & Filtering ---
data_cleaned_full = data_aggregated.copy() # <<<< USE
   AGGREGATED DATA
print("Filtering invalid rows post-aggregation...")
initial_rows_agg = data_cleaned_full.shape[0]
data_cleaned_full = data_cleaned_full.dropna(subset=
   essential_info_cols).copy()
data_cleaned_full['team'] = data_cleaned_full['team'].astype(
   str)
```

```
C_setofteams_initial_str = [str(team) for team in
   C_setofteams_initial]
data_cleaned_full = data_cleaned_full[data_cleaned_full['team
   '].isin(C_setofteams_initial_str)]
rows_after_filter_agg = data_cleaned_full.shape[0]
print(f"Removed {initial_rows_agg - rows_after_filter_agg} rows
    post-aggregation (NaNs, invalid teams).")
if data_cleaned_full.empty: print("ERROR: No data remaining
   after aggregation/cleaning."); sys.exit()
print(f"Cleaned aggregated data shape: {data_cleaned_full.shape
   }")

# --- Define FINAL FULL Sets and Parameters (using
   data_cleaned_full) ---
print("Defining final sets and parameters...")
# --- Re-check START_GAMEWEEK against the actual cleaned data
   ---
T_setofgameweeks_full = sorted(data_cleaned_full['GW'].unique()
   ) # Update based on cleaned data
if START_GAMEWEEK not in T_setofgameweeks_full:
     available_gws_after_start = [gw for gw in
        T_setofgameweeks_full if gw >= START_GAMEWEEK]
     if not available_gws_after_start: print(f"ERROR:
        START_GAMEWEEK {START_GAMEWEEK} not in cleaned data
        range ({min(T_setofgameweeks_full)}-{max(
        T_setofgameweeks_full)}."); sys.exit()
     actual_start_gw = min(available_gws_after_start); print(f"
        Warning: START_GAMEWEEK {START_GAMEWEEK} adjusted to
        actual start {actual_start_gw}."); START_GAMEWEEK =
        actual_start_gw
     T_setofgameweeks_full = sorted([gw for gw in
        T_setofgameweeks_full if gw >= START_GAMEWEEK])
     if not T_setofgameweeks_full: print("ERROR: No gameweeks
        left after adjusting START_GAMEWEEK."); sys.exit()

p = sorted(data_cleaned_full['player_id'].unique())
```

```
l = list(range(1, 4))
final_player_info = data_cleaned_full.drop_duplicates(subset=['
    player_id'], keep='first')
player_name_map = final_player_info.set_index('player_id')['
    name'].to_dict()
pos_map = pd.Series(final_player_info['position'].values, index
    =final_player_info['player_id'])
C_setofteams_final = sorted(final_player_info['team'].unique())
Pgk = sorted([p_ for p_ in p if pos_map.get(p_) == "GK"])
Pdef = sorted([p_ for p_ in p if pos_map.get(p_) == "DEF"])
Pmid = sorted([p_ for p_ in p if pos_map.get(p_) == "MID"])
Pfwd = sorted([p_ for p_ in p if pos_map.get(p_) == "FWD"])
P_not_gk = sorted([p_ for p_ in p if p_ not in Pgk])
P_c_all = data_cleaned_full[data_cleaned_full['player_id'].isin
    (p)].groupby('team')['player_id'].unique().apply(list).
    to_dict()
P_c = {team: sorted(players) for team, players in P_c_all.items
    () if team in C_setofteams_final and players}
C_setofteams = sorted(list(P_c.keys()))
n_P = len(p); n_C = len(C_setofteams); n_L = len(l)
print(f"Sets defined for run: {len(T_setofgameweeks_full)} GWs
    ({min(T_setofgameweeks_full)}-{max(T_setofgameweeks_full)}),
     {n_P} Players, {n_C} Teams.")
print(f"  Positions: {len(Pgk)} GK, {len(Pdef)} DEF, {len(Pmid)
    } MID, {len(Pfwd)} FWD")

# Define GW subsets for WC logic
season_num = math.ceil(START_GAMEWEEK / 38)
gw1_of_this_season = (season_num - 1) * 38 + 1
mid_season_split_gw = gw1_of_this_season + 19 -1
T_FH_overall = [t_ for t_ in T_setofgameweeks_full if t_ <=
    mid_season_split_gw]
T_SH_overall = [t_ for t_ in T_setofgameweeks_full if t_ >
    mid_season_split_gw]
print(f"  Season {season_num} Halves (Absolute GWs): FH <= {
    mid_season_split_gw}, SH > {mid_season_split_gw}")
```

```
# Define Parameters
R_penalty = 4; MK = 2; MD = 5; MM = 5; MF = 3; MC = 3; E = 11;
   EK = 1
ED = 3; EM = 2; EF = 1; BS = 1000.0 # Scaled budget
phi = (MK + MD + MM + MF) - E; phi_K = MK - EK
Q_bar = 2; Q_under_bar = 1
epsilon = 0.1; kappa = {1: 0.01, 2: 0.005, 3: 0.001}
M_transfer = MK + MD + MM + MF
M_budget = BS + M_transfer * 200
M_alpha = M_transfer + Q_bar
M_q = Q_bar + 1
epsilon_q = 0.1
print("Parameters defined.")

# Prepare Coefficient Data Structures
print("Preparing full coefficient matrices...")
try:
    # Use data_cleaned_full which is now aggregated
    points_matrix_df = data_cleaned_full.pivot(index='player_id
       ', columns='GW', values='predicted_total_points')
    value_matrix_df = data_cleaned_full.pivot(index='player_id
       ', columns='GW', values='value')
    # Reindex using the final player set 'p' and the relevant
       gameweeks 'T_setofgameweeks_full'
    points_matrix_df = points_matrix_df.reindex(index=p,
       columns=T_setofgameweeks_full, fill_value=0.0)
    value_matrix_df = value_matrix_df.reindex(index=p, columns=
       T_setofgameweeks_full)
    # Forward/backward fill values within the reindexed matrix
    for player_id in p: value_matrix_df.loc[player_id] =
       value_matrix_df.loc[player_id].ffill().bfill()
    value_matrix_df.fillna(50.0, inplace=True) # Final fallback
    print("Full coefficient matrices ready.")
except Exception as e: print(f"ERROR creating full pivot tables
   : {e}"); sys.exit()
```

```
# --- Initialize State Variables ---
master_results = []
previous_squad_dict = {}
previous_budget = BS
previous_ft = 1
used_chips_tracker = {'wc1': False, 'wc2': False, 'bb': False,
   'tc': False, 'fh': False}

# --- Rolling Horizon Loop ---
print("\n--- Starting Rolling Horizon ---")
for current_gw in range(START_GAMEWEEK, MAX_GAMEWEEK + 1):
    if current_gw not in T_setofgameweeks_full:
        print(f"Skipping Gameweek {current_gw} as it's not in
           the loaded data range.")
        continue

    print(f"\n{'='*15} Solving for Gameweek {current_gw}
       {'='*15}")
    loop_start_time = time.time()

    # 1. Define Sub-Horizon Gameweeks
    t_sub = sorted([gw for gw in T_setofgameweeks_full if gw >=
        current_gw and gw < current_gw + SUB_HORIZON_LENGTH])
    if not t_sub: print(f"Sub-horizon empty for GW {current_gw
       }."); break
    n_T_sub = len(t_sub); print(f"Sub-horizon GWs: {t_sub}");
       t1_sub = t_sub[0]
    if t1_sub not in points_matrix_df.columns or t1_sub not in
       value_matrix_df.columns: print(f"ERROR: Missing
       coefficient data for GW {t1_sub}. Stopping."); break
    if not all(player_id in points_matrix_df.index for
       player_id in p): missing_p = [pid for pid in p if pid
       not in points_matrix_df.index]; print(f"ERROR: Players {
       missing_p} missing. Stopping."); break
```

```
# 2. Create New Model Instance
model = pulp.LpProblem(f"FPL_Opt_GW{current_gw}_Sub{
    SUB_HORIZON_LENGTH}", pulp.LpMaximize)

# 3. Define Variables for Sub-Horizon
print("Defining variables for sub-horizon...")
var_start = time.time()
x = pulp.LpVariable.dicts("Squad", (p, t_sub), cat='Binary
    ')
x_freehit = pulp.LpVariable.dicts("Squad_FH", (p, t_sub),
    cat='Binary')
y = pulp.LpVariable.dicts("Lineup", (p, t_sub), cat='Binary
    ')
f = pulp.LpVariable.dicts("Captain", (p, t_sub), cat='
    Binary')
h = pulp.LpVariable.dicts("ViceCaptain", (p, t_sub), cat='
    Binary')
is_tc = pulp.LpVariable.dicts("TripleCaptainChipActive", (p
    , t_sub), cat='Binary')
u = pulp.LpVariable.dicts("TransferOut", (p, t_sub), cat='
    Binary')
e = pulp.LpVariable.dicts("TransferIn", (p, t_sub), cat='
    Binary')
lambda_var = pulp.LpVariable.dicts("Aux_LineupInSquad", (p,
     t_sub), cat='Binary')
g = {}
if P_not_gk and l: g = pulp.LpVariable.dicts("Substitution
    ", (P_not_gk, t_sub, l), cat='Binary')
w = pulp.LpVariable.dicts("WildcardChipActive", t_sub, cat
    ='Binary')
b = pulp.LpVariable.dicts("BenchBoostChipActive", t_sub,
    cat='Binary')
r = pulp.LpVariable.dicts("FreeHitChipActive", t_sub, cat='
    Binary')
v = pulp.LpVariable.dicts("RemainingBudget", t_sub,
    lowBound=0, cat='Continuous')
```

```
q = pulp.LpVariable.dicts("FreeTransfersAvailable", t_sub,
    lowBound=0, upBound=Q_bar, cat='Integer')
alpha = pulp.LpVariable.dicts("PenalizedTransfers", t_sub,
    lowBound=0, upBound=M_alpha, cat='Integer')
ft_carried_over_nonneg = pulp.LpVariable.dicts("FT_Carry",
    t_sub, lowBound=0)
print(f"Variables defined in {time.time() - var_start:.2f}s
    ")

# 4. Filter Parameters for Sub-Horizon
points_sub = points_matrix_df.loc[p, t_sub]
value_sub = value_matrix_df.loc[p, t_sub]

# 5. Define Objective for Sub-Horizon
print("Defining objective function...")
obj_start = time.time()
points_from_lineup = pulp.lpSum(points_sub.loc[p_, t_] * y[
    p_][t_] for p_ in p for t_ in t_sub)
points_from_captain = pulp.lpSum(points_sub.loc[p_, t_] * f
    [p_][t_] for p_ in p for t_ in t_sub)
points_from_vice = pulp.lpSum(epsilon * points_sub.loc[p_,
    t_] * h[p_][t_] for p_ in p for t_ in t_sub)
points_from_tc = pulp.lpSum(2 * points_sub.loc[p_, t_] *
    is_tc[p_][t_] for p_ in p for t_ in t_sub)
points_from_subs = 0
if g: points_from_subs = pulp.lpSum(kappa[l_] * points_sub.
    loc[p_ngk][t_] * g[p_ngk][t_][l_] for p_ngk in P_not_gk
    for t_ in t_sub for l_ in l)
transfer_penalty = pulp.lpSum(R_penalty * alpha[t_] for t_
    in t_sub)
objective = (points_from_lineup + points_from_captain +
    points_from_vice +
              points_from_tc + points_from_subs -
                 transfer_penalty)
model += objective, "Total_Expected_Points_Sub"
```

```
print(f"Objective defined in {time.time() - obj_start:.2f}s
   ")

# 6. Define Constraints for Sub-Horizon
print("Adding constraints...")
cons_start = time.time()

# --- Gamechips (with Timing Restrictions) ---
print("  Adding Gamechip constraints (with timing)...")
# <<< This block correctly implements chip timing >>>
wc1_available = not used_chips_tracker['wc1']
wc2_available = not used_chips_tracker['wc2']
tc_available = not used_chips_tracker['tc']
bb_available = not used_chips_tracker['bb']
fh_available = not used_chips_tracker['fh']
for t_ in t_sub:
    wc1_target_is_none = TARGET_GW_WC1_OPPORTUNITY is None;
        wc2_target_is_none = TARGET_GW_WC2_OPPORTUNITY is
       None
    tc_target_is_none = TARGET_GW_TC is None;
       bb_target_is_none = TARGET_GW_BB is None;
       fh_target_is_none = TARGET_GW_FH is None
    # WC1
    is_in_first_half = t_ <= mid_season_split_gw
    if is_in_first_half:
        if not wc1_available or (not wc1_target_is_none and
            t_ != TARGET_GW_WC1_OPPORTUNITY): model += w[t_
          ] == 0, f"WC1_NotAvailableOrNotTarget_{t_}"
    else: model += w[t_] == 0, f"WC1_OnlyInFirstHalf_{t_}"
    # WC2
    is_in_second_half = t_ > mid_season_split_gw
    if is_in_second_half:
         if not wc2_available or (not wc2_target_is_none
            and t_ != TARGET_GW_WC2_OPPORTUNITY): model +=
            w[t_] == 0, f"WC2_NotAvailableOrNotTarget_{t_}"
    else: model += w[t_] == 0, f"WC2_OnlyInSecondHalf_{t_}"
```

```
        # TC
        if not tc_available or (not tc_target_is_none and t_ !=
            TARGET_GW_TC): model += pulp.lpSum(is_tc[p_][t_]
           for p_ in p) == 0, f"TC_NotAvailableOrNotTarget_{t_
           }"
        else: model += pulp.lpSum(is_tc[p_][t_] for p_ in p) <=
            1, f"TC_Possible_{t_}"
        # BB
        if not bb_available or (not bb_target_is_none and t_ !=
            TARGET_GW_BB): model += b[t_] == 0, f"
           BB_NotAvailableOrNotTarget_{t_}"
        # FH
        if not fh_available or (not fh_target_is_none and t_ !=
            TARGET_GW_FH): model += r[t_] == 0, f"
           FH_NotAvailableOrNotTarget_{t_}"
        # One Chip Per Week
        model += w[t_] + pulp.lpSum(is_tc[p_][t_] for p_ in p)
           + b[t_] + r[t_] <= 1, f"GC_OneChipPerWeek_{t_}"
# Overall Chip Limits
t_sub_in_fh_overall = [t for t in t_sub if t <=
   mid_season_split_gw]; t_sub_in_sh_overall = [t for t in
   t_sub if t > mid_season_split_gw]
if t_sub_in_fh_overall: model += pulp.lpSum(w[t_] for t_ in
    t_sub_in_fh_overall) <= (1 if wc1_available else 0), "
   WC_Limit_FH_Sub"
if t_sub_in_sh_overall: model += pulp.lpSum(w[t_] for t_ in
    t_sub_in_sh_overall) <= (1 if wc2_available else 0), "
   WC_Limit_SH_Sub"


# --- Squad, FH Squad, Lineup, Captain, Subs (Iterate over
   t_sub) ---
print("  Adding Squad, Lineup, Captain, Sub constraints
   ...")
# ... (These constraints remain the same - Correct) ...
squad_size_total = MK + MD + MM + MF
```

```
for t_ in t_sub:
    # Regular Squad (4.8 - 4.12)
    if Pgk: model += pulp.lpSum(x[p_gk][t_] for p_gk in Pgk
       ) == MK, f"Squad_GK_{t_}"
    if Pdef: model += pulp.lpSum(x[p_def][t_] for p_def in
       Pdef) == MD, f"Squad_DEF_{t_}"
    if Pmid: model += pulp.lpSum(x[p_mid][t_] for p_mid in
       Pmid) == MM, f"Squad_MID_{t_}"
    if Pfwd: model += pulp.lpSum(x[p_fwd][t_] for p_fwd in
       Pfwd) == MF, f"Squad_FWD_{t_}"
    for c_team in C_setofteams:
        players_in_team = P_c.get(c_team, [])
        if players_in_team: model += pulp.lpSum(x[p_tm][t_]
            for p_tm in players_in_team) <= MC, f"
           Squad_TeamLimit_{c_team}_{t_}"
    # Free Hit Squad (4.13 - 4.17)
    if Pgk: model += pulp.lpSum(x_freehit[p_gk][t_] for
       p_gk in Pgk) == MK * r[t_], f"FH_Squad_GK_{t_}"
    if Pdef: model += pulp.lpSum(x_freehit[p_def][t_] for
       p_def in Pdef) == MD * r[t_], f"FH_Squad_DEF_{t_}"
    if Pmid: model += pulp.lpSum(x_freehit[p_mid][t_] for
       p_mid in Pmid) == MM * r[t_], f"FH_Squad_MID_{t_}"
    if Pfwd: model += pulp.lpSum(x_freehit[p_fwd][t_] for
       p_fwd in Pfwd) == MF * r[t_], f"FH_Squad_FWD_{t_}"
    model += pulp.lpSum(x_freehit[p_][t_] for p_ in p) ==
       squad_size_total * r[t_], f"FH_Squad_TotalSize_{t_}"
    for c_team in C_setofteams:
        players_in_team = P_c.get(c_team, [])
        if players_in_team: model += pulp.lpSum(x_freehit[
           p_tm][t_] for p_tm in players_in_team) <= MC * r
           [t_], f"FH_Squad_TeamLimit_{c_team}_{t_}"
    # Starting Line-up (4.18 - 4.26)
    model += pulp.lpSum(y[p_][t_] for p_ in p) == E + phi *
        b[t_], f"Start_Size_{t_}"
    if Pgk: model += pulp.lpSum(y[p_gk][t_] for p_gk in Pgk
       ) == EK + phi_K * b[t_], f"Start_GK_{t_}"
```

```
if Pdef: model += pulp.lpSum(y[p_def][t_] for p_def in
   Pdef) >= ED, f"Start_MinDEF_{t_}"
if Pmid: model += pulp.lpSum(y[p_mid][t_] for p_mid in
   Pmid) >= EM, f"Start_MinMID_{t_}"
if Pfwd: model += pulp.lpSum(y[p_fwd][t_] for p_fwd in
   Pfwd) >= EF, f"Start_MinFWD_{t_}"
for p_ in p:
    model += y[p_][t_] <= x_freehit[p_][t_] +
       lambda_var[p_][t_], f"Start_InSquad_LinkA_{p_}_{
       t_}"
    model += lambda_var[p_][t_] <= x[p_][t_], f"
       Start_InSquad_LinkB_{p_}_{t_}"
    model += lambda_var[p_][t_] <= 1 - r[t_], f"
       Start_InSquad_LinkC_{p_}_{t_}"
# Captain/Vice (4.27 - 4.29)
model += pulp.lpSum(f[p_][t_] for p_ in p) + pulp.lpSum
   (is_tc[p_][t_] for p_ in p) == 1, f"
   Captain_Or_TC_Unique_{t_}"
model += pulp.lpSum(h[p_][t_] for p_ in p) == 1, f"
   ViceCaptain_Unique_{t_}"
for p_ in p:
    model += f[p_][t_] + is_tc[p_][t_] + h[p_][t_] <= y
       [p_][t_], f"Captaincy_In_Lineup_{p_}_{t_}"
    model += f[p_][t_] + h[p_][t_] <= 1, f"
       Cap_Not_Vice_{p_}_{t_}"
# Substitution (4.30 - 4.32)
if g:
    for p_ngk in P_not_gk:
        is_sub = pulp.lpSum(g[p_ngk][t_][l_] for l_ in
           l)
        model += y[p_ngk][t_] + is_sub <= x_freehit[
           p_ngk][t_] + lambda_var[p_ngk][t_], f"
           Sub_If_Benched_A_{p_ngk}_{t_}"
        model += is_sub <= 1 - y[p_ngk][t_], f"
           Sub_Only_If_Benched_B_{p_ngk}_{t_}"
    for l_ in l:
```

```
            model += pulp.lpSum(g[p_ngk][t_][l_] for p_ngk
                in P_not_gk) <= 1, f"Sub_Priority_Unique_{t_
                }_{l_}"
    # Transfer In/Out Limit (4.36)
    for p_ in p:
        model += e[p_][t_] + u[p_][t_] <= 1, f"
            Transfer_In_Out_Limit_{p_}_{t_}"
    # Explicit Transfer Balance
    model += pulp.lpSum(e[p_][t_] for p_ in p) == pulp.
        lpSum(u[p_][t_] for p_ in p), f"Transfer_Balance_{t_
        }"


# --- Linking Constraints & Evolution ---
print("  Adding Linking and Evolution constraints...")
t1_sub = t_sub[0] # First GW of this sub-problem

# Link q[t1_sub] to previous state
model += q[t1_sub] == previous_ft, f"FT_Link_{t1_sub}"

# Link budget and squad for t1_sub
if current_gw == START_GAMEWEEK:
    # Constraint 4.33: Initial budget limit (Use <= to
        allow saving budget)
    model += v[t1_sub] + pulp.lpSum(value_sub.loc[p_,
        t1_sub] * x[p_][t1_sub]) <= BS, f"Budget_Initial_{
        t1_sub}"
    # No transfers possible in GW1, force e=u=0
    model += pulp.lpSum(e[p_][t1_sub] for p_ in p) == 0, f"
        No_Transfers_In_StartGW"
    model += pulp.lpSum(u[p_][t1_sub] for p_ in p) == 0, f"
        No_Transfers_Out_StartGW"
else:
    # Link budget based on previous week's end budget (
        Constraint 4.34)
```

```
    sales_value_t1 = pulp.lpSum(value_sub.loc[p_, t1_sub] *
        u[p_][t1_sub] for p_ in p)
    purchase_cost_t1 = pulp.lpSum(value_sub.loc[p_, t1_sub]
        * e[p_][t1_sub] for p_ in p)
    model += v[t1_sub] == previous_budget + sales_value_t1
       - purchase_cost_t1, f"Budget_Link_{t1_sub}"
    # Link squad based on previous week's squad (Constraint
        4.35)
    for p_ in p:
        model += x[p_][t1_sub] == previous_squad_dict.get(
           p_, 0) - u[p_][t1_sub] + e[p_][t1_sub], f"
           Squad_Link_{p_}_{t1_sub}"

# Penalized Transfers for t1_sub
model += alpha[t1_sub] >= pulp.lpSum(e[p_][t1_sub] for p_
   in p) - q[t1_sub], f"PenalizedTransfers_Calc_{t1_sub}"
model += alpha[t1_sub] <= M_alpha * (1 - w[t1_sub]), f"
   PenalizedTransfers_WC_Override_{t1_sub}"
model += alpha[t1_sub] <= M_alpha * (1 - r[t1_sub]), f"
   PenalizedTransfers_FH_Override_{t1_sub}"

# Evolution constraints for t > t1_sub within the sub-
   horizon
for gw_idx in range(n_T_sub - 1):
    t_curr_sub = t_sub[gw_idx + 1]
    t_prev_sub = t_sub[gw_idx]

    # Budget Evolution (4.34)
    sales_value = pulp.lpSum(value_sub.loc[p_, t_curr_sub]
       * u[p_][t_curr_sub] for p_ in p)
    purchase_cost = pulp.lpSum(value_sub.loc[p_, t_curr_sub
       ] * e[p_][t_curr_sub] for p_ in p)
    model += v[t_curr_sub] == v[t_prev_sub] + sales_value -
        purchase_cost, f"Budget_Evolution_{t_curr_sub}"

    # Squad Continuity (4.35)
```

```
for p_ in p:
    model += x[p_][t_curr_sub] == x[p_][t_prev_sub] - u
       [p_][t_curr_sub] + e[p_][t_curr_sub], f"
       Squad_Continuity_{p_}_{t_curr_sub}"

# Free Hit Budget Limit (4.37)
cost_fh_squad_t = pulp.lpSum(value_sub.loc[p_,
   t_curr_sub] * x_freehit[p_][t_curr_sub] for p_ in p)
value_nonfh_squad_prev = pulp.lpSum(value_sub.loc[p_,
   t_prev_sub] * x[p_][t_prev_sub] for p_ in p)
model += cost_fh_squad_t <= v[t_prev_sub] +
   value_nonfh_squad_prev + M_budget * (1 - r[
   t_curr_sub]), f"FH_Budget_Limit_Upper_{t_curr_sub}"

# Free Hit Transfer Restriction (4.38 / 4.39)
model += pulp.lpSum(u[p_][t_curr_sub] for p_ in p) <=
   M_transfer * (1 - r[t_curr_sub]), f"
   FH_NoTransfersOut_{t_curr_sub}"
model += pulp.lpSum(e[p_][t_curr_sub] for p_ in p) <=
   M_transfer * (1 - r[t_curr_sub]), f"
   FH_NoTransfersIn_{t_curr_sub}"

# Penalized Transfers Calculation (for t_curr_sub)
model += alpha[t_curr_sub] >= pulp.lpSum(e[p_][
   t_curr_sub] for p_ in p) - q[t_curr_sub], f"
   PenalizedTransfers_Calc_{t_curr_sub}"
model += alpha[t_curr_sub] <= M_alpha * (1 - w[
   t_curr_sub]), f"PenalizedTransfers_WC_Override_{
   t_curr_sub}"
model += alpha[t_curr_sub] <= M_alpha * (1 - r[
   t_curr_sub]), f"PenalizedTransfers_FH_Override_{
   t_curr_sub}"

# Free Transfer Evolution (q[t_curr_sub] based on q[
   t_prev_sub])
```

```
        ft_used_effectively_prev = pulp.lpSum(e[p_][t_prev_sub]
            for p_ in p) - alpha[t_prev_sub]
        model += ft_carried_over_nonneg[t_prev_sub] >= q[
            t_prev_sub] - ft_used_effectively_prev, f"
            FT_Carry_Calc_{t_prev_sub}"
        model += ft_carried_over_nonneg[t_prev_sub] <= Q_bar -
            Q_under_bar, f"FT_Carry_Cap_{t_prev_sub}"

        chip_active_prev = w[t_prev_sub] + r[t_prev_sub]
        q_normal_calc = ft_carried_over_nonneg[t_prev_sub] +
            Q_under_bar

        model += q[t_curr_sub] <= q_normal_calc + M_q *
            chip_active_prev, f"FT_Evo_Normal_Upper_{t_curr_sub
            }"
        model += q[t_curr_sub] >= q_normal_calc - M_q *
            chip_active_prev, f"FT_Evo_Normal_Lower_{t_curr_sub
            }"
        model += q[t_curr_sub] <= Q_under_bar + M_q * (1 -
            chip_active_prev), f"FT_Evo_Chip_Reset_Upper_{
            t_curr_sub}"
        model += q[t_curr_sub] >= Q_under_bar - M_q * (1 -
            chip_active_prev), f"FT_Evo_Chip_Reset_Lower_{
            t_curr_sub}"

        # Constraint 4.42: No penalized transfers if max FT
            carried to next
        model += alpha[t_prev_sub] + M_alpha * q[t_curr_sub] <=
            M_alpha * Q_bar, f"Transfer_NoPaidIfMaxFT_{
            t_prev_sub}"


print(f"Constraints added in {time.time() - cons_start:.2f}
    s")

# 7. Solve Sub-Problem
```

```
print(f"[{time.strftime('%H:%M:%S')}] Starting solve for GW
    {current_gw} (Solver Internal Limit: {SOLVER_TIME_LIMIT
   }s)...")
print(f"[{time.strftime('%H:%M:%S')}] Solver output (msg=
   True) should appear below:")
solve_start_time = time.time()
solver_exception = None
# --- Direct solve call ---
try:
    print("Solver starting...")
    status = model.solve(solver_to_use)
    print("Solver finished.")
except Exception as e:
    solver_exception = e
    print(f"Solver encountered an error: {e}")
    status = pulp.LpStatusUndefined

solve_time = time.time() - solve_start_time

if solver_exception: print("\n--- Solver Error ---"); print
   (f"Error: {solver_exception}"); status = -1000
else: status = status

if status is None: print("\nERROR: Solver status not
   captured."); status = -1000

print(f"\n--- Solve Complete for GW {current_gw} ---")
print(f"Final Solver Status Code: {status}")
status_str = pulp.LpStatus.get(status, 'Unknown Status')
print(f"Final Solver Status: {status_str}")
print(f"Sub-problem Solve Time: {solve_time:.2f} seconds")
time_limit_hit = SOLVER_TIME_LIMIT is not None and
   solve_time >= SOLVER_TIME_LIMIT * 0.98

# 8. Extract Results and Update State
objective_value_extracted = None; use_fallback = False
```

```
try:
    if model.objective is not None:
       objective_value_extracted = model.objective.value()
except AttributeError: pass
solution_truly_optimal = (status == pulp.LpStatusOptimal)
solution_acceptable_timeout = (status == pulp.
   LpStatusNotSolved and time_limit_hit and
   objective_value_extracted is not None)
solution_acceptable = solution_truly_optimal or
   solution_acceptable_timeout

if not solution_acceptable:
    print(f"!!! Solver failed/timed out without acceptable
       solution for GW {current_gw} (Status: {status_str}).
        Implementing FALLBACK. !!!")
    use_fallback = True
    try:
         lp_filename = f"failed_model_gw{current_gw}.lp";
            model.writeLP(lp_filename)
         print(f"Model written to {lp_filename} for
            debugging.")
    except Exception as write_err: print(f"Could not write
       LP file: {write_err}")

# --- Implement decision or fallback ---
if not use_fallback:
    t1_sub = t_sub[0] # Gameweek to implement decisions for
    try:
        print(f"Extracting results for GW {current_gw} (t={
           t1_sub})...")
        gw_results = {'gameweek': current_gw}
        def get_var_val(var, default=0.0):
            try: val = var.varValue; return val if val is
               not None else default
            except AttributeError: return default
```

```
# --- Initialize previous_squad_dict in the first
   GW ---
if current_gw == START_GAMEWEEK:
      previous_squad_dict = {p_: 1 for p_ in p if
         get_var_val(x[p_][t1_sub]) > 0.9}
      print(f"Initialized previous_squad_dict with {
         len(previous_squad_dict)} players for GW{
         current_gw+1}")


# Extract decisions for t1_sub
gw_results['squad'] = sorted([p_ for p_ in p if
   get_var_val(x[p_][t1_sub]) > 0.9])
gw_results['lineup'] = sorted([p_ for p_ in p if
   get_var_val(y[p_][t1_sub]) > 0.9])
captain_id = None; tc_active_gw = False;
   tc_player_id = None
potential_captains = [p_ for p_ in p if get_var_val
   (f[p_][t1_sub]) > 0.9]
potential_tc = [p_ for p_ in p if get_var_val(is_tc
   [p_][t1_sub]) > 0.9]
if potential_tc: captain_id = potential_tc[0];
   tc_active_gw = True; tc_player_id = captain_id
elif potential_captains: captain_id =
   potential_captains[0]
gw_results['captain'] = [captain_id] if captain_id
   is not None else []
gw_results['vice_captain'] = sorted([p_ for p_ in p
    if get_var_val(h[p_][t1_sub]) > 0.9])
gw_results['transfers_in'] = sorted([p_ for p_ in p
    if get_var_val(e[p_][t1_sub]) > 0.9])
gw_results['transfers_out'] = sorted([p_ for p_ in
   p if get_var_val(u[p_][t1_sub]) > 0.9])
gw_results['budget_end'] = get_var_val(v[t1_sub],
   default=previous_budget)
```

```
alpha_val = get_var_val(alpha[t1_sub]); gw_results
   ['alpha'] = round(alpha_val)
gw_results['q_start'] = previous_ft # FT available
   at the start of this GW
gw_results['objective_value'] =
   objective_value_extracted # Total objective for
   sub-horizon
wc_active_gw = get_var_val(w[t1_sub]) > 0.9;
   bb_active_gw = get_var_val(b[t1_sub]) > 0.9;
   fh_active_gw = get_var_val(r[t1_sub]) > 0.9
chip_name_display = None
if wc_active_gw: chip_name_display = 'WC'
if bb_active_gw: chip_name_display = 'BB'
if fh_active_gw: chip_name_display = 'FH'
if tc_active_gw and tc_player_id is not None:
   tc_player_name = player_name_map.get(
   tc_player_id, f"ID:{tc_player_id}");
   chip_name_display = f'TC_{tc_player_name}'
gw_results['chip_played'] = chip_name_display
gw_results['budget_start'] = previous_budget #
   Store starting budget

# --- Calculate and Store Weekly Objective
   Contributions ---
weekly_objectives = {}
for t_calc in t_sub:
    points_lineup_t = sum(get_var_val(y[p_][t_calc
       ]) * points_sub.loc[p_, t_calc] for p_ in p)
    points_cap_t = sum(get_var_val(f[p_][t_calc]) *
        points_sub.loc[p_, t_calc] for p_ in p)
    points_vice_t = sum(get_var_val(h[p_][t_calc])
       * epsilon * points_sub.loc[p_, t_calc] for
       p_ in p)
    points_tc_t = sum(get_var_val(is_tc[p_][t_calc
       ]) * 2 * points_sub.loc[p_, t_calc] for p_
       in p)
```

```
        points_subs_t = 0
        if g: points_subs_t = sum(kappa[l_] *
           get_var_val(g[p_ngk][t_calc][l_]) *
           points_sub.loc[p_ngk, t_calc] for p_ngk in
           P_not_gk for l_ in l)
        alpha_t_val = get_var_val(alpha[t_calc])
        penalty_t = R_penalty * alpha_t_val
        weekly_obj_val = points_lineup_t + points_cap_t
            + points_vice_t + points_tc_t +
           points_subs_t - penalty_t
        weekly_objectives[t_calc] = weekly_obj_val
        print(f"  GW {t_calc} Calculated Objective
           Contribution: {weekly_obj_val:.4f} (Penalty:
            {penalty_t:.1f})")
    # Store the objective for the first week (the one
       being implemented)
    gw_results['objective_gw'] = weekly_objectives.get(
       t1_sub, None)

    master_results.append(gw_results)

    # --- State Update Logic ---
    chip_played_this_gw = gw_results['chip_played']
    next_ft_value = previous_ft # Default
    try:
        if chip_played_this_gw == 'WC' or
           chip_played_this_gw == 'FH':
            next_ft_value = Q_under_bar
        else: # BB, TC, or None
            q_current_val = gw_results['q_start']
            alpha_current_val = gw_results['alpha']
            e_current_sum = len(gw_results['
               transfers_in'])
            ft_used_current_eff = max(0, e_current_sum
               - alpha_current_val)
```

```
            ft_carry_current = max(0, q_current_val -
                ft_used_current_eff)
            next_ft_value = min(Q_bar, math.floor(
                ft_carry_current + Q_under_bar))
    except Exception as ft_err: print(f"Warning: Error
        calculating next FT: {ft_err}. Using default: {
        next_ft_value}")

    # Update persistent state (Squad/Budget)
    if fh_active_gw:
        print(f"FH played in GW {current_gw}. State (
            Squad/Budget) for next GW reverts.")
        previous_budget = gw_results['budget_start'] #
            Revert budget
    else:
        previous_squad_dict = {p_: 1 for p_ in
            gw_results['squad']}
        previous_budget = gw_results['budget_end'] #
            Update budget normally

    previous_ft = next_ft_value # Update FT for the *
        next* iteration

    # Update Chip Tracker
    if chip_played_this_gw:
         if chip_played_this_gw == 'WC':
             if current_gw <= mid_season_split_gw and
                not used_chips_tracker['wc1']:
                used_chips_tracker['wc1'] = True; print
                (f"--- WC1 activated in GW {current_gw}
                 ---")
             elif current_gw > mid_season_split_gw and
                not used_chips_tracker['wc2']:
                used_chips_tracker['wc2'] = True; print
                (f"--- WC2 activated in GW {current_gw}
                 ---")
```

```
                elif chip_played_this_gw == 'BB' and not
                    used_chips_tracker['bb']:
                    used_chips_tracker['bb'] = True; print(f
                    "--- BB activated in GW {current_gw} ---")
                elif chip_played_this_gw == 'FH' and not
                    used_chips_tracker['fh']:
                    used_chips_tracker['fh'] = True; print(f
                    "--- FH activated in GW {current_gw} ---")
                elif chip_played_this_gw.startswith('TC_') and
                     not used_chips_tracker['tc']:
                    used_chips_tracker['tc'] = True; print(f
                    "--- {chip_played_this_gw} activated ---")

            print(f"End of GW {current_gw}: Budget={
                previous_budget:.1f}, Next FT={previous_ft}")

        except Exception as extract_err:
            print(f"!!! Error extracting results for GW {
                current_gw}: {extract_err} !!!")
            import traceback; traceback.print_exc()
            print("Stopping rolling horizon due to extraction
                error."); break

    else: # --- Handle Fallback Case ---
        print(f"Applying Fallback for GW {current_gw}: No
            transfers made.")
        next_ft_value = min(Q_bar, math.floor(previous_ft +
            Q_under_bar))
        gw_results = {
            'gameweek': current_gw, 'squad': sorted(list(
                previous_squad_dict.keys())),
            'lineup': [], 'captain': [], 'vice_captain': [],
            'transfers_in': [], 'transfers_out': [], '
                budget_end': previous_budget,
            'alpha': 0, 'q_start': previous_ft, '
                objective_value': None,
```

```
            'chip_played': 'FALLBACK_NO_TRANSFERS', '
               budget_start': previous_budget,
            'objective_gw': None # No objective calculated in
               fallback
        }
        master_results.append(gw_results)
        previous_ft = next_ft_value # State update for FT
        print(f"End of GW {current_gw} (Fallback): Budget={
           previous_budget:.1f}, Next FT={previous_ft}")

    # --- Loop continuation logic ---
    print(f"Total time for GW {current_gw} loop: {time.time() -
        loop_start_time:.2f}s")
    print("-" * 50)


# --- Post-Loop: Process and Display Results ---
print("\n" + "="*20 + " Final Rolling Horizon Results " +
   "="*20)
if not master_results:
    print("No results were generated.")
else:
    results_df = pd.DataFrame(master_results)
    # --- Add weekly objective to the summary printout ---
    def map_ids_to_names(id_list):
        if not isinstance(id_list, (list, tuple, set)): return
           id_list
        if not id_list: return []
        return sorted([player_name_map.get(p_id, f"ID:{p_id}")
           for p_id in id_list])
    id_list_columns = ['squad', 'lineup', 'captain', '
       vice_captain', 'transfers_in', 'transfers_out']
    print("\nConverting player IDs to names in results
       DataFrame...")
    results_df_named = results_df.copy()
    for col in id_list_columns:
```

```
    if col in results_df_named.columns:
        results_df_named[col] = results_df_named[col].apply
           (map_ids_to_names)
print("ID to Name conversion complete.")

# Add this after the line "results_df_named = results_df.
   copy()" in the final results processing section

# Calculate actual points if available in the source data
if 'actual_total_points' in data_full_range_raw.columns:
    print("\nCalculating actual points from results...")
    # Create pivot table for actual points lookup
    actual_points_matrix = data_full_range_raw.pivot(index
       ='player_id', columns='GW', values='
       actual_total_points')

    # Add actual points columns to results
    results_df_named['actual_squad_points'] = 0
    results_df_named['actual_lineup_points'] = 0
    results_df_named['actual_captain_points'] = 0  # This
       will store ONLY the bonus points
    results_df_named['actual_total_points'] = 0    # New
       field for the correct total

    for idx, row in results_df_named.iterrows():
        gw = row['gameweek']
        squad_ids = results_df.loc[idx, 'squad']
        lineup_ids = results_df.loc[idx, 'lineup']
        captain_ids = results_df.loc[idx, 'captain']

        # Calculate base points for all players
        squad_actual = sum(actual_points_matrix.loc[p_id,
           gw] if p_id in actual_points_matrix.index and gw
            in actual_points_matrix.columns and not pd.isna
           (actual_points_matrix.loc[p_id, gw]) else 0 for
           p_id in squad_ids)
```

```
lineup_actual = sum(actual_points_matrix.loc[p_id,
   gw] if p_id in actual_points_matrix.index and gw
    in actual_points_matrix.columns and not pd.isna
   (actual_points_matrix.loc[p_id, gw]) else 0 for
   p_id in lineup_ids)

# Calculate captain BONUS points (not total)
captain_actual_bonus = 0
if captain_ids:
    captain_id = captain_ids[0]
    if captain_id in actual_points_matrix.index and
        gw in actual_points_matrix.columns and not
       pd.isna(actual_points_matrix.loc[captain_id,
        gw]):
        captain_base_pts = actual_points_matrix.loc
           [captain_id, gw]

        # Only calculate the BONUS points (1x or 2x
           )
        if row['chip_played'] and row['chip_played
           '].startswith('TC_'):
            captain_actual_bonus = captain_base_pts
                * 2  # 2x bonus for TC
        else:
            captain_actual_bonus = captain_base_pts
                 # 1x bonus for regular captain

# Add bench points if Bench Boost was used
bench_pts = 0
if row['chip_played'] == 'BB':
    bench_players = [p_id for p_id in squad_ids if
       p_id not in lineup_ids]
    bench_pts = sum(actual_points_matrix.loc[p_id,
       gw] if p_id in actual_points_matrix.index
       and gw in actual_points_matrix.columns and
       not pd.isna(actual_points_matrix.loc[p_id,
```

```
                gw]) else 0 for p_id in bench_players)
            lineup_actual += bench_pts  # Add bench to
                lineup when BB active

        # Store the values
        results_df_named.at[idx, 'actual_squad_points'] =
            squad_actual
        results_df_named.at[idx, 'actual_lineup_points'] =
            lineup_actual
        results_df_named.at[idx, 'actual_captain_points'] =
             captain_actual_bonus
        results_df_named.at[idx, 'actual_total_points'] =
            lineup_actual + captain_actual_bonus

# Print summary per GW
for index, row in results_df_named.iterrows():
    gw = row['gameweek']; print(f"\n--- GW {gw} Summary
        ---")
    print(f"  Chip Played: {row['chip_played'] if row['
        chip_played'] else 'None'}")
    transfers_in_str = ', '.join(map(str, row['transfers_in
        ']))
    transfers_out_str = ', '.join(map(str, row['
        transfers_out']))
    print(f"  Transfers In ({len(row['transfers_in'])}): {
        transfers_in_str}")
    print(f"  Transfers Out ({len(row['transfers_out'])}):
        {transfers_out_str}")
    print(f"  FT Available (Start): {row['q_start']}")
    print(f"  Penalized Transfers (Hits): {row['alpha']}")
    cap_name = row['captain'][0] if row['captain'] else '
        None'; vice_name = row['vice_captain'][0] if row['
        vice_captain'] else 'None'
    print(f"  Captain: {cap_name}"); print(f"  Vice-Captain
        : {vice_name}")
    print(f"  Budget End: {row['budget_end']:.1f}")
```

```
        obj_val_str = f"{row['objective_value']:.2f}" if row['
            objective_value'] is not None else "N/A"; print(f"
            Objective Value (Sub-problem Total): {obj_val_str}")
        # --- Print the calculated weekly objective for this GW
            ---
        obj_gw_str = f"{row['objective_gw']:.2f}" if row['
            objective_gw'] is not None else "N/A (Fallback or
            Error)"; print(f"  Objective Value (GW {gw}
            Contribution): {obj_gw_str}")
        if 'actual_lineup_points' in results_df_named.columns:
            print(f"  Actual Points (Squad): {row['
                actual_squad_points']:.1f}")
            print(f"  Actual Points (Lineup): {row['
                actual_lineup_points']:.1f}")
            print(f"  Captain Bonus Points: {row['
                actual_captain_points']:.1f}")
            print(f"  Actual Total Points: {row['
                actual_total_points']:.1f}")


# Save results
try:
    output_csv_name = f"Squad Selection t-auto, W{
        START_GAMEWEEK}-{MAX_GAMEWEEK},SHL{
        SUB_HORIZON_LENGTH}.csv"
    results_df_to_save = results_df_named.copy() # Use
        named df for saving
    for col in id_list_columns:
        if col in results_df_to_save.columns:
            results_df_to_save[col] = results_df_to_save[
                col].apply(lambda x: ', '.join(map(str, x))
                    if isinstance(x, list) else x)
    # Add the weekly objective column before saving
    results_df_to_save['objective_gw'] = results_df['
        objective_gw'] # Add the numeric column from
        original results
```

```
        results_df_to_save.to_csv(output_csv_name, index=False)
           ; print(f"\nResults saved to {output_csv_name}")
    except Exception as save_e: print(f"\nERROR: Could not save
        results to CSV. Reason: {save_e}")

print("\n--- Script Finished ---")
```

# E Squad Results R Plots

```
rm(list = ls(all = TRUE))
# Load required libraries
library(tidyverse)
library(xtable)

# Set to git folder
setwd("C:/Users/peram/Documents/test")

# Create an output directory
output_dir <- "C:/Users/peram/Documents/test/MILP Py/
   output_plots"

# Create the directory if it doesn't exist
if (!dir.exists(output_dir)) {
  dir.create(output_dir)
}

# Post train residuals
df <- read_csv("Validation_Predictions_Clean_v2.csv")
colnames(df)
res <- df |>
  mutate(residuals = actual_total_points -
     predicted_total_points) |>
  mutate(ressqr = residuals^2)

# Create a residuals distribution plot by position
# First, ensure residuals are calculated
```

```
res <- res |>
  mutate(residuals = actual_total_points -
     predicted_total_points) |>
  mutate(ressqr = residuals^2)

# Ensure position is ordered the same way as in EDA
res <- res |>
  mutate(position = factor(position, levels = c("GK", "DEF", "
     MID", "FWD")))

# Create density plots faceted by position with matching color
   scheme
position_residuals_plot <- ggplot(res, aes(x = residuals, fill
   = position, color = position)) +
  geom_density(alpha = 0.2) +  # Changed to 0.2 to match EDA
  facet_wrap(~ position) +
  scale_color_brewer(palette = "Set1") +  # Added to match EDA
  scale_fill_brewer(palette = "Set1") +   # Added to match EDA
  labs(title = "Distribution of Prediction Residuals by Player
     Position",
       x = "Residuals (Actual - Predicted)",
       y = "Density") +
  theme_grey() +
  geom_vline(xintercept = 0, linetype = "dashed", color = "red
     ") +
  theme(legend.position = "bottom")

# Display the plot
print(position_residuals_plot)

# Save the plot to the output directory
ggsave(file.path(output_dir, "position_residuals_distribution.
   png"),
       plot = position_residuals_plot,
       width = 10, height = 8)
```

```
# Calculate MSE and RMSE by position - UPDATED to include mean
   residuals
mse_rmse_by_position <- res |>
  group_by(position) |>
  summarize(
    n = n(),
    mean_residual = mean(residuals),  # Add mean of raw
       residuals
    mse = mean(ressqr),
    rmse = sqrt(mean(ressqr)),
    mean_actual = mean(actual_total_points),
    mean_predicted = mean(predicted_total_points)
  ) |>
  ungroup()

# Calculate overall MSE and RMSE - UPDATED to include mean
   residuals
overall_mse_rmse <- res |>
  summarize(
    position = "ALL",
    n = n(),
    mean_residual = mean(residuals),  # Add mean of raw
       residuals
    mse = mean(ressqr),
    rmse = sqrt(mean(ressqr)),
    mean_actual = mean(actual_total_points),
    mean_predicted = mean(predicted_total_points)
  )

# Combine position-specific and overall metrics
all_metrics <- bind_rows(mse_rmse_by_position, overall_mse_rmse
   ) |>
  arrange(position != "ALL", mse)

# Print the results
print(all_metrics)
```

```
# Create a formatted table
metrics_table <- xtable::xtable(
  all_metrics,
  caption = "Post Training Metrics by Player Position",
  digits = c(0, 0, 0, 2, 2, 2, 2)
)
# Save the xtable output to a text file
capture.output(
  print(metrics_table,
        include.rownames = FALSE,
        caption.placement = "top"),
  file = file.path(output_dir, "position_error_metrics.txt")
)

# Forecasted ----
# Set working directory (adjust path as needed)
setwd("C:/Users/peram/Documents/test/MILP Py")

# Initialize an empty data frame to store combined data
combined_data_lstm <- data.frame()

# Loop through the n files
for (i in 1:10) {
  file_name <- paste0("Squad Selection t-auto, W77-108,SHL", i,
      ".csv")
  temp_data <- read.csv(file_name)
  temp_data <- temp_data %>%
    select(gameweek, objective_gw, actual_total_points, alpha)
       %>%
    mutate(sub_horizon = paste0("SHL", i))
  combined_data_lstm <- rbind(combined_data_lstm, temp_data)
}

# FIX actual_total_points to be net
colnames(combined_data_lstm)
```

```
transfer_penalty <- 4
combined_data_lstm <- combined_data_lstm |>
  mutate(actual_total_points = actual_total_points - alpha*
     transfer_penalty)

# --- FIX SUB-HORIZON ORDER FOR LSTM ---
# Extract numeric part and order
horizon_numbers_lstm <- as.integer(stringr::str_extract(unique(
   combined_data_lstm$sub_horizon), "\\d+"))
ordered_levels_lstm <- paste0("SHL", sort(horizon_numbers_lstm)
   )

combined_data_lstm <- combined_data_lstm %>%
  mutate(sub_horizon = factor(sub_horizon, levels =
     ordered_levels_lstm))


# Reshape data to long format for ggplot
plot_data_lstm <- combined_data_lstm %>%
  pivot_longer(cols = c(objective_gw, actual_total_points),
               names_to = "line_type",
               values_to = "points") %>%
  mutate(line_type = recode(line_type,
                            "objective_gw" = "Forecasted Squad
                               Points",
                            "actual_total_points" = "Actual
                               Squad Points"))  # FIXED:
                               actual_lineup_points to
                               actual_total_points

# Create the original line plot - fixed syntax error with
   line_type
p1_lstm <- ggplot(plot_data_lstm, aes(x = gameweek, y = points,
    color = sub_horizon, linetype = line_type)) +
  geom_line(linewidth = 0.8) +
  labs(title = "Gameweek Points by Sub-Horizon, Squads from
```

```
     forecasts with actual squad points",
       x = "Gameweek (GW)",
       y = "Points",
       color = "Sub-Horizon",
       linetype = "Line Type") +
  theme_grey() +
  theme(legend.position = "right")
print(p1_lstm)
ggsave(file.path(output_dir, "fantasy_points_plot_lstm.png"),
   plot = p1_lstm, width = 10, height = 6) # Increased width
   for legend

# --- CUMULATIVE PLOT FOR LSTM DATA ---
cumulative_plot_data_lstm <- combined_data_lstm %>%
  group_by(sub_horizon) %>%
  arrange(gameweek) %>%
  mutate(
    forecast_cumu = cumsum(objective_gw),
    actual_cumu = cumsum(actual_total_points)
  ) %>%
  ungroup()

# Create long format for cumulative plot with both metrics
cumulative_plot_data_lstm_long <- cumulative_plot_data_lstm %>%
  pivot_longer(
    cols = c(forecast_cumu, actual_cumu),
    names_to = "cumulative_type",
    values_to = "cumulative_points"
  ) %>%
  mutate(cumulative_type = recode(cumulative_type,
                                  "forecast_cumu" = "Cumulative
                                     Forecasted Squad Points",
                                  "actual_cumu" = "Cumulative
                                     Actual Squad Points"))

# Updated cumulative plot for LSTM
```

```
p_cumulative_lstm <- ggplot(cumulative_plot_data_lstm_long,
       aes(x = gameweek, y = cumulative_points, color =
          sub_horizon, linetype = cumulative_type)) +
  geom_line(linewidth = 0.8) +
  labs(title = "Cumulative Points by Sub-Horizon (LSTM)",
       x = "Gameweek (GW)",
       y = "Cumulative Points",
       color = "Sub-Horizon",
       linetype = "Point Type") +
  theme_grey() +
  theme(legend.position = "right")
print(p_cumulative_lstm)
ggsave(file.path(output_dir, "
   cumulative_fantasy_points_plot_lstm.png"), plot =
   p_cumulative_lstm, width = 10, height = 6)

# Extract end values for LSTM - include both metrics
end_values_lstm <- cumulative_plot_data_lstm %>%
  group_by(sub_horizon) %>%
  filter(gameweek == max(gameweek)) %>%
  select(
    sub_horizon,
    end_gameweek = gameweek,
    final_forecasted_points = forecast_cumu,
    final_actual_points = actual_cumu
  ) %>%
  ungroup()
cat("\n--- End Cumulative Values for LSTM Data ---\n")

end_values_lstm_x <- print(xtable::xtable(end_values_lstm))


# Actual ----
#setwd("C:/Users/peram/Documents/test/MILP Py")
combined_data_actual <- data.frame()
```

```
# Loop through the n files
for (i in 1:10) {
  file_name <- paste0("Squad Selection t-auto-actual, W77-108,
     SHL", i, ".csv")
  temp_data <- read.csv(file_name)
  temp_data <- temp_data %>%
    select(gameweek, actual_total_points, alpha) %>%
    mutate(sub_horizon = paste0("SHL", i))
  combined_data_actual <- rbind(combined_data_actual, temp_data
     )
}
# Hot fix
combined_data_actual <- combined_data_actual |>
  mutate(actual_total_points = actual_total_points - alpha*
     transfer_penalty)


# Extract numeric part and order
horizon_numbers_actual <- as.integer(stringr::str_extract(
   unique(combined_data_actual$sub_horizon), "\\d+"))
ordered_levels_actual <- paste0("SHL", sort(
   horizon_numbers_actual))

combined_data_actual <- combined_data_actual %>%
  mutate(sub_horizon = factor(sub_horizon, levels =
     ordered_levels_actual))


plot_data_actual <- combined_data_actual %>%
  pivot_longer(cols = actual_total_points,
               names_to = "line_type",
               values_to = "points") %>%
  mutate(line_type = recode(line_type,
                            "actual_total_points" = "Actual
                               Points"))
```

```
p1_actual <- ggplot(plot_data_actual, aes(x = gameweek, y =
   points, color = sub_horizon, linetype = line_type)) +
  geom_line(linewidth = 0.8) +
  labs(title = "Optimised Squad Points Gameweek by Sub-Horizon,
      with actual data",
       x = "Gameweek (GW)",
       y = "Points",
       color = "Sub-Horizon",
       linetype = "Line Type") +
  theme_grey() +
  theme(legend.position = "right")
print(p1_actual)
ggsave(file.path(output_dir, "fantasy_points_plot_actual.png"),
    plot = p1_actual, width = 10, height = 6)


# --- CUMULATIVE PLOT FOR ACTUAL DATA ---
cumulative_plot_data_actual <- combined_data_actual %>% # Use
   already factor-ordered combined_data_actual
  group_by(sub_horizon) %>%
  arrange(gameweek) %>%
  mutate(forecast_cumu = cumsum(actual_total_points)) %>%
  ungroup()

p_cumulative_actual <- ggplot(cumulative_plot_data_actual, aes(
   x = gameweek, y = forecast_cumu, color = sub_horizon)) +
  geom_line(linewidth = 0.5) +
  labs(title = "Cumulative Optimal Squad Points by Sub-Horizon,
      with actual data",
       x = "Gameweek (GW)",
       y = "Cumulative Points",
       color = "Sub-Horizon") +
  theme_grey() +
  theme(legend.position = "right")
print(p_cumulative_actual)
ggsave(file.path(output_dir, "
```

```
   cumulative_fantasy_points_plot_actual.png"), plot =
   p_cumulative_actual, width = 10, height = 6)

end_values_actual <- cumulative_plot_data_actual %>%
  group_by(sub_horizon) %>%
  filter(gameweek == max(gameweek)) %>%
  select(sub_horizon, end_gameweek = gameweek,
     final_cumulative_points = forecast_cumu) %>%
  ungroup()
cat("\n--- End Cumulative Values for Actual Data ---\n")

end_values_actual_x <- print(xtable::xtable(end_values_actual))

# FORCED GAMECHIPS SUB in range(1,5) for predicted ----
combined_data_forced <- data.frame()

# Loop through the 5 files with forced gamechips
for (i in 1:5) {
  file_name <- paste0("Squad Selection t-auto-hard, W77-108,SHL
     ", i, ".csv")
  temp_data <- read.csv(file_name)
  temp_data <- temp_data %>%
    select(gameweek, objective_gw, actual_total_points, alpha)
       %>%  # Added actual_total_points
    mutate(sub_horizon = paste0("SHL", i))
  combined_data_forced <- rbind(combined_data_forced, temp_data
     )
}
# Hot fix
combined_data_forced <- combined_data_forced |>
  mutate(actual_total_points = actual_total_points - alpha*
     transfer_penalty)


# Fix sub-horizon order for forced gamechips data
horizon_numbers_forced <- as.integer(stringr::str_extract(
```

```
   unique(combined_data_forced$sub_horizon), "\\d+"))
ordered_levels_forced <- paste0("SHL", sort(
   horizon_numbers_forced))

combined_data_forced <- combined_data_forced %>%
  mutate(sub_horizon = factor(sub_horizon, levels =
     ordered_levels_forced))

# Create line plot for forced gamechips data with both metrics
plot_data_forced <- combined_data_forced %>%
  pivot_longer(cols = c(objective_gw, actual_total_points),  #
     Added actual_total_points
               names_to = "line_type",
               values_to = "points") %>%
  mutate(line_type = recode(line_type,
                            "objective_gw" = "Forecasted Squad
                               Points",
                            "actual_total_points" = "Actual
                               Squad Points"))

p1_forced <- ggplot(plot_data_forced, aes(x = gameweek, y =
   points, color = sub_horizon, linetype = line_type)) +
  geom_line(linewidth = 0.8) +
  labs(title = "Gameweek Points by Sub-Horizon (Forced
     Gamechips)",
       x = "Gameweek (GW)",
       y = "Points",
       color = "Sub-Horizon",
       linetype = "Line Type") +
  theme_grey() +
  theme(legend.position = "right")
print(p1_forced)
ggsave(file.path(output_dir, "fantasy_points_plot_forced.png"),
    plot = p1_forced, width = 10, height = 6)

# Create cumulative plot for forced gamechips data with both
```

```
   metrics
cumulative_plot_data_forced <- combined_data_forced %>%
  group_by(sub_horizon) %>%
  arrange(gameweek) %>%
  mutate(
    forecast_cumu = cumsum(objective_gw),
    actual_cumu = cumsum(actual_total_points)  # Added actual
       cumulative
  ) %>%
  ungroup()

# Create long format for cumulative plot
cumulative_plot_data_forced_long <- cumulative_plot_data_forced
    %>%
  pivot_longer(
    cols = c(forecast_cumu, actual_cumu),
    names_to = "cumulative_type",
    values_to = "cumulative_points"
  ) %>%
  mutate(cumulative_type = recode(cumulative_type,
                                  "forecast_cumu" = "Cumulative
                                     Forecasted Squad Points",
                                  "actual_cumu" = "Cumulative
                                     Actual Squad Points"))

p_cumulative_forced <- ggplot(cumulative_plot_data_forced_long,
       aes(x = gameweek, y = cumulative_points, color =
          sub_horizon, linetype = cumulative_type)) +
  geom_line(linewidth = 0.8) +
  labs(title = "Cumulative Points by Sub-Horizon (Forced
     Gamechips)",
       x = "Gameweek (GW)",
       y = "Cumulative Points",
       color = "Sub-Horizon",
       linetype = "Point Type") +
  theme_grey() +
```

```
  theme(legend.position = "right")
print(p_cumulative_forced)
ggsave(file.path(output_dir, "
   cumulative_fantasy_points_plot_forced.png"), plot =
   p_cumulative_forced, width = 10, height = 6)

# Extract end values for forced gamechips - include both
   metrics
end_values_forced <- cumulative_plot_data_forced %>%
  group_by(sub_horizon) %>%
  filter(gameweek == max(gameweek)) %>%
  select(
    sub_horizon,
    end_gameweek = gameweek,
    final_forecasted_points = forecast_cumu,
    final_actual_points = actual_cumu
  ) %>%
  ungroup()

# ACTUAL FORCED GAMECHIPS SUB in range(1,5)
combined_data_actual_forced <- data.frame()

# Loop through the 5 files with actual forced gamechips
for (i in 1:5) {
  file_name <- paste0("Squad Selection t-auto-forced-actual,
     W77-108,SHL", i, ".csv")
  temp_data <- read.csv(file_name)
  temp_data <- temp_data %>%
    select(gameweek, actual_total_points, alpha) %>%  # Only
       using actual points here
    mutate(sub_horizon = paste0("SHL", i))
  combined_data_actual_forced <- rbind(
     combined_data_actual_forced, temp_data)
}
# Hot fix
combined_data_actual_forced <- combined_data_actual_forced |>
```

```
  mutate(actual_total_points = actual_total_points - alpha*
     transfer_penalty)

# Fix sub-horizon order for actual forced gamechips data
horizon_numbers_actual_forced <- as.integer(stringr::
   str_extract(unique(combined_data_actual_forced$sub_horizon),
    "\\d+"))
ordered_levels_actual_forced <- paste0("SHL", sort(
   horizon_numbers_actual_forced))

combined_data_actual_forced <- combined_data_actual_forced %>%
  mutate(sub_horizon = factor(sub_horizon, levels =
     ordered_levels_actual_forced))

# Create line plot for actual forced gamechips data
plot_data_actual_forced <- combined_data_actual_forced %>%
  pivot_longer(cols = actual_total_points,
               names_to = "line_type",
               values_to = "points") %>%
  mutate(line_type = recode(line_type,
                            "actual_total_points" = "Actual
                               Squad Points"))

p1_actual_forced <- ggplot(plot_data_actual_forced, aes(x =
   gameweek, y = points, color = sub_horizon, linetype =
   line_type)) +
  geom_line(linewidth = 0.8) +
  labs(title = "Gameweek Points by Sub-Horizon (Actual Forced
     Gamechips)",
       x = "Gameweek (GW)",
       y = "Points",
       color = "Sub-Horizon",
       linetype = "Line Type") +
  theme_grey() +
  theme(legend.position = "right")
print(p1_actual_forced)
```

```
ggsave(file.path(output_dir, "fantasy_points_plot_actual_forced
   .png"), plot = p1_actual_forced, width = 10, height = 6)

# Create cumulative plot for actual forced gamechips data
cumulative_plot_data_actual_forced <-
   combined_data_actual_forced %>%
  group_by(sub_horizon) %>%
  arrange(gameweek) %>%
  mutate(actual_cumu = cumsum(actual_total_points)) %>%
  ungroup()

p_cumulative_actual_forced <- ggplot(
   cumulative_plot_data_actual_forced, aes(x = gameweek, y =
   actual_cumu, color = sub_horizon)) +
  geom_line(linewidth = 0.5) +
  labs(title = "Cumulative Actual Squad Points by Sub-Horizon (
     Actual Forced Gamechips)",
       x = "Gameweek (GW)",
       y = "Cumulative Actual Points",
       color = "Sub-Horizon") +
  theme_grey() +
  theme(legend.position = "right")
print(p_cumulative_actual_forced)
ggsave(file.path(output_dir, "
   cumulative_fantasy_points_plot_actual_forced.png"), plot =
   p_cumulative_actual_forced, width = 10, height = 6)

# Extract end values for actual forced gamechips
end_values_actual_forced <- cumulative_plot_data_actual_forced
   %>%
  group_by(sub_horizon) %>%
  filter(gameweek == max(gameweek)) %>%
  select(
    sub_horizon,
    end_gameweek = gameweek,
    final_actual_points = actual_cumu
```

```
  ) %>%
  ungroup()

# Update the combined comparison table to include the new model
all_end_values <- bind_rows(
  mutate(end_values_lstm, model_type = "LSTM"),
  mutate(end_values_actual %>%
           rename(final_forecasted_points =
              final_cumulative_points) %>%
           mutate(final_actual_points = NA),
         model_type = "Actual"),
  mutate(end_values_forced, model_type = "Forced"),
  # Add actual_forced with NA for forecasted points to match
     column structure
  mutate(end_values_actual_forced %>%
           mutate(final_forecasted_points = NA),
         model_type = "Actual Forced")
) %>%
  # Reorder columns for better readability
  select(model_type, sub_horizon, end_gameweek,
     final_forecasted_points, final_actual_points)

# Create a comprehensive table
comprehensive_xtable <- xtable::xtable(
  all_end_values,
  caption = "Comprehensive Comparison of All Models and Sub-
     Horizons",
  digits = c(0, 0, 0, 0, 0, 0)  # Format decimal places
)

# Save as a single text file with proper formatting
capture.output(
  print(comprehensive_xtable,
        include.rownames = FALSE,
        caption.placement = "bottom"),
  file = file.path(output_dir, "comprehensive_model_comparison.
```

```
    txt")
)

# Create a separate well-formatted table for the residual
   analysis - UPDATED for new column
residual_xtable <- xtable(
  all_metrics,
  caption = "Prediction Error Metrics by Position",
  digits = c(0, 0, 0, 2, 2, 2, 2,2)  # Added digit for new
     column
)

# Save the residual analysis to a separate file
capture.output(
  print(residual_xtable,
        include.rownames = FALSE,
        caption.placement = "bottom"),
  file = file.path(output_dir, "detailed_residual_analysis.txt
     ")
)
```

# F R-Script 1: Fetching Data

```
# Hente og slå sammen full tibble for sesong 22/23, 23/24 og
   24/25.
rm(list = ls(all = TRUE))

library(tidyverse)
library(stringi)

# Henting av data =====

# Hent data. Sesong 22 og 23 er tilgjengelig fra vastav sin
   github
s22 <- read_csv("https://raw.githubusercontent.com/vaastav/
   Fantasy-Premier-League/refs/heads/master/data/2022-23/gws/
   merged_gw.csv",
                locale = locale(encoding = "UTF-8"))
s23 <- read_csv("https://raw.githubusercontent.com/vaastav/
   Fantasy-Premier-League/refs/heads/master/data/2023-24/gws/
   merged_gw.csv",
                locale = locale(encoding = "UTF-8"))

# Ettersom sesong 24 introduserte assistent manager, må vi
   skaffe dataen ukentlig fra vaastav, og fjerne de
# ekstra kollonnene som assistent mangaerene introduserer.
# Funksjon for å hente ukentlig data for sesong 24/25
hent_ukentlig <- function(gw) {
  url <- paste0("https://raw.githubusercontent.com/vaastav/
     Fantasy-Premier-League/master/data/2024-25/gws/gw", gw, ".
     csv")
  data <- read_csv(url, locale = locale(encoding = "UTF-8"))
  return(data)
}

# Hent data for gameweeks 1 til 31 (18.04.2025)
sistegw <- 31
for (i in 1:sistegw) {
```

```
  # Hent data for hver gameweek
  gw_data <- hent_ukentlig(i)

  # Lagre dataen i separate dataframes
  assign(str_c("gameweek", i), gw_data, envir = .GlobalEnv)
}

# Finn og fjern ekstra kollonner
# Lagre kolonnenavn fra første sesong som referanse
colnames(s22);colnames(gameweek28)
kolonner <- s22 |> select(1:40) |> colnames() # fjern GW,
   ettersom ukentlig ikke har GW

# Loop gjennom alle gameweeks og fjern ekstra kolonner
for (i in 1:sistegw) {
  # Hent data for hver gameweek
  gw_data <- get(str_c("gameweek", i))

  # Behold bare kolonnene som finnes i gameweek1
  behold <- gw_data |> select(all_of(kolonner[1:40]))

  # Lagre den rensed dataen tilbake til gameweek-variabelen
  assign(str_c("gameweek", i), behold, envir = .GlobalEnv)
}

# Sjekk
sum(colnames(gameweek1) == colnames(gameweek28)) # 40

# Sett sammen alle gameweeks til en dataframe
sesong_2425 <- tibble()

# Løkke for å sette sammen alle gameweeks
for (i in 1:sistegw) {
  # Trekk ut df navn
  df_name <- str_c("gameweek", i)
  df <- get(df_name)
```

```
  # Legg til GW kolonnen fra tidligere sesonger
  if (!"GW" %in% colnames(df)) {
    df <- df |> mutate(GW = i)
  }

  # Legg sammen til df
  sesong_2425 <- bind_rows(sesong_2425, df)
}

glimpse(sesong_2425)

# Gjøre om alt fra utf 8 til latin aasci
sesong2223_latinsk <- s22 |> mutate(name = stri_trans_general(
   name, "latin-ascii"))
sesong2324_latinsk <- s23 |> mutate(name = stri_trans_general(
   name, "latin-ascii"))
sesong2425_latinsk <- sesong_2425 |> mutate(name =
   stri_trans_general(name, "latin-ascii"))

sesong2223_latinsk |> select(GW) |> unique() |> print(n = 50) #
    Dronning Elisabeth II døde denne sesongen, derav ingen gw 7
sesong2324_latinsk |> select(GW) |> unique() |> print(n = 50)
sesong2425_latinsk |> select(GW) |> unique() |> print(n = 50)

## Fikse navn ====
## Siden det er noen navnefeil så må vi fikse det
riktigenavn <- c(
  "Mitoma Kaoru" = "Kaoru Mitoma",
  "Tomiyasu Takehiro" = "Takehiro Tomiyasu",
  "Endo Wataru" = "Wataru Endo",
  "Kim Ji-Soo" = "Ji-Soo Kim",
  "Olu Aina" = "Ola Aina",
  "Dominic Solanke-Mitchell" = "Dominic Solanke",
  "Kaine Kesler-Hayden" = "Kaine Kesler Hayden",
  "Adama Traore Diarra" = "Adama Traore",
```

```
  "Omari Giraud-Hutchinson" = "Omari Hutchinson",
  "Joe Gomez" = "Joseph Gomez",
  "Rodrigo 'Rodri' Hernandez" = "Rodrigo Hernandez",
  "Yehor Yarmoliuk" = "Yegor Yarmoliuk",
  "Michale Olakigbe" = "Michael Olakigbe",
  "Djordje Petrovic" = "Dorde Petrovic",
  "Joshua King" = "Josh King",
  "Ben Brereton" = "Ben Brereton Diaz",
  "Jaden Philogene" = "Jaden Philogene-Bidace",
  "Carlos Alcaraz" = "Carlos Alcaraz Duran",
  "Luis Sinisterra" = "Luis Sinisterra Lucumi",
  "Joe Aribo" = "Joe Ayodele-Aribo",
  "Joseph Hodge" = "Joe Hodge",
  "Tom Cannon" = "Thomas Cannon",
  "Yerson Mosquera" = "Yerson Mosquera Valdelamar",
  "Max Kinsey" = "Max Kinsey-Wellings",
  "Alexandre Moreno Lopera" = "Alex Moreno Lopera",
  "Joe Whitworth" = "Joseph Whitworth"
)

Sesong_22_23 <- sesong2223_latinsk |>
  mutate(name = str_replace_all(name, riktigenavn))
Sesong_23_24 <- sesong2324_latinsk |>
  mutate(name = str_replace_all(name, riktigenavn))
Sesong_24_25 <- sesong2425_latinsk |>
  mutate(name = str_replace_all(name, riktigenavn))

# Ben Davies
Sesong_22_23 <- Sesong_22_23 %>%
  mutate(name = if_else(name == "Ben Davies" & team == "
     Liverpool",
                        "Ben Davies Liverpool", name))
Sesong_23_24 <- Sesong_23_24 %>%
  mutate(name = if_else(name == "Ben Davies" & team == "
     Liverpool",
                        "Ben Davies Liverpool", name))
```

```
Sesong_24_25 <- Sesong_24_25 %>%
  mutate(name = if_else(name == "Ben Davies" & team == "
     Liverpool",
                        "Ben Davies Liverpool", name))

#View(Sesong_24_25)

# Sjekk
dim(Sesong_22_23)
dim(Sesong_23_24)
dim(Sesong_24_25)

sum(is.na(Sesong_22_23) |> sum()) + (is.na(Sesong_23_24) |> sum
   ()) + (is.na(Sesong_24_25) |> sum())

# Fiks lagnavn
lag2022 <- c(
  "Arsenal", "Aston Villa", "Bournemouth", "Brentford", "
     Brighton",
  "Chelsea", "Crystal Palace", "Everton", "Fulham", "Leeds",
  "Leicester", "Liverpool", "Man City", "Man United", "
     Newcastle",
  "Nottingham Forest", "Southampton", "Tottenham", "West Ham",
     "Wolves"
)
Sesong_22_23 <- Sesong_22_23 %>%
  mutate(opponent_team = lag2022[opponent_team])

Sesong_22_23 <- Sesong_22_23 %>%
  mutate(team = recode(team,
                       "Spurs" = "Tottenham",
                       "Nott'm Forest" = "Nottingham Forest",
                       "Man Utd" = "Man United",
                       "Leicester City" = "Leicester"))

lag2023 <- c(
```

```
  "Arsenal", "Aston Villa", "Bournemouth", "Brentford", "
     Brighton",
  "Burnley", "Chelsea", "Crystal Palace", "Everton", "Fulham",
  "Liverpool", "Luton", "Man City", "Man United", "Newcastle",
  "Nottingham Forest", "Sheffield Utd", "Tottenham", "West Ham
     ", "Wolves"
)
Sesong_23_24 <- Sesong_23_24 %>%
  mutate(opponent_team = lag2023[opponent_team])
Sesong_23_24 <- Sesong_23_24 %>%
  mutate(team = recode(team,
                       "Spurs" = "Tottenham",
                       "Nott'm Forest" = "Nottingham Forest",
                       "Man Utd" = "Man United"
  ))

lag2024 <- c(
  "Arsenal", "Aston Villa", "Bournemouth", "Brentford", "
     Brighton",
  "Chelsea", "Crystal Palace", "Everton", "Fulham", "Ipswich",
  "Leicester", "Liverpool", "Man City", "Man United", "
     Newcastle",
  "Nottingham Forest", "Southampton", "Tottenham", "West Ham",
     "Wolves"
)
Sesong_24_25 <- Sesong_24_25 %>%
  mutate(opponent_team = as.numeric(opponent_team)) %>%
  mutate(opponent_team = lag2024[opponent_team])
Sesong_24_25 <- Sesong_24_25 %>%
  mutate(team = recode(team,
                       "Spurs" = "Tottenham",
                       "Man Utd" = "Man United",
                       "Nott'm Forest" = "Nottingham Forest"
  ))
```

```
# Fjern Element Kolonner, denne inneholder ID til spillere men
   varierer fra sesong til sesong

Sesong_22_23 <- Sesong_22_23 |> select(-element)
Sesong_23_24 <- Sesong_23_24 |> select(-element)
Sesong_24_25 <- Sesong_24_25 |> select(-element)

# Sjekk
dim(Sesong_22_23);dim(Sesong_23_24);dim(Sesong_24_25) # 40

# Fjern Assistent Managere fra datasettet.

# Define on-field positions
onfield_positions <- c("GK", "DEF", "MID", "FWD")

# Identify assistant managers as those with positions not in
   onfield_positions
assistant_managers <- Sesong_24_25 %>%
  filter(!(position %in% onfield_positions)) %>%
  pull(name) %>%
  unique()

# Display identified assistant managers
cat("Identified", length(assistant_managers), "assistant
   managers\n")

# Filter out assistant managers from the dataset
Sesong_24_25 <- Sesong_24_25 %>%
  filter(!(name %in% assistant_managers))

# Sjekk om noen assistentmanagere fortsatt finnes i datasettet
Sesong_24_25 |> filter(name %in% assistant_managers)

# Sammenslåing og Lagring ====

## Sammenslåing og numerifisering ====
```

```
### Egen kolonne med Sesong ID og Spiller ID ====
sesongogspiller_id <- bind_rows("Sesong 22/23" = Sesong_22_23,
   "Sesong 23/24" = Sesong_23_24, "Sesong 24/25" = Sesong_24_25
   , .id = "season")

# Numerifisering

sesong_ogspiller_id <- sesong_ogspiller_id |>
  mutate(player_id = as.integer(factor(name)))

sesong_ogspiller_id <- sesong_ogspiller_id %>%
  mutate(was_home = as.integer(was_home))

### Datasett med GW i stigende rekkefølge ====
Alt_22_23 <- Sesong_22_23
Alt_23_24 <- Sesong_23_24 |> mutate(GW = GW + 38)
Alt_24_25 <- Sesong_24_25 |> mutate(GW = GW + 76)

# Sammenslåing
stigenderekkefølge <- bind_rows(Alt_22_23, Alt_23_24, Alt_24_25
   )

# Numerifisering

stigenderekkefølge <- stigenderekkefølge %>%
  mutate(was_home = as.integer(was_home))

stigenderekkefølge <- stigenderekkefølge %>%
  mutate(player_id = as.integer(factor(name)))

### Alternativt ====
alternativtsammensatt <- bind_rows(Alt_22_23, Alt_23_24,
   Alt_24_25)

alternativtsammensatt <- alternativtsammensatt %>%
```

```
  mutate(
    kickoff_time = as.POSIXct(kickoff_time, format = "%Y-%m-%dT
       %H:%M:%SZ", tz = "UTC"),
    player_id = as.integer(factor(name)),          # Unique
       global player IDs
    oID = as.integer(factor(opponent_team)),  # Unique opponent
        IDs
    tID = as.integer(factor(team)),           # Unique team IDs
    hID = as.integer(was_home)                # Home/away
       indicator
  ) %>%
  arrange(player_id, kickoff_time) %>%  # Arrange by the new
     pID and kickoff_time
  mutate(row_id = row_number())   # Create a row identifier

# Lagre data =====

## Sesongvis ====
write_csv(Sesong_22_23, file = "Datasett/Sesong 22 til 23.csv")
write_csv(Sesong_23_24, file = "Datasett/Sesong 23 til 24.csv")
write_csv(Sesong_24_25, file = "Datasett/Sesong 24 til 25.csv")
cat("Lagret filene\n")

## Samlet ====

# Samlet sett
write_csv(sesongogspiller_id, file = "Datasett/Sesong id, alle
   tre sesonger(22-24).csv")
cat("Lagret filen\n")

# Stigende rekkefølger GW
write_csv(stigenderekkefølge, "Datasett/Stigende GW, alle tre
   sesonger(22-24).csv")

# Alternativt Datasett
write_csv(alternativtsammensatt, "Datasett/Ekstra kolonner,
```

```
   stigende GW, alle tre sesonger(22-24), heltall.csv")
cat("Lagret alternativ filen\n")
```

# G R-Script 2: Exploratory Data Analysis

```
# EDA ----
# Analysis of point distributions for players based on
   different criteria
rm(list = ls(all = TRUE))
# Load required packages for statistical description
if (!require("pacman")) install.packages("pacman")
pacman::p_load(pastecs,tidyverse,ggridges, xtable, moments,
qqplotr)

# Create directory for saving plots
setwd("C:/Users/peram/Documents/test")
plot_directory <- "C:/Users/peram/Documents/test/R Forecast/EDA
   "
dir.create(plot_directory, recursive = TRUE)



# Data Fetch ----
season_22_23 <- read_csv("Datasett/Sesong 22 til 23.csv")
season_23_24 <- read_csv("Datasett/Sesong 23 til 24.csv")
season_24_25 <- read_csv("Datasett/Sesong 24 til 25.csv")

# Define position order for consistent plotting
position_levels <- c("GK", "DEF", "MID", "FWD")

# Starting Players ----

## Create datasets of starting players ----

starters_22_23 <- season_22_23 |>
  filter(starts == 1) |>
  select(total_points, position, GW) |>
```

```
  mutate(position = factor(position, levels = position_levels))

starters_23_24 <- season_23_24 |>
  filter(starts == 1) |>
  select(total_points, position, GW) |>
  mutate(position = factor(position, levels = position_levels))

starters_24_25 <- season_24_25 |>
  filter(starts == 1) |>
  select(total_points, position, GW) |>
  mutate(position = factor(position, levels = position_levels))

## Display dataset structure ----
glimpse(starters_22_23)

## Create boxplots for point distributions by position for
   starters ----

plot1 <- ggplot(starters_22_23, aes(x = position, y =
   total_points)) +
  geom_boxplot() +
  labs(title = "Boxplot for starting players in season 22/23 by
      position",
       x = "Position",
       y = "Observed points per week") +
  scale_y_continuous(breaks = seq(0, max(
     starters_22_23$total_points, na.rm = TRUE), 2))

plot2 <- ggplot(starters_23_24, aes(x = position, y =
   total_points)) +
  geom_boxplot() +
  labs(title = "Boxplot for starting players in season 23/24 by
      position",
       x = "Position",
       y = "Observed points per week") +
  scale_y_continuous(breaks = seq(0, max(
```

```
    starters_23_24$total_points, na.rm = TRUE), 2))

plot3 <- ggplot(starters_24_25, aes(x = position, y =
   total_points)) +
  geom_boxplot() +
  labs(title = "Boxplot for starting players in season 24/25 by
      position",
       x = "Position",
       y = "Observed points per week") +
  scale_y_continuous(breaks = seq(0, max(
    starters_24_25$total_points, na.rm = TRUE), 2))

## Display box plots ----
plot1;plot2;plot3

## Distribution density plots for starting players ----
# Create distribution plots with different colors by position
dist_plot1 <- ggplot(starters_22_23, aes(x = total_points, fill
    = position, color = position)) +
  geom_density(alpha = 0.2) +
  scale_color_brewer(palette = "Set1") +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "Point distribution for starting players in
    season 22/23 by position",
       x = "Total points",
       y = "Density") +
  theme_grey() +
  scale_x_continuous(breaks = seq(0, max(
    starters_22_23$total_points, na.rm = TRUE), 2))

dist_plot2 <- ggplot(starters_23_24, aes(x = total_points, fill
    = position, color = position)) +
  geom_density(alpha = 0.2) +
  scale_color_brewer(palette = "Set1") +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "Point distribution for starting players in
```

```
    season 23/24 by position",
      x = "Total points",
      y = "Density") +
  theme_grey() +
  scale_x_continuous(breaks = seq(0, max(
    starters_23_24$total_points, na.rm = TRUE), 2))

dist_plot3 <- ggplot(starters_24_25, aes(x = total_points, fill
   = position, color = position)) +
  geom_density(alpha = 0.2) +
  scale_color_brewer(palette = "Set1") +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "Point distribution for starting players in
    season 24/25 by position",
      x = "Total points",
      y = "Density") +
  theme_grey() +
  scale_x_continuous(breaks = seq(0, max(
    starters_24_25$total_points, na.rm = TRUE), 2))

## Display density plots for starters
dist_plot1;dist_plot2;dist_plot3

# Atleaset 1 min of play time ----

## Create datasets of players with at least 1 minute ----
players_played_1 <- season_22_23 |>
  filter(minutes >= 1) |>
  select(total_points, position) |>
  mutate(position = factor(position, levels = position_levels))

players_played_2 <- season_23_24 |>
  filter(minutes >= 1) |>
  select(total_points, position) |>
  mutate(position = factor(position, levels = position_levels))
```

```
players_played_3 <- season_24_25 |>
  filter(minutes >= 1) |>
  select(total_points, position) |>
  mutate(position = factor(position, levels = position_levels))

## Create boxplots for players with atleast 1 min playtime ----
plot4 <- ggplot(players_played_1, aes(x = position, y =
   total_points)) +
  geom_boxplot() +
  labs(title = "Boxplots for players with at least minute of
     play time by position (22/23)",
       x = "Position",
       y = "Observed points per week") +
  scale_y_continuous(breaks = seq(0, max(
     players_played_1$total_points, na.rm = TRUE), 2))

plot5 <- ggplot(players_played_2, aes(x = position, y =
   total_points)) +
  geom_boxplot() +
  labs(title = "Boxplots for players with at least minute of
     play time by position (23/24)",
       x = "Position",
       y = "Observed points per week") +
  scale_y_continuous(breaks = seq(0, max(
     players_played_2$total_points, na.rm = TRUE), 2))

plot6 <- ggplot(players_played_3, aes(x = position, y =
   total_points)) +
  geom_boxplot() +
  labs(title = "Boxplots for players with at least minute of
     play time by position (24/25)",
       x = "Position",
       y = "Observed points per week") +
  scale_y_continuous(breaks = seq(0, max(
     players_played_3$total_points, na.rm = TRUE), 2))
```

```
# Display plots
plot4;plot5;plot6

## Create Distribution plots for players with atleast 1 minute
   ----
dist_plot4 <- ggplot(players_played_1, aes(x = total_points,
   fill = position, color = position)) +
  geom_density(alpha = 0.2) +
  scale_color_brewer(palette = "Set1") +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "Point distribution for players with at least
     minute (22/23)",
       x = "Total points",
       y = "Density") +
  theme_grey() +
  scale_x_continuous(breaks = seq(0, max(
     players_played_1$total_points, na.rm = TRUE), 2))

dist_plot5 <- ggplot(players_played_2, aes(x = total_points,
   fill = position, color = position)) +
  geom_density(alpha = 0.2) +
  scale_color_brewer(palette = "Set1") +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "Point distribution for players with at least
     minute (23/24)",
       x = "Total points",
       y = "Density") +
  theme_grey() +
  scale_x_continuous(breaks = seq(0, max(
     players_played_2$total_points, na.rm = TRUE), 2))

dist_plot6 <- ggplot(players_played_3, aes(x = total_points,
   fill = position, color = position)) +
  geom_density(alpha = 0.2) +
  scale_color_brewer(palette = "Set1") +
  scale_fill_brewer(palette = "Set1") +
```

```
  labs(title = "Point distribution for players with at least
     minute (24/25)",
       x = "Total points",
       y = "Density") +
  theme_grey() +
  scale_x_continuous(breaks = seq(0, max(
     players_played_3$total_points, na.rm = TRUE), 2))

# Display density plots for players with atleast 1 min of play
   time
dist_plot4;dist_plot5;dist_plot6

# All players ----
## Create datasets of all players ----
all1 <- season_22_23 |>
  select(total_points, position) |>
  mutate(position = factor(position, levels = position_levels))
all2 <- season_23_24 |>
  select(total_points, position) |>
  mutate(position = factor(position, levels = position_levels))
all3 <- season_24_25 |>
  select(total_points, position) |>
  mutate(position = factor(position, levels = position_levels))
      |>
  na.omit()

## Create boxplots for all players ----

plot7 <- ggplot(all1, aes(x = position, y = total_points)) +
  geom_boxplot() +
  labs(title = "Boxplots for all players by position (22/23)",
       x = "Position",
       y = "Observed points per week") +
  scale_y_continuous(breaks = seq(0, max(all1$total_points, na.
     rm = TRUE), 2))
```

```
plot8 <- ggplot(all2, aes(x = position, y = total_points)) +
  geom_boxplot() +
  labs(title = "Boxplots for all players by position (23/24)",
       x = "Position",
       y = "Observed points per week") +
  scale_y_continuous(breaks = seq(0, max(all2$total_points, na.
    rm = TRUE), 2))

plot9 <- ggplot(all3, aes(x = position, y = total_points)) +
  geom_boxplot() +
  labs(title = "Boxplots for all players by position (24/25)",
       x = "Position",
       y = "Observed points per week") +
  scale_y_continuous(breaks = seq(0, max(all3$total_points, na.
    rm = TRUE), 2))

plot7;plot8;plot9

## Create distribution density plots for all players ----

dist_plot7 <- ggplot(all1, aes(x = total_points, fill =
   position, color = position)) +
  geom_density(alpha = 0.2) +
  scale_color_brewer(palette = "Set1") +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "Point distribution for all players by position
    (22/23)",
       x = "Total points",
       y = "Density") +
  theme_grey() +
  scale_x_continuous(breaks = seq(0, max(
    players_played_1$total_points, na.rm = TRUE), 2))

dist_plot8 <- ggplot(all2, aes(x = total_points, fill =
   position, color = position)) +
  geom_density(alpha = 0.2) +
```

```
  scale_color_brewer(palette = "Set1") +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "Point distribution for all players by position
     (23/24)",
       x = "Total points",
       y = "Density") +
  theme_grey() +
  scale_x_continuous(breaks = seq(0, max(all2$total_points, na.
     rm = TRUE), 2))

dist_plot9 <- ggplot(all3, aes(x = total_points, fill =
    position, color = position)) +
  geom_density(alpha = 0.2) +
  scale_color_brewer(palette = "Set1") +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "Point distribution for all players by position
     (24/25)",
       x = "Total points",
       y = "Density") +
  theme_grey() +
  scale_x_continuous(breaks = seq(0, max(all3$total_points, na.
     rm = TRUE), 2))

dist_plot7;dist_plot8;dist_plot9

# Export plots ----

## Save plots for starting players ----

## Save boxplots ----
ggsave(file.path(plot_directory, "Box Plot for starting players
   (POS) in season 22-23.png"),
       plot = plot1, width = 8, height = 6)
ggsave(file.path(plot_directory, "Box Plot for starting players
   (POS) in season 23-24.png"),
       plot = plot2, width = 8, height = 6)
```

```
ggsave(file.path(plot_directory, "Box Plot for starting players
   (POS) in season 24-25.png"),
       plot = plot3, width = 8, height = 6)

### Save density plots for starters ----
ggsave(file.path(plot_directory, "Point density distribution
   for starting players 22-23.png"),
       plot = dist_plot1, width = 10, height = 6)
ggsave(file.path(plot_directory, "Point density distribution
   for starting players 23-24.png"),
       plot = dist_plot2, width = 10, height = 6)
ggsave(file.path(plot_directory, "Point density distribution
   for starting players 24-25.png"),
       plot = dist_plot3, width = 10, height = 6)

##  Save plots for 1+ minutes players ----


### Save boxplots plots ----
ggsave(file.path(plot_directory, "Boxplots for players with 1+
   playing time 22-23 by position.png"),
       plot = plot3, width = 8, height = 6)
ggsave(file.path(plot_directory, "Boxplots for players with 1+
   playing time 23-24 by position.png"),
       plot = plot4, width = 8, height = 6)
ggsave(file.path(plot_directory, "Boxplots for players with 1+
   playing time 24-25 by position.png"),
       plot = plot4, width = 8, height = 6)

### Save density plots for 1+ minute players ----
ggsave(file.path(plot_directory, "Point density distribution
   for players with 1min+ 22-23 by position.png"),
       plot = dist_plot4, width = 10, height = 6)
ggsave(file.path(plot_directory, "Point density distribution
   for players with 1min+ 23-24 by position.png"),
       plot = dist_plot5, width = 10, height = 6)
```

```
ggsave(file.path(plot_directory, "Point density distribution
   for players with 1min+ 24-25 by position.png"),
       plot = dist_plot6, width = 10, height = 6)

## Save plots for all players ----

### Save boxplots ----
ggsave(file.path(plot_directory, "Boxplots for all players by
   position 22-23.png"),
       plot = plot7, width = 8, height = 6)
ggsave(file.path(plot_directory, "Boxplots for all players by
   position 23-24.png"),
       plot = plot8, width = 8, height = 6)
ggsave(file.path(plot_directory, "Boxplots for all players by
   position 24-25.png"),
       plot = plot9, width = 8, height = 6)

### Save density plots for all players ----
ggsave(file.path(plot_directory, "Point density distribution
   for all players 22-23 by position.png"),
       plot = dist_plot7, width = 10, height = 6)
ggsave(file.path(plot_directory, "Point density distribution
   for all players 23-24 by position.png"),
       plot = dist_plot8, width = 10, height = 6)
ggsave(file.path(plot_directory, "Point density distribution
   for all players 24-25 by position.png"),
       plot = dist_plot9, width = 10, height = 6)

# Comparison by position across seasons ----

## For starters ----
combined_starters <- bind_rows(
  mutate(starters_22_23, season = "22/23"),
  mutate(starters_23_24, season = "23/24"),
  mutate(starters_24_25, season = "24/25")
)
```

```
box_compare_starter <- ggplot(combined_starters, aes(x =
   total_points, color = season)) +
  geom_boxplot() +
  facet_wrap(~ position, scales = "free_y") +
  scale_color_manual(values = c("22/23" = "blue" , "23/24" = "
     navy", "24/25" = "grey")) +
  labs(title = "Comparing boxplots across seasons for starting
     players",
       x = "Total points",
       y = "Density") +
  theme_grey()

box_compare_starter

### Create Ridgeline Plot by Gameweek and Position ----

# Generate the ridgeline plot

gameweek_pos_ridgeline <- ggplot(combined_starters,
                                 aes(x = total_points,
                                     y = factor(GW), # Treat
                                        gameweek as a
                                        categorical variable on
                                         y-axis
                                     fill = factor(season))) +
                                        # Fill by position
  geom_density_ridges(alpha = 0.4) + # Use geom_density_ridges
     for overlapping densities
  scale_fill_manual(values = c("22/23" = "yellow", # Define
     colors for each position
                               "23/24" = "red",
                               "24/25" = "blue")) +
  labs(title = "Distribution of Total Points by Season for
     Starters",
       x = "Total Points",
```

```
        y = "Gameweek") +
  theme_ridges() + # Apply a ridgeline theme
  theme_grey()
  theme(legend.position = "bottom") # Position the legend at
     the bottom

# Display the plot and save ridge for all players
gameweek_pos_ridgeline

#save
ggsave(file.path(plot_directory, "
   Gameweek_Ridgeline_Plot_by_Position.png"),
        plot = gameweek_pos_ridgeline, width = 12, height = 10)

## For 1+ minute players ----
combined_1_min <- bind_rows(
  mutate(players_played_1, season = "22/23"),
  mutate(players_played_2, season = "23/24"),
  mutate(players_played_3, season = "24/25")
)

box_compare_1_min <- ggplot(combined_1_min, aes(x =
   total_points, color = season)) +
  geom_boxplot() +
  facet_wrap(~ position, scales = "free_y") +
  scale_color_manual(values = c("22/23" = "blue", "23/24" = "
     navy", "24/25" = "grey")) +
  labs(title = "Comparing boxplots across seasons for players
     with at least minutes by position",
        x = "Total points",
        y = "Density") +
  theme_grey()

box_compare_1_min

## For all players ----
```

```
combined_all <- bind_rows(
  mutate(all1, season = "22/23"),
  mutate(all2, season = "23/24"),
  mutate(all3, season = "24/25")
)

box_compare_all <- ggplot(combined_all, aes(x = total_points,
   color = season)) +
  geom_boxplot() +
  facet_wrap(~ position, scales = "free_y") +
  scale_color_manual(values = c("22/23" = "blue", "23/24" = "
     navy", "24/25" = "grey")) +
  labs(title = "Comparing boxplots across seasons for all
     players by position",
       x = "Total points",
       y = "Density") +
  theme_grey()

box_compare_all


# Save the combined comparison plots
ggsave(file.path(plot_directory, "Season comparison starters by
    position.png"),
       plot = box_compare_starter, width = 10, height = 8)
ggsave(file.path(plot_directory, "Season comparison at least 1
   min by position.png"),
       plot = box_compare_1_min, width = 10, height = 8)
ggsave(file.path(plot_directory, "Season comparison all players
    by position.png"),
       plot = box_compare_all, width = 10, height = 8)


# Descriptives tables ----
# Function to calculate comprehensive descriptive statistics
```

```
calc_stats <- function(data) {
  stats <- stat.desc(data)
  # Add skewness and kurtosis
  stats["skewness"] <- skewness(data, na.rm = TRUE)
  stats["kurtosis"] <- kurtosis(data, na.rm = TRUE)
  return(stats)
}

# Calculate statistics for all groups
stats_starter_1 <- calc_stats(starters_22_23$total_points)
stats_starter_2 <- calc_stats(starters_23_24$total_points)
stats_starter_3 <- calc_stats(starters_24_25$total_points)

stats_1min_1 <- calc_stats(players_played_1$total_points)
stats_1min_2 <- calc_stats(players_played_2$total_points)
stats_1min_3 <- calc_stats(players_played_3$total_points)

stats_all_1 <- calc_stats(all1$total_points)
stats_all_2 <- calc_stats(all2$total_points)
stats_all_3 <- calc_stats(all3$total_points)

# Create comparison dataframes
stats_comparison <- data.frame(
  Metric = names(stats_starter_1),
  "Starting players (22/23)" = stats_starter_1,
  "Starting players (23/24)" = stats_starter_2,
  "Starting players (24/25)" = stats_starter_3
)

stats_comparison2 <- data.frame(
  Metric = names(stats_1min_1),
  "Minimum 1 minute players (22/23)" = stats_1min_1,
  "Minimum 1 minute players (23/24)" = stats_1min_2,
  "Minimum 1 minute players (24/25)" = stats_1min_3
)
```

```
stats_comparison3 <- data.frame(
  Metric = names(stats_all_1),
  "All players (22/23)" = stats_all_1,
  "All players (23/24)" = stats_all_2,
  "All players (24/25)" = stats_all_3
)

print(stats_comparison)
print(stats_comparison2)
print(stats_comparison3)

# Create normal text tables for all three stats comparison
   dataframes
# For stats_comparison (starting players)
print(xtable(stats_comparison,
caption = "Descriptive Statistics for Starting Players"),
file = file.path(plot_directory, "
   Descriptive_statistics_starters.txt"),
include.rownames = FALSE,
floating = TRUE,
latex.environments = "center",
type = "latex")

# For stats_comparison2 (players with at least 1 minute)
print(xtable(stats_comparison2,
caption = "Descriptive Statistics for Players with At Least 1
   Minute"),
file = file.path(plot_directory, "
   Descriptive_statistics_1min_players.txt"),
include.rownames = FALSE,
floating = TRUE,
latex.environments = "center",
type = "latex")

  # For stats_comparison3 (all players)
print(xtable(stats_comparison3,
```

```
caption = "Descriptive Statistics for All Players"),
file = file.path(plot_directory, "
   Descriptive_statistics_all_players.txt"),
include.rownames = FALSE,
floating = TRUE,
latex.environments = "center",
type = "latex")

# QQ Plots to check normality (using ggplot) ----

## Function to create ggplot QQ plot with theme_grey
create_qq_plot <- function(data, title) {
  ggplot(data = data.frame(x = data), aes(sample = x)) +
    stat_qq_point(color = "blue", size = 2) +
    stat_qq_line(color = "red", size = 1) +
    labs(title = title,
         x = "Theoretical Quantiles",
         y = "Sample Quantiles") +
    theme_grey()
}

## QQ Plots for starters ----
# QQ Plot for starters 22/23
qq_starters_22_23 <- create_qq_plot(
  starters_22_23$total_points,
  "Q-Q Plot of Starters 22/23 Points"
)
ggsave(file.path(plot_directory, "QQ Plot of Starters 22-23
   Points.png"),
       plot = qq_starters_22_23, width = 8, height = 6)

# QQ Plot for starters 23/24
qq_starters_23_24 <- create_qq_plot(
  starters_23_24$total_points,
  "Q-Q Plot of Starters 23/24 Points"
)
```

```
ggsave(file.path(plot_directory, "QQ Plot of Starters 23-24
   Points.png"),
       plot = qq_starters_23_24, width = 8, height = 6)

# QQ Plot for starters 24/25
qq_starters_24_25 <- create_qq_plot(
  starters_24_25$total_points,
  "Q-Q Plot of Starters 24/25 Points"
)
ggsave(file.path(plot_directory, "QQ Plot of Starters 24-25
   Points.png"),
       plot = qq_starters_24_25, width = 8, height = 6)

## QQ Plots for players with at least 1 minute ----
# QQ Plot for players with 1+ min 22/23
qq_1min_22_23 <- create_qq_plot(
  players_played_1$total_points,
  "Q-Q Plot of Players with 1+ Min 22/23 Points"
)
ggsave(file.path(plot_directory, "QQ Plot of Players with 1+
   Min 22-23 Points.png"),
       plot = qq_1min_22_23, width = 8, height = 6)

# QQ Plot for players with 1+ min 23/24
qq_1min_23_24 <- create_qq_plot(
  players_played_2$total_points,
  "Q-Q Plot of Players with 1+ Min 23/24 Points"
)
ggsave(file.path(plot_directory, "QQ Plot of Players with 1+
   Min 23-24 Points.png"),
       plot = qq_1min_23_24, width = 8, height = 6)

# QQ Plot for players with 1+ min 24/25
qq_1min_24_25 <- create_qq_plot(
  players_played_3$total_points,
  "Q-Q Plot of Players with 1+ Min 24/25 Points"
```

```
)
ggsave(file.path(plot_directory, "QQ Plot of Players with 1+
   Min 24-25 Points.png"),
       plot = qq_1min_24_25, width = 8, height = 6)

## QQ Plots for all players ----
# QQ Plot for all players 22/23
qq_all_22_23 <- create_qq_plot(
  all1$total_points,
  "Q-Q Plot of All Players 22/23 Points"
)
ggsave(file.path(plot_directory, "QQ Plot of All Players 22-23
   Points.png"),
       plot = qq_all_22_23, width = 8, height = 6)

# QQ Plot for all players 23/24
qq_all_23_24 <- create_qq_plot(
  all2$total_points,
  "Q-Q Plot of All Players 23/24 Points"
)
ggsave(file.path(plot_directory, "QQ Plot of All Players 23-24
   Points.png"),
       plot = qq_all_23_24, width = 8, height = 6)

# QQ Plot for all players 24/25
qq_all_24_25 <- create_qq_plot(
  all3$total_points,
  "Q-Q Plot of All Players 24/25 Points"
)
ggsave(file.path(plot_directory, "QQ Plot of All Players 24-25
   Points.png"),
       plot = qq_all_24_25, width = 8, height = 6)

# Additional analysis - Points by position ----

## Position statistics for starters across all seasons ----
```

```
position_stats_starters_22_23 <- starters_22_23 |>
  group_by(position) |>
  summarize(
    Count = n(),
    Mean = mean(total_points, na.rm = TRUE),
    Median = median(total_points, na.rm = TRUE),
    SD = sd(total_points, na.rm = TRUE),
    Min = min(total_points, na.rm = TRUE),
    Max = max(total_points, na.rm = TRUE),
    Skewness = skewness(total_points, na.rm = TRUE),
    Kurtosis = kurtosis(total_points, na.rm = TRUE)
  )
postion_stats_starters_22_23
position_stats_starters_23_24 <- starters_23_24 |>
  group_by(position) |>
  summarize(
    Count = n(),
    Mean = mean(total_points, na.rm = TRUE),
    Median = median(total_points, na.rm = TRUE),
    SD = sd(total_points, na.rm = TRUE),
    Min = min(total_points, na.rm = TRUE),
    Max = max(total_points, na.rm = TRUE),
    Skewness = skewness(total_points, na.rm = TRUE),
    Kurtosis = kurtosis(total_points, na.rm = TRUE)
  )

position_stats_starters_24_25 <- starters_24_25 |>
  group_by(position) |>
  summarize(
    Count = n(),
    Mean = mean(total_points, na.rm = TRUE),
    Median = median(total_points, na.rm = TRUE),
    SD = sd(total_points, na.rm = TRUE),
    Min = min(total_points, na.rm = TRUE),
    Max = max(total_points, na.rm = TRUE),
    Skewness = skewness(total_points, na.rm = TRUE),
```

```
    Kurtosis = kurtosis(total_points, na.rm = TRUE)
  )

## Position statistics for players with at least 1 minute ----
position_stats_1min_22_23 <- players_played_1 |>
  group_by(position) |>
  summarize(
    Count = n(),
    Mean = mean(total_points, na.rm = TRUE),
    Median = median(total_points, na.rm = TRUE),
    SD = sd(total_points, na.rm = TRUE),
    Min = min(total_points, na.rm = TRUE),
    Max = max(total_points, na.rm = TRUE),
    Skewness = skewness(total_points, na.rm = TRUE),
    Kurtosis = kurtosis(total_points, na.rm = TRUE)
  )

position_stats_1min_23_24 <- players_played_2 |>
  group_by(position) |>
  summarize(
    Count = n(),
    Mean = mean(total_points, na.rm = TRUE),
    Median = median(total_points, na.rm = TRUE),
    SD = sd(total_points, na.rm = TRUE),
    Min = min(total_points, na.rm = TRUE),
    Max = max(total_points, na.rm = TRUE),
    Skewness = skewness(total_points, na.rm = TRUE),
    Kurtosis = kurtosis(total_points, na.rm = TRUE)
  )

position_stats_1min_24_25 <- players_played_3 |>
  group_by(position) |>
  summarize(
    Count = n(),
    Mean = mean(total_points, na.rm = TRUE),
    Median = median(total_points, na.rm = TRUE),
```

```
    SD = sd(total_points, na.rm = TRUE),
    Min = min(total_points, na.rm = TRUE),
    Max = max(total_points, na.rm = TRUE),
    Skewness = skewness(total_points, na.rm = TRUE),
    Kurtosis = kurtosis(total_points, na.rm = TRUE)
  )

## Position statistics for all players ----
position_stats_all_22_23 <- all1 |>
  group_by(position) |>
  summarize(
    Count = n(),
    Mean = mean(total_points, na.rm = TRUE),
    Median = median(total_points, na.rm = TRUE),
    SD = sd(total_points, na.rm = TRUE),
    Min = min(total_points, na.rm = TRUE),
    Max = max(total_points, na.rm = TRUE),
    Skewness = skewness(total_points, na.rm = TRUE),
    Kurtosis = kurtosis(total_points, na.rm = TRUE)
  )

position_stats_all_23_24 <- all2 |>
  group_by(position) |>
  summarize(
    Count = n(),
    Mean = mean(total_points, na.rm = TRUE),
    Median = median(total_points, na.rm = TRUE),
    SD = sd(total_points, na.rm = TRUE),
    Min = min(total_points, na.rm = TRUE),
    Max = max(total_points, na.rm = TRUE),
    Skewness = skewness(total_points, na.rm = TRUE),
    Kurtosis = kurtosis(total_points, na.rm = TRUE)
  )

position_stats_all_24_25 <- all3 |>
  group_by(position) |>
```

```
  summarize(
    Count = n(),
    Mean = mean(total_points, na.rm = TRUE),
    Median = median(total_points, na.rm = TRUE),
    SD = sd(total_points, na.rm = TRUE),
    Min = min(total_points, na.rm = TRUE),
    Max = max(total_points, na.rm = TRUE),
    Skewness = skewness(total_points, na.rm = TRUE),
    Kurtosis = kurtosis(total_points, na.rm = TRUE)
  )

# Save position statistics in TeX format to text files
print(xtable(position_stats_starters_22_23,
  caption = "Position Statistics for Starters 22/23"),
    file = file.path(plot_directory, "
       Position_Statistics_Starters_22_23.txt"),
    include.rownames = FALSE,
    floating = TRUE,
    latex.environments = "center",
    type = "latex")

print(xtable(position_stats_starters_23_24,
  caption = "Position Statistics for Starters 23/24"),
    file = file.path(plot_directory, "
       Position_Statistics_Starters_23_24.txt"),
    include.rownames = FALSE,
    floating = TRUE,
    latex.environments = "center",
    type = "latex")

print(xtable(position_stats_starters_24_25,
  caption = "Position Statistics for Starters 24/25"),
    file = file.path(plot_directory, "
       Position_Statistics_Starters_24_25.txt"),
    include.rownames = FALSE,
    floating = TRUE,
```

```
    latex.environments = "center",
    type = "latex")

print(xtable(position_stats_1min_22_23,
  caption = "Position Statistics for Players with 1+ Min
     22/23"),
    file = file.path(plot_directory, "
       Position_Statistics_1min_22_23.txt"),
    include.rownames = FALSE,
    floating = TRUE,
    latex.environments = "center",
    type = "latex")

print(xtable(position_stats_1min_23_24,
  caption = "Position Statistics for Players with 1+ Min
     23/24"),
    file = file.path(plot_directory, "
       Position_Statistics_1min_23_24.txt"),
    include.rownames = FALSE,
    floating = TRUE,
    latex.environments = "center",
    type = "latex")

print(xtable(position_stats_1min_24_25,
  caption = "Position Statistics for Players with 1+ Min
     24/25"),
    file = file.path(plot_directory, "
       Position_Statistics_1min_24_25.txt"),
    include.rownames = FALSE,
    floating = TRUE,
    latex.environments = "center",
    type = "latex")

print(xtable(position_stats_all_22_23,
  caption = "Position Statistics for All Players 22/23"),
    file = file.path(plot_directory, "
```

```
      Position_Statistics_All_22_23.txt"),
    include.rownames = FALSE,
    floating = TRUE,
    latex.environments = "center",
    type = "latex")

print(xtable(position_stats_all_23_24,
  caption = "Position Statistics for All Players 23/24"),
    file = file.path(plot_directory, "
      Position_Statistics_All_23_24.txt"),
    include.rownames = FALSE,
    floating = TRUE,
    latex.environments = "center",
    type = "latex")

print(xtable(position_stats_all_24_25,
  caption = "Position Statistics for All Players 24/25"),
    file = file.path(plot_directory, "
      Position_Statistics_All_24_25.txt"),
    include.rownames = FALSE,
    floating = TRUE,
    latex.environments = "center",
    type = "latex")

# Print to console as well
print(position_stats_starters_22_23)
print(position_stats_starters_23_24)
print(position_stats_starters_24_25)
print(position_stats_1min_22_23)
print(position_stats_1min_23_24)
print(position_stats_1min_24_25)
print(position_stats_all_22_23)
print(position_stats_all_23_24)
print(position_stats_all_24_25)

# After thought
```

```
df1 <- season_22_23
df2 <- season_23_24
df3 <- season_24_25
# Count unique names
df1 |> distinct(name) |> nrow()
df2 |> distinct(name) |> nrow()
df3 |> distinct(name) |> nrow()

# Count unique names starts
df1 |> filter(starts == 1) |>distinct(name) |> nrow()
df2 |> filter(starts == 1) |>distinct(name) |> nrow()
df3 |> filter(starts == 1) |>distinct(name) |> nrow()

# Same for \geq 1
df1 |> filter(minutes >= 1) |>distinct(name) |> nrow()
df2 |> filter(minutes >= 1) |>distinct(name) |> nrow()
df3 |> filter(minutes >= 1) |>distinct(name) |> nrow()

# Formal Tests for norm and statio

rm(list = ls(all = TRUE))

set.seed(1)

pacman::p_load(tidyverse, tseries)
# Fetch data and prelim data manipulation ----
df <- read_csv("C:/Users/peram/Documents/test/Datasett/Ekstra
   kolonner, stigende GW, alle tre sesonger(22-24), heltall.csv
   ")
num <- df |>
  group_by(name,GW) |> select(name,GW,total_points, value, bps,
      transfers_balance)
num <- as.data.frame(num)

# Get unique player names
unique_players <- unique(num$name)
```

```
# Create an empty list to store results for each player
player_results <- list()

# Loop through each unique player
for (player_name in unique_players) {
  # Subset the data for the current player
  player_data <- num[num$name == player_name, ]
  player_data <- player_data[order(player_data$GW), ]

  n_obs <- nrow(player_data)
  player_mean_total_points <- mean(player_data$total_points)
  adf_p_value <- NA # Initialize p-value

  # ACF analysis removed - not needed for LSTM assumptions

  adf_result_obj <- NA # Initialize ADF result object

  if (n_obs >= 5) {
    k_value <- min(floor((n_obs - 1) / 3), 2)
    tryCatch({
      adf_result_obj <- adf.test(player_data$total_points,
         alternative = "stationary", k = k_value)
      adf_p_value <- adf_result_obj$p.value
    }, error = function(e) {
      adf_result_obj <- paste("ADF Error:", e$message)
      adf_p_value <- NA
    })
  } else {
    adf_result_obj <- "Insufficient data (< 6 observations)"
    adf_p_value <- NA
  }

  # Add Kolmogorov-Smirnov test for normality
  ks_p_value <- NA
  ks_result_obj <- NA
```

```
  points <- player_data$total_points

  if (n_obs >= 3 && !any(is.na(points)) && length(unique(points
     )) >= 2 && sd(points) > 0) {
    tryCatch({
      ks_result_obj <- ks.test(points, "pnorm", mean(points),
         sd(points))
      ks_p_value <- ks_result_obj$p.value
    }, error = function(e) {
      ks_result_obj <- paste("KS Error:", e$message)
      ks_p_value <- NA
    })
  } else {
    ks_result_obj <- "Insufficient data or zero variance"
    ks_p_value <- NA
  }

  player_results[[player_name]] <- list(
    player_data = player_data,
    adf_result_object = adf_result_obj,
    adf_p_value = adf_p_value,
    ks_result_object = ks_result_obj,
    ks_p_value = ks_p_value,
    mean_total_points = player_mean_total_points,
    n_observations = n_obs
  )
}

# To calculate the mean p-value across all players where the
   ADF test was run:
valid_p_values <- unlist(lapply(player_results, function(x)
   x$adf_p_value))
valid_p_values <- valid_p_values[!is.na(valid_p_values)]
mean_adf_p_value <- mean(valid_p_values)
cat(paste("Mean ADF p-value (for players with sufficient data)
   :", mean_adf_p_value, "\n"))
```

```
# To get a classification of reject/fail to reject based on a
   significance level (e.g., 0.05):
alpha <- 0.05
adf_classification <- lapply(player_results, function(x) {
  if (!is.na(x$adf_p_value)) {
    if (x$adf_p_value < alpha) {
      return("Reject")
    } else {
      return("Fail to Reject")
    }
  } else {
    return("Insufficient Data")
  }
})

# You can then analyze the frequency of "Reject" and "Fail to
   Reject"
table(unlist(adf_classification))

# Kolmogorov-Smirnov test for normality (PERFECT for small
   samples 3-5!)
ks_classification <- lapply(player_results, function(x) {
  if (!is.na(x$ks_p_value)) {
    if (x$ks_p_value < alpha) {
      return("Reject Normality")  # Data is NOT normal
    } else {
      return("Fail to Reject Normality")  # Data appears normal
    }
  } else {
    # Determine the reason for missing p-value
    player_data <- x$player_data
    points <- player_data$total_points
    n_obs <- length(points)

    if (n_obs < 3) {
```

```
      return("Insufficient Data")
    } else if (any(is.na(points))) {
      return("Contains NA")
    } else if (length(unique(points)) < 2 || sd(points) == 0) {
      return("Zero Variance")
    } else {
      return("Test Error")
    }
  }
})

# Create summary table for KS results
ks_table <- table(unlist(ks_classification))
print("Kolmogorov-Smirnov Test Results:")
print(ks_table)

# Calculate mean p-value for KS tests (where available)
valid_ks_p <- unlist(lapply(player_results, function(x)
   x$ks_p_value))
valid_ks_p <- valid_ks_p[!is.na(valid_ks_p)]
mean_ks_p_value <- ifelse(length(valid_ks_p) > 0, mean(
   valid_ks_p), NA)

if (!is.na(mean_ks_p_value)) {
  cat(paste("Mean Kolmogorov-Smirnov p-value:", round(
     mean_ks_p_value, 4), "\n"))
}

# Create data frames for plotting
# Convert the ADF classification results to a data frame for
   ggplot
classification_df <- data.frame(
  Classification = names(table(unlist(adf_classification))),
  Count = as.numeric(table(unlist(adf_classification)))
)
classification_df$Percentage <- round(classification_df$Count /
```

```
    sum(classification_df$Count) * 100, 1)

# Convert KS results to data frame for ggplot
ks_df <- data.frame(
  Classification = names(ks_table),
  Count = as.numeric(ks_table)
)
ks_df$Percentage <- round(ks_df$Count / sum(ks_df$Count) * 100,
    1)

# Load patchwork for combining plots
library(patchwork)  # You might need to install this: install.
   packages("patchwork")

# Create ADF plot
adf_plot <- ggplot(classification_df, aes(x = Classification, y
    = Count, fill = Classification)) +
  geom_col(alpha = 0.8, color = "black", size = 0.5) +
  geom_text(aes(label = paste0(Count, "\n(", Percentage, "%)"))
     ,
            vjust = -0.5, size = 3.5) +
  scale_fill_manual(values = c("Insufficient Data" = "#999999",
                               "Fail to Reject" = "#2E8B57",
                               "Reject" = "#DC143C"),
                    guide = "none") +
  labs(title = "ADF Stationarity Test",
       subtitle = paste("Total players analyzed:", sum(
          classification_df$Count)),
       x = "Stationarity Decision (Alpha = 0.05)",
       y = "Number of Players") +
  theme_gray() +
  ylim(0, max(classification_df$Count) * 1.15) +
  theme(plot.title = element_text(hjust = 0.5, size = 12, face
     = "bold"),
        plot.subtitle = element_text(hjust = 0.5, size = 10),
        axis.text.x = element_text(angle = 45, hjust = 1))
```

```
# Create KS plot
ks_plot <- ggplot(ks_df, aes(x = Classification, y = Count,
   fill = Classification)) +
  geom_col(alpha = 0.8, color = "black", size = 0.5) +
  geom_text(aes(label = paste0(Count, "\n(", Percentage, "%)"))
     ,
            vjust = -0.5, size = 3.5) +
  scale_fill_manual(values = c("Fail to Reject Normality" = "#2
     E8B57",     # Green
                               "Reject Normality" = "#DC143C",
                                             # Red
                               "Insufficient Data" = "#999999",
                                             # Gray
                               "Contains NA" = "#FFA500",
                                                  # Orange
                               "Zero Variance" = "#FF1493",
                                               # Hot Pink
                               "Test Error" = "#800080"),
                                                  # Purple
                    guide = "none") +
  labs(title = "Kolmogorov-Smirnov Normality Test",
       subtitle = ifelse(!is.na(mean_ks_p_value),
                         paste("Total players analyzed:", sum(
                            ks_df$Count),
                               "| Mean p-value:", round(
                                  mean_ks_p_value, 4)),
                         paste("Total players analyzed:", sum(
                            ks_df$Count))),
       x = "Normality Decision (Alpha = 0.05)",
       y = "Number of Players")

# Combine both plots (SINGLE TIME ONLY)
combined_plot <- adf_plot + ks_plot +
  plot_annotation(title = "Formal Tests Results",
                  theme = theme(plot.title = element_text(hjust
```

```
                    = 0.5, size = 16, face = "bold")))

print(combined_plot)

# Save the combined plot
ggsave("Formal_Stationarity_and_Normality_Results.png",
       plot = combined_plot,
       width = 24, height = 12, dpi = 300)
```

# H R-Script 3: Keras FPL LSTM Model

```
# Forecasts for FPL Script to Master thesis EBA&PAM
rm(list = ls(all = TRUE))

## 0: Preamble ----
Sys.setenv(WORKON_HOME = "C:/Users/peram/Documents/test/R
   Forecast/.virtualenvs")
reticulate::use_virtualenv("C:/Users/peram/Documents/test/R
   Forecast/.virtualenvs/r-reticulate", required = TRUE)

library(reticulate)
library(keras)

if (!require("pacman")) install.packages("pacman")
pacman::p_load(ggthemes, tidyverse, slider,
               glmnet,httr,jsonlite,tensorflow,
               Metrics, pastecs, stats, png, grid)

set.seed(1)  # Set R's random seed :/
tensorflow::set_random_seed(1) # Doesnt work :/

# Lists to store results ----
model_list       <- list()
scaling_factors  <- list()
predictions_list <- list()
metrics_list     <- list() # User's original list
```

```
validation_metrics <- list() # For storing evaluation results

# Global user inputs ----
split_gw <- 38+38
epoker <- 15
vindu <- 3
emb_dim <- 16
num_patience <- round(epoker*0.5)

# Fetch data and prelim data manipulation ----
df <- read_csv("C:/Users/peram/Documents/test/Datasett/Ekstra
   kolonner, stigende GW, alle tre sesonger(22-24), heltall.csv
   ")
alternativsammensatt <- df

## Positions Partitions ----
gk <- df %>%
  filter(position == "GK") %>%
  mutate(
    row_id  = row_number(),
    pID_idx = as.integer(factor(player_id)),
    tID_idx = as.integer(factor(tID)),
    oID_idx = as.integer(factor(oID))
  )
def <- df %>%
  filter(position == "DEF") %>%
  mutate(
    row_id  = row_number(),
    pID_idx = as.integer(factor(player_id)),
    tID_idx = as.integer(factor(tID)),
    oID_idx = as.integer(factor(oID))
  )
mid <- df %>%
  filter(position == "MID") %>%
  mutate(
    row_id  = row_number(),
```

```
    pID_idx = as.integer(factor(player_id)),
    tID_idx = as.integer(factor(tID)),
    oID_idx = as.integer(factor(oID))
  )
fwd <- df %>%
  filter(position == "FWD") %>%
  mutate(
    row_id  = row_number(),
    pID_idx = as.integer(factor(player_id)),
    tID_idx = as.integer(factor(tID)),
    oID_idx = as.integer(factor(oID))
  )

unscaled_gk <- gk
unscaled_def <- def
unscaled_mid <- mid
unscaled_fwd <- fwd


# 1: Goalkeepers (GK) ----

## GK 1.1 Define Features, Target & Scale ----
numF <- c(
  "assists","creativity","minutes","goals_conceded","saves","
     bonus","bps",
  "expected_assists","expected_goal_involvements","
     expected_goals","ict_index",
  "own_goals","red_cards","threat","transfers_in","
     transfers_out","yellow_cards",
  "expected_goals_conceded","penalties_saved","value","selected
     ",
  "transfers_balance","starts","influence","clean_sheets"
)
tar <- "total_points"
initial_numF_gk <- length(numF)
```

```
gk <- gk %>%
  arrange(pID_idx, GW) %>%
  group_by(pID_idx) %>%
  ungroup()

mu <- mean(gk[[tar]], na.rm = TRUE)
sigma <- sd(gk[[tar]], na.rm = TRUE)

gk <- gk %>%
  mutate(across(all_of(numF), ~ (.x - mean(.x, na.rm = TRUE)) /
      sd(.x, na.rm = TRUE)),
         !!tar := (.data[[tar]] - mu) / sigma)

## GK 1.2: Build rolling numerical windows ----
ws <- vindu
numW <- gk %>%
  group_by(pID_idx) %>%
  filter(n() > ws) %>%
  group_modify(~{
    W <- slide(.x[, numF], .f = ~as.matrix(.x), .before = ws, .
       after = -1, .complete = TRUE)
    valid <- W[(ws + 1):length(W)]
    tgt <- .x[[tar]][(ws + 1):nrow(.x)]
    rid <- .x$row_id[(ws + 1):nrow(.x)]
    tibble(window = valid, target = tgt, row_id = rid)
  }) %>%
  ungroup() %>%
  left_join(select(gk, row_id, GW), by = "row_id")

## GK 1.3: Lasso Regression ----
Xl <- do.call(rbind, lapply(numW$window, colMeans, na.rm = TRUE
  ))
yl <- numW$target
cvfit <- cv.glmnet(Xl, yl, alpha = 1)
minlambda <- cvfit$lambda.min
lassofit <- glmnet(Xl, yl, alpha = 1, lambda = minlambda)
```

```
keepF <- rownames(as.matrix(coef(lassofit)))[coef(lassofit)[,
   1] != 0] %>% setdiff("(Intercept)")
numF_gk <- keepF
numF <- keepF # Update numF globally for this position block

## GK 1.4.1: Rebuild Numeric Windows ----
ws <- vindu # Reset ws just in case, though it's global
numW <- gk %>%
  group_by(pID_idx) %>%
  filter(n() > ws) %>%
  group_modify(~{
    W <- slide(.x[, numF], .f = ~as.matrix(.x), .before = ws, .
       after = -1, .complete = TRUE)
    valid <- W[(ws + 1):length(W)]
    tgt <- .x[[tar]][(ws + 1):nrow(.x)]
    rid <- .x$row_id[(ws + 1):nrow(.x)]
    tibble(window = valid, target = tgt, row_id = rid)
  }) %>%
  ungroup() %>%
  left_join(select(gk, row_id, GW), by = "row_id")

### GK 1.4.2: Build & align categorical windows ----
catW_all <- gk %>%
  group_by(player_id) %>%
  filter(n() > ws) %>%
  group_modify(~{
    W <- slide(.x %>% select(pID_idx, tID_idx, oID_idx, hID), .
       f = ~as.matrix(.x), .before = ws, .after = -1, .complete
        = TRUE)
    valid <- W[(ws + 1):length(W)]
    rid <- .x$row_id[(ws + 1):nrow(.x)]
    tibble(window = valid, row_id = rid)
  }) %>%
  ungroup()
catW <- numW %>%
  select(row_id) %>%
```

```
  left_join(catW_all, by = "row_id")

### GK 1.4.3: To Arrays ----
nSamp <- nrow(numW)
num_array <- array(unlist(numW$window), dim = c(nSamp, ws,
   length(numF)))
targets <- matrix(numW$target, ncol = 1)
pID_hist_array <- t(sapply(catW$window, function(m) m[, "
   pID_idx"]))
tID_hist_array <- t(sapply(catW$window, function(m) m[, "
   tID_idx"]))
oID_hist_array <- t(sapply(catW$window, function(m) m[, "
   oID_idx"]))
hID_hist_array <- t(sapply(catW$window, function(m) m[, "hID"])
   )
cat_cur_mat <- do.call(rbind, lapply(catW$window, function(m) m
   [nrow(m), ]))
pID_cur_array <- matrix(cat_cur_mat[, "pID_idx"], ncol = 1)
tID_cur_array <- matrix(cat_cur_mat[, "tID_idx"], ncol = 1)
oID_cur_array <- matrix(cat_cur_mat[, "oID_idx"], ncol = 1)
hID_cur_array <- matrix(cat_cur_mat[, "hID"], ncol = 1)

## GK 1.5: Split Data ----
idx <- which(numW$GW <= split_gw)
Xnum_train <- num_array[idx,,,drop=F]
Xnum_val <- num_array[-idx,,,drop=F]
y_train <- targets[idx,,drop=F]
y_val <- targets[-idx,,drop=F]
Xcat_hist_tr <- list(pID_hist=pID_hist_array[idx,,drop=F],
   tID_hist=tID_hist_array[idx,,drop=F], oID_hist=
   oID_hist_array[idx,,drop=F], hID_hist=hID_hist_array[idx,,
   drop=F])
Xcat_hist_va <- list(pID_hist=pID_hist_array[-idx,,drop=F],
   tID_hist=tID_hist_array[-idx,,drop=F], oID_hist=
   oID_hist_array[-idx,,drop=F], hID_hist=hID_hist_array[-idx,,
   drop=F])
```

```
Xcat_cur_tr <- list(pID_cur=pID_cur_array[idx,,drop=F], tID_cur
   =tID_cur_array[idx,,drop=F], oID_cur=oID_cur_array[idx,,drop
   =F], hID_cur=hID_cur_array[idx,,drop=F])
Xcat_cur_va <- list(pID_cur=pID_cur_array[-idx,,drop=F],
   tID_cur=tID_cur_array[-idx,,drop=F], oID_cur=oID_cur_array[-
   idx,,drop=F], hID_cur=hID_cur_array[-idx,,drop=F])

## GK 1.6: Building the Keras Neural Network Model ----
nP <- max(gk$pID_idx,na.rm=T); nT <- max(gk$tID_idx,na.rm=T);
   nO <- max(gk$oID_idx,na.rm=T)
inp_num <- layer_input(shape=c(ws,length(numF)),name="input_seq
   ")
inp_ph  <- layer_input(shape=c(ws),dtype="int32",name="pID_hist
   ")
inp_th  <- layer_input(shape=c(ws),dtype="int32",name="tID_hist
   ")
inp_oh  <- layer_input(shape=c(ws),dtype="int32",name="oID_hist
   ")
inp_hh  <- layer_input(shape=c(ws),dtype="int32",name="hID_hist
   ")
inp_pc  <- layer_input(shape=c(1),dtype="int32",name="pID_cur")
inp_tc  <- layer_input(shape=c(1),dtype="int32",name="tID_cur")
inp_oc  <- layer_input(shape=c(1),dtype="int32",name="oID_cur")
inp_hc  <- layer_input(shape=c(1),dtype="int32",name="hID_cur")

emb_ph_hist_stream <- inp_ph %>% layer_embedding(input_dim=nP
   +1,output_dim=emb_dim,mask_zero=T)
emb_th_hist_stream <- inp_th %>% layer_embedding(input_dim=nT
   +1,output_dim=emb_dim,mask_zero=T)
emb_oh_hist_stream <- inp_oh %>% layer_embedding(input_dim=nO
   +1,output_dim=emb_dim,mask_zero=T)
emb_hh_hist_stream <- inp_hh %>% layer_lambda(function(x){
   tf$expand_dims(tf$cast(x,tf$float32),axis=as.integer(-1))})
cat_hist_seq_stream <- layer_concatenate(list(
   emb_ph_hist_stream,emb_th_hist_stream,emb_oh_hist_stream,
   emb_hh_hist_stream),axis=-1)
```

```
emb_pc_cur_stream <- inp_pc %>% layer_embedding(input_dim=nP+1,
   output_dim=emb_dim) %>% layer_flatten()
emb_tc_cur_stream <- inp_tc %>% layer_embedding(input_dim=nT+1,
   output_dim=emb_dim) %>% layer_flatten()
emb_oc_cur_stream <- inp_oc %>% layer_embedding(input_dim=nO+1,
   output_dim=emb_dim) %>% layer_flatten()
emb_hc_cur_stream <- inp_hc %>% layer_lambda(function(x)tf$cast
   (x,tf$float32))
seq_pc_cur_stream <- emb_pc_cur_stream %>% layer_repeat_vector(
   n=ws)
seq_tc_cur_stream <- emb_tc_cur_stream %>% layer_repeat_vector(
   n=ws)
seq_oc_cur_stream <- emb_oc_cur_stream %>% layer_repeat_vector(
   n=ws)
seq_hc_cur_stream <- emb_hc_cur_stream %>% layer_repeat_vector(
   n=ws)
current_cat_as_seq_stream <- layer_concatenate(list(
   seq_pc_cur_stream,seq_tc_cur_stream,seq_oc_cur_stream,
   seq_hc_cur_stream),axis=-1)

lstm_in_stream <- layer_concatenate(list(inp_num,
   cat_hist_seq_stream,current_cat_as_seq_stream),axis=-1)
lstm_out_stream <- lstm_in_stream %>% layer_lstm(units=64,
   return_sequences=F) %>% layer_dropout(rate=0.2)

head_stream <- lstm_out_stream %>% layer_dense(units=32,
   activation="relu")
out_stream <- head_stream %>% layer_dense(units=1,activation="
   linear")

model <- keras_model(inputs=list(inp_num,inp_ph,inp_th,inp_oh,
   inp_hh,inp_pc,inp_tc,inp_oc,inp_hc),outputs=out_stream,name
   ="gk_lstm_model_v2")
model %>% compile(optimizer=optimizer_adam(),loss="mse",metrics
   =metric_root_mean_squared_error())
```

```
summary(model)
keras_python <- import("keras")
keras_python$utils$plot_model(model,to_file="keras-model-gk-v2.
   png",show_shapes=T,show_dtype=F,show_layer_names=T,rankdir="
   TB",expand_nested=F,dpi=200,show_layer_activations=F)
img <- readPNG("keras-model-gk-v2.png")
grid::grid.raster(img)

## GK 1.7: Callback Functions and Scaling Factors ----
weights_filepath_gk <- "R Forecast/best_gk_v2.hdf5"
early_stopping <- callback_early_stopping(monitor="val_loss",
   patience=num_patience,restore_best_weights=T)
model_checkpoint_gk <- callback_model_checkpoint(filepath=
   weights_filepath_gk,monitor="val_loss",save_best_only=T,
   save_weights_only=T,mode="min",verbose=1)
reduce_lr <- callback_reduce_lr_on_plateau(monitor="val_loss",
   factor=0.2,patience=floor(num_patience*0.5),min_lr=1e-6,
   verbose=1)
scaling_factors$gk <- list(mu=mu,sigma=sigma,numF=numF)

## GK 1.8: Train Goalkeeper LSTM Model ----
history <- model %>% fit(
  x = c(list(input_seq=Xnum_train), Xcat_hist_tr, Xcat_cur_tr),
  y = y_train,
  epochs = epoker,
  batch_size = 16,
  validation_data = list(c(list(input_seq=Xnum_val),
     Xcat_hist_va, Xcat_cur_va), y_val),
  callbacks = list(early_stopping, model_checkpoint_gk,
     reduce_lr)
)

## GK 1.9: Generate Predictions & Invert Scaling ----
val_row_ids <- numW$row_id[-idx]
pred_all <- model %>% predict(c(list(input_seq=Xnum_val),
```

```
   Xcat_hist_va, Xcat_cur_va))
preds_gk <- tibble(
  row_id=val_row_ids,
  predicted_total_points_scaled=as.vector(pred_all),
  actual_total_points_scaled=as.vector(y_val)
) %>%
  mutate(
    predicted_total_points = predicted_total_points_scaled *
       sigma + mu,
    actual_total_points = actual_total_points_scaled * sigma +
       mu
  ) %>%
  left_join(unscaled_gk, by="row_id")

## GK 1.10 Save model and validation set predictions ----
model_list$gk <- model
preds_gk_clean <- preds_gk %>%
  select(row_id, GW, player_id, name, position, team,
     actual_total_points, predicted_total_points, value)

## GK 1.11: Plots for Evaluation ----
history_gk <- history
history_df_gk <- data.frame(history_gk)
plot_history_gk <- plot(history_gk)
print(plot_history_gk) # Explicitly print

plot_actual_predicted_gk <- ggplot(preds_gk, aes(x=
   actual_total_points, y=predicted_total_points)) +
  geom_point(alpha=0.5) +
  geom_abline(intercept=0, slope=1, linetype="dashed", color="
     red") +
  labs(title="GK Actual vs. Predicted", x="Actual", y="
     Predicted") +
  facet_wrap(~position) +
  theme_grey() +
  coord_cartesian(xlim=range(preds_gk$actual_total_points,na.rm
```

```
    =T), ylim=range(preds_gk$predicted_total_points,na.rm=T))
print(plot_actual_predicted_gk)

preds_gk_residuals <- preds_gk %>% mutate(residual=
   actual_total_points-predicted_total_points)
plot_residuals_predicted_gk <- ggplot(preds_gk_residuals, aes(x
   =predicted_total_points, y=residual)) +
  geom_point(alpha=0.5) +
  geom_hline(yintercept=0, linetype="dashed", color="red") +
  labs(title="GK Residuals vs. Predicted", x="Predicted", y="
     Residual") +
  facet_wrap(~position) +
  theme_grey()
print(plot_residuals_predicted_gk)

plot_distribution_residuals_gk <- ggplot(preds_gk_residuals,
   aes(x=residual)) +
  geom_histogram(aes(y=after_stat(density)), bins=30, fill="
     blue", alpha=0.7) +
  geom_density(color="red") +
  geom_vline(xintercept=0, linetype="dashed", color="black") +
  labs(title="GK Residual Distribution", x="Residual", y="
     Density") +
  facet_wrap(~position) +
  theme_grey()
print(plot_distribution_residuals_gk)

model_evaluation_gk <- model |> evaluate(c(list(input_seq=
   Xnum_val), Xcat_hist_va, Xcat_cur_va), y_val, verbose=0)
validation_metrics$gk <- model_evaluation_gk


# 2: Defenders (DEF) ----

## DEF 2.1 Define Features, Target & Scale ----
numF <- c("goals_scored","assists", "creativity", "minutes", "
```

```
   goals_conceded",
          "bonus", "bps", "expected_assists", "
             expected_goal_involvements",
          "expected_goals", "ict_index", "own_goals", "
             red_cards",
          "threat", "transfers_in", "transfers_out", "
             yellow_cards",
          "expected_goals_conceded", "value",
          "selected", "transfers_balance", "starts", "influence
             ",
          "clean_sheets")
tar <- "total_points"
initial_numF_def <- length(numF)

def <- def %>%
  arrange(pID_idx, GW) %>%
  group_by(pID_idx) %>%
  ungroup()

mu <- mean(def[[tar]], na.rm=T)
sigma <- sd(def[[tar]], na.rm=T)

def <- def %>%
  mutate(across(all_of(numF),~(.x-mean(.x,na.rm=T))/sd(.x,na.rm
     =T)),
         !!tar:=(.data[[tar]]-mu)/sigma)

## DEF 2.2: Build rolling numerical windows ----
ws <- vindu
numW <- def %>%
  group_by(pID_idx) %>%
  filter(n()>ws) %>%
  group_modify(~{
    W<-slide(.x[,numF],.f=~as.matrix(.x),.before=ws,.after=-1,.
       complete=T)
    valid<-W[(ws+1):length(W)]
```

```
    tgt<-.x[[tar]][(ws+1):nrow(.x)]
    rid<-.x$row_id[(ws+1):nrow(.x)]
    tibble(window=valid,target=tgt,row_id=rid)
  }) %>%
  ungroup() %>%
  left_join(select(def,row_id,GW),by="row_id")

## DEF 2.3: Lasso Regression ----
Xl <- do.call(rbind,lapply(numW$window,colMeans,na.rm=T))
yl <- numW$target
cvfit <- cv.glmnet(Xl,yl,alpha=1)
minlambda <- cvfit$lambda.min
lassofit <- glmnet(Xl,yl,alpha=1,lambda=minlambda)
keepF <- rownames(as.matrix(coef(lassofit)))[coef(lassofit)
   [,1]!=0] %>% setdiff("(Intercept)")
numF_def <- keepF
numF <- keepF

## DEF 2.4.1: Rebuild Numeric Windows ----
ws <- vindu
numW <- def %>%
  group_by(pID_idx) %>%
  filter(n()>ws) %>%
  group_modify(~{
    W<-slide(.x[,numF],.f=~as.matrix(.x),.before=ws,.after=-1,.
       complete=T)
    valid<-W[(ws+1):length(W)]
    tgt<-.x[[tar]][(ws+1):nrow(.x)]
    rid<-.x$row_id[(ws+1):nrow(.x)]
    tibble(window=valid,target=tgt,row_id=rid)
  }) %>%
  ungroup() %>%
  left_join(select(def,row_id,GW),by="row_id")

### DEF 2.4.2: Build & align categorical windows ----
catW_all <- def %>%
```

```
  group_by(player_id) %>%
  filter(n()>ws) %>%
  group_modify(~{
    W<-slide(.x%>%select(pID_idx,tID_idx,oID_idx,hID),.f=~as.
       matrix(.x),.before=ws,.after=-1,.complete=T)
    valid<-W[(ws+1):length(W)]
    rid<-.x$row_id[(ws+1):nrow(.x)]
    tibble(window=valid,row_id=rid)
  }) %>%
  ungroup()
catW <- numW %>%
  select(row_id) %>%
  left_join(catW_all,by="row_id")

### DEF 2.4.3: To Arrays ----
nSamp<-nrow(numW)
num_array<-array(unlist(numW$window),dim=c(nSamp,ws,length(numF
   )))
targets<-matrix(numW$target,ncol=1)
pID_hist_array<-t(sapply(catW$window,function(m)m[,"pID_idx"]))
tID_hist_array<-t(sapply(catW$window,function(m)m[,"tID_idx"]))
oID_hist_array<-t(sapply(catW$window,function(m)m[,"oID_idx"]))
hID_hist_array<-t(sapply(catW$window,function(m)m[,"hID"]))
cat_cur_mat<-do.call(rbind,lapply(catW$window,function(m)m[nrow
   (m),]))
pID_cur_array<-matrix(cat_cur_mat[,"pID_idx"],ncol=1)
tID_cur_array<-matrix(cat_cur_mat[,"tID_idx"],ncol=1)
oID_cur_array<-matrix(cat_cur_mat[,"oID_idx"],ncol=1)
hID_cur_array<-matrix(cat_cur_mat[,"hID"],ncol=1)

## DEF 2.5: Split Data ----
idx<-which(numW$GW<=split_gw)
Xnum_train<-num_array[idx,,,drop=F]
Xnum_val<-num_array[-idx,,,drop=F]
y_train<-targets[idx,,drop=F]
y_val<-targets[-idx,,drop=F]
```

```
Xcat_hist_tr<-list(pID_hist=pID_hist_array[idx,,drop=F],
   tID_hist=tID_hist_array[idx,,drop=F],oID_hist=oID_hist_array
   [idx,,drop=F],hID_hist=hID_hist_array[idx,,drop=F])
Xcat_hist_va<-list(pID_hist=pID_hist_array[-idx,,drop=F],
   tID_hist=tID_hist_array[-idx,,drop=F],oID_hist=
   oID_hist_array[-idx,,drop=F],hID_hist=hID_hist_array[-idx,,
   drop=F])
Xcat_cur_tr<-list(pID_cur=pID_cur_array[idx,,drop=F],tID_cur=
   tID_cur_array[idx,,drop=F],oID_cur=oID_cur_array[idx,,drop=F
   ],hID_cur=hID_cur_array[idx,,drop=F])
Xcat_cur_va<-list(pID_cur=pID_cur_array[-idx,,drop=F],tID_cur=
   tID_cur_array[-idx,,drop=F],oID_cur=oID_cur_array[-idx,,drop
   =F],hID_cur=hID_cur_array[-idx,,drop=F])

## DEF 2.6: Building the Keras Neural Network Model ----
nP<-max(def$pID_idx,na.rm=T);nT<-max(def$tID_idx,na.rm=T);nO<-
   max(def$oID_idx,na.rm=T)
inp_num<-layer_input(shape=c(ws,length(numF)),name="input_seq")
inp_ph<-layer_input(shape=c(ws),dtype="int32",name="pID_hist")
inp_th<-layer_input(shape=c(ws),dtype="int32",name="tID_hist")
inp_oh<-layer_input(shape=c(ws),dtype="int32",name="oID_hist")
inp_hh<-layer_input(shape=c(ws),dtype="int32",name="hID_hist")
inp_pc<-layer_input(shape=c(1),dtype="int32",name="pID_cur")
inp_tc<-layer_input(shape=c(1),dtype="int32",name="tID_cur")
inp_oc<-layer_input(shape=c(1),dtype="int32",name="oID_cur")
inp_hc<-layer_input(shape=c(1),dtype="int32",name="hID_cur")

emb_ph_hist_stream<-inp_ph%>%layer_embedding(input_dim=nP+1,
   output_dim=emb_dim,mask_zero=T)
emb_th_hist_stream<-inp_th%>%layer_embedding(input_dim=nT+1,
   output_dim=emb_dim,mask_zero=T)
emb_oh_hist_stream<-inp_oh%>%layer_embedding(input_dim=nO+1,
   output_dim=emb_dim,mask_zero=T)
emb_hh_hist_stream<-inp_hh%>%layer_lambda(function(x){
   tf$expand_dims(tf$cast(x,tf$float32),axis=as.integer(-1))})
cat_hist_seq_stream<-layer_concatenate(list(emb_ph_hist_stream,
```

```
   emb_th_hist_stream,emb_oh_hist_stream,emb_hh_hist_stream),
   axis=-1)

emb_pc_cur_stream<-inp_pc%>%layer_embedding(input_dim=nP+1,
   output_dim=emb_dim)%>%layer_flatten()
emb_tc_cur_stream<-inp_tc%>%layer_embedding(input_dim=nT+1,
   output_dim=emb_dim)%>%layer_flatten()
emb_oc_cur_stream<-inp_oc%>%layer_embedding(input_dim=nO+1,
   output_dim=emb_dim)%>%layer_flatten()
emb_hc_cur_stream<-inp_hc%>%layer_lambda(function(x)tf$cast(x,
   tf$float32))
seq_pc_cur_stream<-emb_pc_cur_stream%>%layer_repeat_vector(n=ws
   )
seq_tc_cur_stream<-emb_tc_cur_stream%>%layer_repeat_vector(n=ws
   )
seq_oc_cur_stream<-emb_oc_cur_stream%>%layer_repeat_vector(n=ws
   )
seq_hc_cur_stream<-emb_hc_cur_stream%>%layer_repeat_vector(n=ws
   )
current_cat_as_seq_stream<-layer_concatenate(list(
   seq_pc_cur_stream,seq_tc_cur_stream,seq_oc_cur_stream,
   seq_hc_cur_stream),axis=-1)

lstm_in_stream<-layer_concatenate(list(inp_num,
   cat_hist_seq_stream,current_cat_as_seq_stream),axis=-1)
lstm_out_stream<-lstm_in_stream%>%layer_lstm(units=64,
   return_sequences=F)%>%layer_dropout(rate=0.2)

head_stream<-lstm_out_stream%>%layer_dense(units=32,activation
   ="relu")
out_stream<-head_stream%>%layer_dense(units=1,activation="
   linear")

model<-keras_model(inputs=list(inp_num,inp_ph,inp_th,inp_oh,
   inp_hh,inp_pc,inp_tc,inp_oc,inp_hc),outputs=out_stream,name
   ="def_lstm_model_v2")
```

```
model%>%compile(optimizer=optimizer_adam(),loss="mse",metrics=
   metric_root_mean_squared_error())

summary(model)
keras_python<-import("keras")
keras_python$utils$plot_model(model,to_file="keras-model-def-v2
   .png",show_shapes=T,show_dtype=F,show_layer_names=T,rankdir
   ="TB",expand_nested=F,dpi=200,show_layer_activations=F)
img<-readPNG("keras-model-def-v2.png")
grid::grid.raster(img)

## DEF 2.7: Callback Functions and Scaling Factors ----
weights_filepath_def<-"R Forecast/best_def_v2.hdf5"
early_stopping<-callback_early_stopping(monitor="val_loss",
   patience=num_patience,restore_best_weights=T)
model_checkpoint_def<-callback_model_checkpoint(filepath=
   weights_filepath_def,monitor="val_loss",save_best_only=T,
   save_weights_only=T,mode="min",verbose=1)
reduce_lr<-callback_reduce_lr_on_plateau(monitor="val_loss",
   factor=0.2,patience=floor(num_patience*0.5),min_lr=1e-6,
   verbose=1)
scaling_factors$def<-list(mu=mu,sigma=sigma,numF=numF)

## DEF 2.8: Train Defender LSTM Model ----
history<-model%>%fit(
  x=c(list(input_seq=Xnum_train),Xcat_hist_tr,Xcat_cur_tr),
  y=y_train,
  epochs=epoker,
  batch_size=16,
  validation_data=list(c(list(input_seq=Xnum_val),Xcat_hist_va,
     Xcat_cur_va),y_val),
  callbacks=list(early_stopping,model_checkpoint_def,reduce_lr)
)

## DEF 2.9: Generate Predictions & Invert Scaling ----
val_row_ids<-numW$row_id[-idx]
```

```
pred_all<-model%>%predict(c(list(input_seq=Xnum_val),
   Xcat_hist_va,Xcat_cur_va))
preds_def<-tibble(
  row_id=val_row_ids,
  predicted_total_points_scaled=as.vector(pred_all),
  actual_total_points_scaled=as.vector(y_val)
) %>%
  mutate(
    predicted_total_points=predicted_total_points_scaled*sigma+
       mu,
    actual_total_points=actual_total_points_scaled*sigma+mu
  ) %>%
  left_join(unscaled_def,by="row_id")

## DEF 2.10 Save model and validation set predictions ----
model_list$def<-model
preds_def_clean<-preds_def%>%select(row_id,GW,player_id,name,
   position,team,actual_total_points,predicted_total_points,
   value)

## DEF 2.11: Plots for Evaluation ----
history_def<-history
history_df_def<-data.frame(history_def)
plot_history_def<-plot(history_def)
print(plot_history_def)

plot_actual_predicted_def<-ggplot(preds_def,aes(x=
   actual_total_points,y=predicted_total_points))+
  geom_point(alpha=0.5)+
  geom_abline(intercept=0,slope=1,linetype="dashed",color="red
     ")+
  labs(title="DEF Actual vs. Predicted",x="Actual",y="Predicted
     ")+
  facet_wrap(~position)+
  theme_grey()+
  coord_cartesian(xlim=range(preds_def$actual_total_points,na.
```

```
    rm=T),ylim=range(preds_def$predicted_total_points,na.rm=T)
    )
print(plot_actual_predicted_def)

preds_def_residuals<-preds_def%>%mutate(residual=
   actual_total_points-predicted_total_points)
plot_residuals_predicted_def<-ggplot(preds_def_residuals,aes(x=
   predicted_total_points,y=residual))+
  geom_point(alpha=0.5)+
  geom_hline(yintercept=0,linetype="dashed",color="red")+
  labs(title="DEF Residuals vs. Predicted",x="Predicted",y="
     Residual")+
  facet_wrap(~position)+
  theme_grey()
print(plot_residuals_predicted_def)

plot_distribution_residuals_def<-ggplot(preds_def_residuals,aes
   (x=residual))+
  geom_histogram(aes(y=after_stat(density)),bins=30,fill="blue
     ",alpha=0.7)+
  geom_density(color="red")+
  geom_vline(xintercept=0,linetype="dashed",color="black")+
  labs(title="DEF Residual Distribution",x="Residual",y="
     Density")+
  facet_wrap(~position)+
  theme_grey()
print(plot_distribution_residuals_def)

model_evaluation_def <-model |>
  evaluate(c(list(input_seq=Xnum_val), Xcat_hist_va,Xcat_cur_va
     ), y_val, verbose=0)
validation_metrics$def <- model_evaluation_def
validation_metrics$def

# 3: Midfielders (MID) ----
```

```
## MID 3.1 Define Features, Target & Scale ----
numF <- c("goals_scored","assists", "creativity", "minutes", "
   goals_conceded",
          "bonus", "bps", "expected_assists", "
             expected_goal_involvements",
          "expected_goals", "ict_index", "own_goals", "
             red_cards",
          "threat", "transfers_in", "transfers_out", "
             yellow_cards",
          "expected_goals_conceded", "value",
          "selected", "transfers_balance", "starts", "influence
             ",
          "clean_sheets")
tar <- "total_points"
initial_numF_mid <- length(numF) # Corrected

mid <- mid %>%
  arrange(pID_idx, GW) %>%
  group_by(pID_idx) %>%
  ungroup()

mu <- mean(mid[[tar]], na.rm=T)
sigma <- sd(mid[[tar]], na.rm=T)

mid <- mid %>%
  mutate(across(all_of(numF),~(.x-mean(.x,na.rm=T))/sd(.x,na.rm
     =T)),
         !!tar:=(.data[[tar]]-mu)/sigma)

## MID 3.2: Build rolling numerical windows ----
ws <- vindu
numW <- mid %>%
  group_by(pID_idx) %>%
  filter(n()>ws) %>%
  group_modify(~{
    W<-slide(.x[,numF],.f=~as.matrix(.x),.before=ws,.after=-1,.
```

```
      complete=T)
    valid<-W[(ws+1):length(W)]
    tgt<-.x[[tar]][(ws+1):nrow(.x)]
    rid<-.x$row_id[(ws+1):nrow(.x)]
    tibble(window=valid,target=tgt,row_id=rid)
  }) %>%
  ungroup() %>%
  left_join(select(mid,row_id,GW),by="row_id")

## MID 3.3: Lasso Regression ----
Xl <- do.call(rbind,lapply(numW$window,colMeans,na.rm=T))
yl <- numW$target
cvfit <- cv.glmnet(Xl,yl,alpha=1)
minlambda <- cvfit$lambda.min
lassofit <- glmnet(Xl,yl,alpha=1,lambda=minlambda)
keepF <- rownames(as.matrix(coef(lassofit)))[coef(lassofit)
   [,1]!=0] %>% setdiff("(Intercept)")
numF_mid <- keepF
numF <- keepF

## MID 3.4.1: Rebuild Numeric Windows ----
ws <- vindu
numW <- mid %>%
  group_by(pID_idx) %>%
  filter(n()>ws) %>%
  group_modify(~{
    W<-slide(.x[,numF],.f=~as.matrix(.x),.before=ws,.after=-1,.
      complete=T)
    valid<-W[(ws+1):length(W)]
    tgt<-.x[[tar]][(ws+1):nrow(.x)]
    rid<-.x$row_id[(ws+1):nrow(.x)]
    tibble(window=valid,target=tgt,row_id=rid)
  }) %>%
  ungroup() %>%
  left_join(select(mid,row_id,GW),by="row_id")
```

```
### MID 3.4.2: Build & align categorical windows ----
catW_all <- mid %>%
  group_by(player_id) %>%
  filter(n()>ws) %>%
  group_modify(~{
    W<-slide(.x%>%select(pID_idx,tID_idx,oID_idx,hID),.f=~as.
       matrix(.x),.before=ws,.after=-1,.complete=T)
    valid<-W[(ws+1):length(W)]
    rid<-.x$row_id[(ws+1):nrow(.x)]
    tibble(window=valid,row_id=rid)
  }) %>%
  ungroup()
catW <- numW %>%
  select(row_id) %>%
  left_join(catW_all,by="row_id")

### MID 3.4.3: To Arrays ----
nSamp<-nrow(numW)
num_array<-array(unlist(numW$window),dim=c(nSamp,ws,length(numF
   )))
targets<-matrix(numW$target,ncol=1)
pID_hist_array<-t(sapply(catW$window,function(m)m[,"pID_idx"]))
tID_hist_array<-t(sapply(catW$window,function(m)m[,"tID_idx"]))
oID_hist_array<-t(sapply(catW$window,function(m)m[,"oID_idx"]))
hID_hist_array<-t(sapply(catW$window,function(m)m[,"hID"]))
cat_cur_mat<-do.call(rbind,lapply(catW$window,function(m)m[nrow
   (m),]))
pID_cur_array<-matrix(cat_cur_mat[,"pID_idx"],ncol=1)
tID_cur_array<-matrix(cat_cur_mat[,"tID_idx"],ncol=1)
oID_cur_array<-matrix(cat_cur_mat[,"oID_idx"],ncol=1)
hID_cur_array<-matrix(cat_cur_mat[,"hID"],ncol=1)

## MID 3.5: Split Data ----
idx<-which(numW$GW<=split_gw)
Xnum_train<-num_array[idx,,,drop=F]
Xnum_val<-num_array[-idx,,,drop=F]
```

```
y_train<-targets[idx,,drop=F]
y_val<-targets[-idx,,drop=F]
Xcat_hist_tr<-list(pID_hist=pID_hist_array[idx,,drop=F],
   tID_hist=tID_hist_array[idx,,drop=F],oID_hist=oID_hist_array
   [idx,,drop=F],hID_hist=hID_hist_array[idx,,drop=F])
Xcat_hist_va<-list(pID_hist=pID_hist_array[-idx,,drop=F],
   tID_hist=tID_hist_array[-idx,,drop=F],oID_hist=
   oID_hist_array[-idx,,drop=F],hID_hist=hID_hist_array[-idx,,
   drop=F])
Xcat_cur_tr<-list(pID_cur=pID_cur_array[idx,,drop=F],tID_cur=
   tID_cur_array[idx,,drop=F],oID_cur=oID_cur_array[idx,,drop=F
   ],hID_cur=hID_cur_array[idx,,drop=F])
Xcat_cur_va<-list(pID_cur=pID_cur_array[-idx,,drop=F],tID_cur=
   tID_cur_array[-idx,,drop=F],oID_cur=oID_cur_array[-idx,,drop
   =F],hID_cur=hID_cur_array[-idx,,drop=F])

## MID 3.6: Building the Keras Neural Network Model ----
nP<-max(mid$pID_idx,na.rm=T);nT<-max(mid$tID_idx,na.rm=T);nO<-
   max(mid$oID_idx,na.rm=T)
inp_num<-layer_input(shape=c(ws,length(numF)),name="input_seq")
inp_ph<-layer_input(shape=c(ws),dtype="int32",name="pID_hist")
inp_th<-layer_input(shape=c(ws),dtype="int32",name="tID_hist")
inp_oh<-layer_input(shape=c(ws),dtype="int32",name="oID_hist")
inp_hh<-layer_input(shape=c(ws),dtype="int32",name="hID_hist")
inp_pc<-layer_input(shape=c(1),dtype="int32",name="pID_cur")
inp_tc<-layer_input(shape=c(1),dtype="int32",name="tID_cur")
inp_oc<-layer_input(shape=c(1),dtype="int32",name="oID_cur")
inp_hc<-layer_input(shape=c(1),dtype="int32",name="hID_cur")

emb_ph_hist_stream<-inp_ph%>%layer_embedding(input_dim=nP+1,
   output_dim=emb_dim,mask_zero=T)
emb_th_hist_stream<-inp_th%>%layer_embedding(input_dim=nT+1,
   output_dim=emb_dim,mask_zero=T)
emb_oh_hist_stream<-inp_oh%>%layer_embedding(input_dim=nO+1,
   output_dim=emb_dim,mask_zero=T)
emb_hh_hist_stream<-inp_hh%>%layer_lambda(function(x){
```

```
   tf$expand_dims(tf$cast(x,tf$float32),axis=as.integer(-1))})
cat_hist_seq_stream<-layer_concatenate(list(emb_ph_hist_stream,
   emb_th_hist_stream,emb_oh_hist_stream,emb_hh_hist_stream),
   axis=-1)

emb_pc_cur_stream<-inp_pc%>%layer_embedding(input_dim=nP+1,
   output_dim=emb_dim)%>%layer_flatten()
emb_tc_cur_stream<-inp_tc%>%layer_embedding(input_dim=nT+1,
   output_dim=emb_dim)%>%layer_flatten()
emb_oc_cur_stream<-inp_oc%>%layer_embedding(input_dim=nO+1,
   output_dim=emb_dim)%>%layer_flatten()
emb_hc_cur_stream<-inp_hc%>%layer_lambda(function(x)tf$cast(x,
   tf$float32))
seq_pc_cur_stream<-emb_pc_cur_stream%>%layer_repeat_vector(n=ws
   )
seq_tc_cur_stream<-emb_tc_cur_stream%>%layer_repeat_vector(n=ws
   )
seq_oc_cur_stream<-emb_oc_cur_stream%>%layer_repeat_vector(n=ws
   )
seq_hc_cur_stream<-emb_hc_cur_stream%>%layer_repeat_vector(n=ws
   )
current_cat_as_seq_stream<-layer_concatenate(list(
   seq_pc_cur_stream,seq_tc_cur_stream,seq_oc_cur_stream,
   seq_hc_cur_stream),axis=-1)

lstm_in_stream<-layer_concatenate(list(inp_num,
   cat_hist_seq_stream,current_cat_as_seq_stream),axis=-1)
lstm_out_stream<-lstm_in_stream%>%layer_lstm(units=64,
   return_sequences=F)%>%layer_dropout(rate=0.2)

head_stream<-lstm_out_stream%>%layer_dense(units=32,activation
   ="relu")
out_stream<-head_stream%>%layer_dense(units=1,activation="
   linear")

model<-keras_model(inputs=list(inp_num,inp_ph,inp_th,inp_oh,
```

```
    inp_hh,inp_pc,inp_tc,inp_oc,inp_hc),outputs=out_stream,name
    ="mid_lstm_model_v2")
model%>%compile(optimizer=optimizer_adam(),loss="mse",metrics=
    metric_root_mean_squared_error())

summary(model)
keras_python<-import("keras")
keras_python$utils$plot_model(model,to_file="keras-model-mid-v2
    .png",show_shapes=T,show_dtype=F,show_layer_names=T,rankdir
    ="TB",expand_nested=F,dpi=200,show_layer_activations=F)
img<-readPNG("keras-model-mid-v2.png")
grid::grid.raster(img)

## MID 3.7: Callback Functions and Scaling Factors ----
weights_filepath_mid<-"R Forecast/best_mid_v2.hdf5"
early_stopping<-callback_early_stopping(monitor="val_loss",
    patience=num_patience,restore_best_weights=T)
model_checkpoint_mid<-callback_model_checkpoint(filepath=
    weights_filepath_mid,monitor="val_loss",save_best_only=T,
    save_weights_only=T,mode="min",verbose=1)
reduce_lr<-callback_reduce_lr_on_plateau(monitor="val_loss",
    factor=0.2,patience=floor(num_patience*0.5),min_lr=1e-6,
    verbose=1)
scaling_factors$mid<-list(mu=mu,sigma=sigma,numF=numF)

## MID 3.8: Train Midfielders LSTM Model ----
history<-model%>%fit(
  x=c(list(input_seq=Xnum_train),Xcat_hist_tr,Xcat_cur_tr),
  y=y_train,
  epochs=epoker,
  batch_size=16,
  validation_data=list(c(list(input_seq=Xnum_val),Xcat_hist_va,
     Xcat_cur_va),y_val),
  callbacks=list(early_stopping,model_checkpoint_mid,reduce_lr)
)
```

```
## MID 3.9: Generate Predictions & Invert Scaling ----
val_row_ids<-numW$row_id[-idx]
pred_all<-model%>%predict(c(list(input_seq=Xnum_val),
   Xcat_hist_va,Xcat_cur_va))
preds_mid<-tibble(
  row_id=val_row_ids,
  predicted_total_points_scaled=as.vector(pred_all),
  actual_total_points_scaled=as.vector(y_val)
) %>%
  mutate(
    predicted_total_points=predicted_total_points_scaled*sigma+
       mu,
    actual_total_points=actual_total_points_scaled*sigma+mu
  ) %>%
  left_join(unscaled_mid,by="row_id")

## MID 3.10 Save model and validation set predictions ----
model_list$mid<-model
preds_mid_clean<-preds_mid%>%select(row_id,GW,player_id,name,
   position,team,actual_total_points,predicted_total_points,
   value)

## MID 3.11: Plots for Evaluation ----
history_mid<-history
history_df_mid<-data.frame(history_mid)
plot_history_mid<-plot(history_mid)
print(plot_history_mid)

plot_actual_predicted_mid<-ggplot(preds_mid,aes(x=
   actual_total_points,y=predicted_total_points))+
  geom_point(alpha=0.5)+
  geom_abline(intercept=0,slope=1,linetype="dashed",color="red
     ")+
  labs(title="MID Actual vs. Predicted",x="Actual",y="Predicted
     ")+
  facet_wrap(~position)+
```

```
  theme_grey()+
  coord_cartesian(xlim=range(preds_mid$actual_total_points,na.
    rm=T),ylim=range(preds_mid$predicted_total_points,na.rm=T)
    )
print(plot_actual_predicted_mid)

preds_mid_residuals<-preds_mid%>%mutate(residual=
   actual_total_points-predicted_total_points)
plot_residuals_predicted_mid<-ggplot(preds_mid_residuals,aes(x=
   predicted_total_points,y=residual))+
  geom_point(alpha=0.5)+
  geom_hline(yintercept=0,linetype="dashed",color="red")+
  labs(title="MID Residuals vs. Predicted",x="Predicted",y="
    Residual")+
  facet_wrap(~position)+
  theme_grey()
print(plot_residuals_predicted_mid)

plot_distribution_residuals_mid<-ggplot(preds_mid_residuals,aes
   (x=residual))+
  geom_histogram(aes(y=after_stat(density)),bins=30,fill="blue
    ",alpha=0.7)+
  geom_density(color="red")+
  geom_vline(xintercept=0,linetype="dashed",color="black")+
  labs(title="MID Residual Distribution",x="Residual",y="
    Density")+
  facet_wrap(~position)+
  theme_grey()
print(plot_distribution_residuals_mid)

model_evaluation_mid <- model |>
  evaluate(c(list(input_seq=Xnum_val), Xcat_hist_va,Xcat_cur_va
    ), y_val, verbose=0)

validation_metrics$mid <- model_evaluation_mid
```

```
# 4: Forwards (FWD) ----

## FWD 4.1 Define Features, Target & Scale ----
numF <- c("goals_scored","assists", "creativity", "minutes", "
   goals_conceded",
          "bonus", "bps", "expected_assists", "
             expected_goal_involvements",
          "expected_goals", "ict_index", "own_goals", "
             red_cards",
          "threat", "transfers_in", "transfers_out", "
             yellow_cards",
          "expected_goals_conceded", "value",
          "selected", "transfers_balance", "starts", "influence
             ",
          "clean_sheets")
tar <- "total_points"
initial_numF_fwd <- length(numF) # Corrected

fwd <- fwd %>%
  arrange(pID_idx, GW) %>%
  group_by(pID_idx) %>%
  ungroup()

mu <- mean(fwd[[tar]], na.rm=T)
sigma <- sd(fwd[[tar]], na.rm=T)

fwd <- fwd %>%
  mutate(across(all_of(numF),~(.x-mean(.x,na.rm=T))/sd(.x,na.rm
     =T)),
         !!tar:=(.data[[tar]]-mu)/sigma)

## FWD 4.2: Build rolling numerical windows ----
ws <- vindu
numW <- fwd %>%
  group_by(pID_idx) %>%
```

```
  filter(n()>ws) %>%
  group_modify(~{
    W<-slide(.x[,numF],.f=~as.matrix(.x),.before=ws,.after=-1,.
       complete=T)
    valid<-W[(ws+1):length(W)]
    tgt<-.x[[tar]][(ws+1):nrow(.x)]
    rid<-.x$row_id[(ws+1):nrow(.x)]
    tibble(window=valid,target=tgt,row_id=rid)
  }) %>%
  ungroup() %>%
  left_join(select(fwd,row_id,GW),by="row_id")

## FWD 4.3: Lasso Regression ----
Xl <- do.call(rbind,lapply(numW$window,colMeans,na.rm=T))
yl <- numW$target
cvfit <- cv.glmnet(Xl,yl,alpha=1)
minlambda <- cvfit$lambda.min
lassofit <- glmnet(Xl,yl,alpha=1,lambda=minlambda)
keepF <- rownames(as.matrix(coef(lassofit)))[coef(lassofit)
   [,1]!=0] %>% setdiff("(Intercept)")
numF_fwd <- keepF
numF <- keepF

## FWD 4.4.1: Rebuild Numeric Windows ----
ws <- vindu
numW <- fwd %>%
  group_by(pID_idx) %>%
  filter(n()>ws) %>%
  group_modify(~{
    W<-slide(.x[,numF],.f=~as.matrix(.x),.before=ws,.after=-1,.
       complete=T)
    valid<-W[(ws+1):length(W)]
    tgt<-.x[[tar]][(ws+1):nrow(.x)]
    rid<-.x$row_id[(ws+1):nrow(.x)]
    tibble(window=valid,target=tgt,row_id=rid)
  }) %>%
```

```
  ungroup() %>%
  left_join(select(fwd,row_id,GW),by="row_id")

### FWD 4.4.2: Build & align categorical windows ----
catW_all <- fwd %>%
  group_by(player_id) %>%
  filter(n()>ws) %>%
  group_modify(~{
    W<-slide(.x%>%select(pID_idx,tID_idx,oID_idx,hID),.f=~as.
       matrix(.x),.before=ws,.after=-1,.complete=T)
    valid<-W[(ws+1):length(W)]
    rid<-.x$row_id[(ws+1):nrow(.x)]
    tibble(window=valid,row_id=rid)
  }) %>%
  ungroup()
catW <- numW %>%
  select(row_id) %>%
  left_join(catW_all,by="row_id")

### FWD 4.4.3: To Arrays ----
nSamp<-nrow(numW)
num_array<-array(unlist(numW$window),dim=c(nSamp,ws,length(numF
   )))
targets<-matrix(numW$target,ncol=1)
pID_hist_array<-t(sapply(catW$window,function(m)m[,"pID_idx"]))
tID_hist_array<-t(sapply(catW$window,function(m)m[,"tID_idx"]))
oID_hist_array<-t(sapply(catW$window,function(m)m[,"oID_idx"]))
hID_hist_array<-t(sapply(catW$window,function(m)m[,"hID"]))
cat_cur_mat<-do.call(rbind,lapply(catW$window,function(m)m[nrow
   (m),]))
pID_cur_array<-matrix(cat_cur_mat[,"pID_idx"],ncol=1)
tID_cur_array<-matrix(cat_cur_mat[,"tID_idx"],ncol=1)
oID_cur_array<-matrix(cat_cur_mat[,"oID_idx"],ncol=1)
hID_cur_array<-matrix(cat_cur_mat[,"hID"],ncol=1)

## FWD 4.5: Split Data ----
```

```
idx<-which(numW$GW<=split_gw)
Xnum_train<-num_array[idx,,,drop=F]
Xnum_val<-num_array[-idx,,,drop=F]
y_train<-targets[idx,,drop=F]
y_val<-targets[-idx,,drop=F]
Xcat_hist_tr<-list(pID_hist=pID_hist_array[idx,,drop=F],
   tID_hist=tID_hist_array[idx,,drop=F],oID_hist=oID_hist_array
   [idx,,drop=F],hID_hist=hID_hist_array[idx,,drop=F])
Xcat_hist_va<-list(pID_hist=pID_hist_array[-idx,,drop=F],
   tID_hist=tID_hist_array[-idx,,drop=F],oID_hist=
   oID_hist_array[-idx,,drop=F],hID_hist=hID_hist_array[-idx,,
   drop=F])
Xcat_cur_tr<-list(pID_cur=pID_cur_array[idx,,drop=F],tID_cur=
   tID_cur_array[idx,,drop=F],oID_cur=oID_cur_array[idx,,drop=F
   ],hID_cur=hID_cur_array[idx,,drop=F])
Xcat_cur_va<-list(pID_cur=pID_cur_array[-idx,,drop=F],tID_cur=
   tID_cur_array[-idx,,drop=F],oID_cur=oID_cur_array[-idx,,drop
   =F],hID_cur=hID_cur_array[-idx,,drop=F])

## FWD 4.6: Building the Keras Neural Network Model ----
nP<-max(fwd$pID_idx,na.rm=T);nT<-max(fwd$tID_idx,na.rm=T);nO<-
   max(fwd$oID_idx,na.rm=T)
inp_num<-layer_input(shape=c(ws,length(numF)),name="input_seq")
inp_ph<-layer_input(shape=c(ws),dtype="int32",name="pID_hist")
inp_th<-layer_input(shape=c(ws),dtype="int32",name="tID_hist")
inp_oh<-layer_input(shape=c(ws),dtype="int32",name="oID_hist")
inp_hh<-layer_input(shape=c(ws),dtype="int32",name="hID_hist")
inp_pc<-layer_input(shape=c(1),dtype="int32",name="pID_cur")
inp_tc<-layer_input(shape=c(1),dtype="int32",name="tID_cur")
inp_oc<-layer_input(shape=c(1),dtype="int32",name="oID_cur")
inp_hc<-layer_input(shape=c(1),dtype="int32",name="hID_cur")

emb_ph_hist_stream<-inp_ph%>%layer_embedding(input_dim=nP+1,
   output_dim=emb_dim,mask_zero=T)
emb_th_hist_stream<-inp_th%>%layer_embedding(input_dim=nT+1,
   output_dim=emb_dim,mask_zero=T)
```

```
emb_oh_hist_stream<-inp_oh%>%layer_embedding(input_dim=nO+1,
   output_dim=emb_dim,mask_zero=T)
emb_hh_hist_stream<-inp_hh%>%layer_lambda(function(x){
   tf$expand_dims(tf$cast(x,tf$float32),axis=as.integer(-1))})
cat_hist_seq_stream<-layer_concatenate(list(emb_ph_hist_stream,
   emb_th_hist_stream,emb_oh_hist_stream,emb_hh_hist_stream),
   axis=-1)

emb_pc_cur_stream<-inp_pc%>%layer_embedding(input_dim=nP+1,
   output_dim=emb_dim)%>%layer_flatten()
emb_tc_cur_stream<-inp_tc%>%layer_embedding(input_dim=nT+1,
   output_dim=emb_dim)%>%layer_flatten()
emb_oc_cur_stream<-inp_oc%>%layer_embedding(input_dim=nO+1,
   output_dim=emb_dim)%>%layer_flatten()
emb_hc_cur_stream<-inp_hc%>%layer_lambda(function(x)tf$cast(x,
   tf$float32))
seq_pc_cur_stream<-emb_pc_cur_stream%>%layer_repeat_vector(n=ws
   )
seq_tc_cur_stream<-emb_tc_cur_stream%>%layer_repeat_vector(n=ws
   )
seq_oc_cur_stream<-emb_oc_cur_stream%>%layer_repeat_vector(n=ws
   )
seq_hc_cur_stream<-emb_hc_cur_stream%>%layer_repeat_vector(n=ws
   )
current_cat_as_seq_stream<-layer_concatenate(list(
   seq_pc_cur_stream,seq_tc_cur_stream,seq_oc_cur_stream,
   seq_hc_cur_stream),axis=-1)

lstm_in_stream<-layer_concatenate(list(inp_num,
   cat_hist_seq_stream,current_cat_as_seq_stream),axis=-1)
lstm_out_stream<-lstm_in_stream%>%layer_lstm(units=64,
   return_sequences=F)%>%layer_dropout(rate=0.2)

head_stream<-lstm_out_stream%>%layer_dense(units=32,activation
   ="relu")
out_stream<-head_stream%>%layer_dense(units=1,activation="
```

```
   linear")

model<-keras_model(inputs=list(inp_num,inp_ph,inp_th,inp_oh,
   inp_hh,inp_pc,inp_tc,inp_oc,inp_hc),outputs=out_stream,name
   ="fwd_lstm_model_v2")
model%>%compile(optimizer=optimizer_adam(),loss="mse",metrics=
   metric_root_mean_squared_error())

summary(model)
keras_python<-import("keras")
keras_python$utils$plot_model(model,to_file="keras-model-fwd-v2
   .png",show_shapes=T,show_dtype=F,show_layer_names=T,rankdir
   ="TB",expand_nested=F,dpi=200,show_layer_activations=F)
img<-readPNG("keras-model-fwd-v2.png")
grid::grid.raster(img)

## FWD 4.7: Callback Functions and Scaling Factors ----
weights_filepath_fwd<-"R Forecast/best_fwd_v2.hdf5"
early_stopping<-callback_early_stopping(monitor="val_loss",
   patience=num_patience,restore_best_weights=T)
model_checkpoint_fwd<-callback_model_checkpoint(filepath=
   weights_filepath_fwd,monitor="val_loss",save_best_only=T,
   save_weights_only=T,mode="min",verbose=1)
reduce_lr<-callback_reduce_lr_on_plateau(monitor="val_loss",
   factor=0.2,patience=floor(num_patience*0.5),min_lr=1e-6,
   verbose=1)
scaling_factors$fwd<-list(mu=mu,sigma=sigma,numF=numF)

## FWD 4.8: Train Forwards LSTM Model ----
history<-model%>%fit(
  x=c(list(input_seq=Xnum_train),Xcat_hist_tr,Xcat_cur_tr),
  y=y_train,
  epochs=epoker,
  batch_size=16,
  validation_data=list(c(list(input_seq=Xnum_val),Xcat_hist_va,
     Xcat_cur_va),y_val),
```

```
  callbacks=list(early_stopping,model_checkpoint_fwd,reduce_lr)
)

## FWD 4.9: Generate Predictions & Invert Scaling ----
val_row_ids<-numW$row_id[-idx]
pred_all<-model%>%predict(c(list(input_seq=Xnum_val),
   Xcat_hist_va,Xcat_cur_va))
preds_fwd<-tibble(
  row_id=val_row_ids,
  predicted_total_points_scaled=as.vector(pred_all),
  actual_total_points_scaled=as.vector(y_val)
) %>%
  mutate(
    predicted_total_points=predicted_total_points_scaled*sigma+
       mu,
    actual_total_points=actual_total_points_scaled*sigma+mu
  ) %>%
  left_join(unscaled_fwd,by="row_id")

## FWD 4.10 Save model and validation set predictions ----
model_list$fwd<-model
preds_fwd_clean<-preds_fwd%>%select(row_id,GW,player_id,name,
   position,team,actual_total_points,predicted_total_points,
   value)

## FWD 4.11: Plots for Evaluation ----
history_fwd<-history
history_df_fwd<-data.frame(history_fwd)
plot_history_fwd<-plot(history_fwd)
print(plot_history_fwd)

plot_actual_predicted_fwd<-ggplot(preds_fwd,aes(x=
   actual_total_points,y=predicted_total_points))+
  geom_point(alpha=0.5)+
  geom_abline(intercept=0,slope=1,linetype="dashed",color="red
     ")+
```

```
  labs(title="FWD Actual vs. Predicted",x="Actual",y="Predicted
    ")+
  facet_wrap(~position)+
  theme_grey()+
  coord_cartesian(xlim=range(preds_fwd$actual_total_points,na.
    rm=T),ylim=range(preds_fwd$predicted_total_points,na.rm=T)
    )
print(plot_actual_predicted_fwd)

preds_fwd_residuals<-preds_fwd%>%mutate(residual=
   actual_total_points-predicted_total_points)
plot_residuals_predicted_fwd<-ggplot(preds_fwd_residuals,aes(x=
   predicted_total_points,y=residual))+
  geom_point(alpha=0.5)+
  geom_hline(yintercept=0,linetype="dashed",color="red")+
  labs(title="FWD Residuals vs. Predicted",x="Predicted",y="
    Residual")+
  facet_wrap(~position)+
  theme_grey()
print(plot_residuals_predicted_fwd)

plot_distribution_residuals_fwd<-ggplot(preds_fwd_residuals,aes
   (x=residual))+
  geom_histogram(aes(y=after_stat(density)),bins=30,fill="blue
    ",alpha=0.7)+
  geom_density(color="red")+
  geom_vline(xintercept=0,linetype="dashed",color="black")+
  labs(title="FWD Residual Distribution",x="Residual",y="
    Density")+
  facet_wrap(~position)+
  theme_grey()
print(plot_distribution_residuals_fwd)

model_evaluation_fwd<-model|>evaluate(c(list(input_seq=Xnum_val
   ),Xcat_hist_va,Xcat_cur_va),y_val,verbose=0)
validation_metrics$fwd<-model_evaluation_fwd
```

```
# Merging Datasets ----
validation_results_df <- bind_rows(
  preds_gk_clean, preds_def_clean, preds_mid_clean,
     preds_fwd_clean
) %>% arrange(player_id, GW)

forecastdf_detailed <- bind_rows(preds_gk, preds_def, preds_mid
   , preds_fwd)

## Evaluation Metrics Dataframe ----
metrics_df <- data.frame(
  metric = names(validation_metrics$mid), # Assumes metric
     names are consistent
  GK = unlist(validation_metrics$gk),
  DEF = unlist(validation_metrics$def),
  MID = unlist(validation_metrics$mid),
  FWD = unlist(validation_metrics$fwd)
)
validation_metrics
print(metrics_df)

# Save plots
plot_dir <- "C:/Users/peram/Documents/test/R Forecast/Plots
   from LSTM Forecast script"
if (!dir.exists(plot_dir)) dir.create(plot_dir, recursive =
   TRUE)

# GK plots
ggsave(file.path(plot_dir,"plot_history_gk_v2.png"), plot=
   plot_history_gk, width=7, height=5)
ggsave(file.path(plot_dir,"plot_actual_predicted_gk_v2.png"),
   plot=plot_actual_predicted_gk, width=7, height=5)
ggsave(file.path(plot_dir,"plot_residuals_predicted_gk_v2.png")
   , plot=plot_residuals_predicted_gk, width=7, height=5)
```

```
ggsave(file.path(plot_dir,"plot_distribution_residuals_gk_v2.
   png"), plot=plot_distribution_residuals_gk, width=7, height
   =5)

# DEF plots
ggsave(file.path(plot_dir,"plot_history_def_v2.png"), plot=
   plot_history_def, width=7, height=5)
ggsave(file.path(plot_dir,"plot_actual_predicted_def_v2.png"),
   plot=plot_actual_predicted_def, width=7, height=5)
ggsave(file.path(plot_dir,"plot_residuals_predicted_def_v2.png
   "), plot=plot_residuals_predicted_def, width=7, height=5)
ggsave(file.path(plot_dir,"plot_distribution_residuals_def_v2.
   png"), plot=plot_distribution_residuals_def, width=7, height
   =5)

# MID plots
ggsave(file.path(plot_dir,"plot_history_mid_v2.png"), plot=
   plot_history_mid, width=7, height=5)
ggsave(file.path(plot_dir,"plot_actual_predicted_mid_v2.png"),
   plot=plot_actual_predicted_mid, width=7, height=5)
ggsave(file.path(plot_dir,"plot_residuals_predicted_mid_v2.png
   "), plot=plot_residuals_predicted_mid, width=7, height=5)
ggsave(file.path(plot_dir,"plot_distribution_residuals_mid_v2.
   png"), plot=plot_distribution_residuals_mid, width=7, height
   =5)

# FWD plots
ggsave(file.path(plot_dir,"plot_history_fwd_v2.png"), plot=
   plot_history_fwd, width=7, height=5)
ggsave(file.path(plot_dir,"plot_actual_predicted_fwd_v2.png"),
   plot=plot_actual_predicted_fwd, width=7, height=5)
ggsave(file.path(plot_dir,"plot_residuals_predicted_fwd_v2.png
   "), plot=plot_residuals_predicted_fwd, width=7, height=5)
ggsave(file.path(plot_dir,"plot_distribution_residuals_fwd_v2.
   png"), plot=plot_distribution_residuals_fwd, width=7, height
   =5)
```

```
## Export Predicted Validation Set as .CSV ----
write_csv(validation_results_df, "
   Validation_Predictions_Clean_v2.csv")
write_csv(forecastdf_detailed, "
   Validation_Predictions_Detailed_v2.csv")
write.table(validation_metrics, "Validation metrics.txt")

### Lasso selected features ----
cat("GK Lasso Features:\n"); print(numF_gk) ; length(numF_gk)
cat("DEF Lasso Features:\n"); print(numF_def) ; length(numF_def
   )
cat("MID Lasso Features:\n"); print(numF_mid) ; length(numF_mid
   )
cat("FWD Lasso Features:\n"); print(numF_fwd) ; length(numF_fwd
   )
initial_numF_gk
initial_numF_def
initial_numF_mid
initial_numF_fwd
```

# I Discussion paper: Responsibility - Emil B. Andersen

This master thesis is centered around the famous Fantasy Premier League (FPL). The FPL is an online game that allows participants to assume the role of football managers, creating their own teams composed of real-life players from the English Premier League (EPL). The game itself is based on real-life outcomes, with assets (real-life players) gaining points based on a plethora of performance metrics for each game throughout the course of a complete season. These performance metrics include goals scored, assists, clean sheets, saves, and even disciplinary actions like yellow and red cards. The structure of the game encourages participants to apply strategic thinking, statistical analysis, and predictive judgment to optimize their team selections on a weekly basis.

The objective of the game is to accumulate as many points as possible, with participants competing against each other through both local and global ranking systems. These competitive elements add layers of social engagement, as players can join mini-leagues with friends or test their skills against millions of other managers worldwide. Furthermore, the game introduces a financial component, with many private leagues offering cash prizes and other incentives to the highest-ranked participants. This competitive and sometimes monetary aspect of FPL often amplifies the emotional and psychological stakes involved in playing the game, blurring the lines between casual gaming and financial commitment.

In this thesis, forecasts of player points are generated through an advanced machine learning technique, namely Long Short-Term Memory (LSTM), a variant of Recurrent Neural Networks (RNNs) specifically designed to capture long-term temporal dependencies in sequential data. The application of LSTM in this context allows for the modelling of player performance based on historical data and recent form among other predictive factors. The forecasts are thereafter fed into an optimization model, built on the framework of Eilertsen et al. (2018), which optimizes team selection over expected points while adhering to FPL's game constraints. These include budget limitations, which restrict the total value of the selected squad to £100 million, and positional requirements, which enforce a fixed structure of two goalkeepers, five defenders, five midfielders, and three forwards. To construct both the LSTM and optimization models in R and Python respectively, OpenAI (2025) has been used. Additionally, the optimization strategy must factor in transfer limitations, weekly point penalties for excess transfers, and the impact of in-game "chips" that can alter team performance during specific game weeks. The integration of machine learning forecasts with a structured optimization framework aims to enhance decision-making, providing a robust tool for maximizing fantasy football performance.

## I.1 Responsibility considerations

### I.1.1 Gambling concerns

While the game poises an interesting challenge for sports enthusiasts, there are concerns regarding it's connections to gambling and anxiety. The "gamblification" of the English Premier League is well-known, with many teams even exhibiting gambling sponsors on their match-worn shirts. This normalization of gambling within the sport introduces ethical concerns, particularly given the global audience of the EPL, which includes millions of underage viewers. The integration of betting into the sport's mainstream culture not only promotes gambling as a regular part of fandom but also potentially influences younger audiences to perceive betting as inherently linked to football enjoyment. On their official websites statement of principles, (https://www.premierleague.com/about/statement-of-principles), the Premier League does not even mention how this may be problematic, or what they do to limit the potential harm caused by it. Griffiths and Parke (2017) explores how the marketing of betting in football has evolved, and found that an increasing amount of money went into shirt sponsorships in the English Premier League, with all teams having at least one official betting partner. With the English Premier League being available to watch for everyone with a tv-subscription or match ticket, gambling is directly marketed to all consumers of it, be it recovering gambling addicts or children. While they found it difficult to prove that this lead to an increase in gambling, they argued that it was at least highly feasible. Martin and Nelson (2014) studied the relationship between gambling-related problems and fantasy sports participation. Their survey consisted of 1556 US college students, 11.5% of whom had participated in fantasy sports in the past year. This group was split into two main categories, separated by gender and whether they played for money or not. Their findings were that fantasy sports participants were significantly more likely to have gambling problems, and that this was especially prevalent for female participants. The relationship was also significant for males who played for money, although not as striking as female participants. The study is limited however, as it required participants to self-report on gambling problems, was restricted to a single US college (although many different majors), and also required participants to remember events as far as a year back in time.

### I.1.2 Internet addiction

Columb et al. (2022) studied the frequency of internet addiction and gambling for Fantasy Sports players compared to the general population. They found that Fantasy Sports players generally spent more time on the internet, with more of them meeting the set criteria for internet addiction compared to the general public. The most frequent amount of time spent

on daily fantasy sports was between 30-60 minutes on weekdays, and 1-2 hours on weekends. In the worst cases, internet addiction is associated with depression, anxiety and substance misuse. Furthermore, they found that the majority of Fantasy Sports players had gambled at least once in the last year.

### I.1.3 Mental health implications

Wilkins et al. (2021) provided the first empirical evidence into how fantasy sports participation can have a negative impact on mental health. Through a survey on 1995 participants, they set out to measure low mood, anxiety, functional impairment and problematic behaviour, i.e. purely negative aspects with regards to mental health. Furthermore, they measured experience and level of engagement to check for potential correlations. They found that 24.6% of participants experienced low mood or worse as a direct consequence of fantasy sports participation, while 19.6% experienced mild anxiety or worse and 13.6% to have significant functional impairment or worse. Lastly, 17.6% were found to have problematic gaming behaviour. The paper is limited however, in the sense that it does not compare these numbers to that of the general population. Furthermore, all data was collected during the Covid-19 pandemic, which saw significant increases in negative mental health in the general population. Nonetheless, the negative aspects were more prevalent for heavily engaged players. Those who spent more time on the game scored significantly worse for mental health, although this seemed to drop off the more experienced the player was. Wilkins et al. (2024) set out to explore the robustness of the findings from Wilkins et al. (2021). This was done through collecting data once the Covid-19 pandemic was over, via a survey similar to that of Wilkins et al. (2021), but now also exploring positive mental health gains through fantasy football. They still found that experienced players seemed to exhibit less anxiety compared to their counterparts, although this did not hold up when testing for stress, depression and negative mood. Regarding heavily engaged players, the findings remained the same, i.e. significantly higher mental health concerns. However, this group of players were also found to gain significantly more positive mood from their participation, showing that their participation is not purely depreciative. The negative health concerns was also significant for participants with the highest financial involvement, showing an increase in stress, anxiety, negative mood and problematic behaviour compared to their counterparts. At the same time, this group of players were also found to gain more positive mood out of the game. In general, the players who experienced more positive mood, were also more susceptible to all negative health concerns. This study is also limited, however, in the sense that the survey was answered only once by each participant, providing a snapshot of their feelings towards the game. With the game running over the course of a season, it could have been interesting

to track their feelings relative to their performance in-game. The same notion is supported by the authors themselves.

## I.2 Paths for mitigation

With all the negative aspects with regards to gambling, addiction and mental health concerns, the platform operators, in this case the English Premier League and EA Sports, have a responsibility to mitigate them. A good start could be to recognize these issues on their own websites statement of principles, and to warn new and existing players of the potential impacts of participation. With regards to privately organized cash-prize leagues, a clear stance should be taken from the organizers, distancing themselves from such events. While more demanding and costly for the organizers, the entrenchment of gambling sponsors in the English Premier League should also be addressed. The marketing of betting platforms to all viewers via shirt sponsorships and more is highly problematic, and normalizes the core issue. Furthermore, built-in tools which lets participants keep track of their own time commitment to the game could be used to encourage responsible participation. This could be done via, for example, pop-up warnings, which lets the participant know that they have spent a significant amount of time, and that it is time to take a break from the game. Whatever path is taken to mitigate the problematic aspects of the Fantasy Premier League, the rapidly growing player base of the game, currently beyond 11 million participants, makes it an increasingly urgent matter.

## I.3 Concluding remarks

This thesis set out to marry advanced machine learning techniques with rigorous optimisation in the context of Fantasy Premier League (FPL). On the technical side, we demonstrated that Long Short-Term Memory (LSTM) networks can successfully learn from extensive historical player data, capturing not only simple trends in goals, assists, and clean sheets but also more subtle temporal patterns in player quality and fixture difficulty. By feeding these forecasts into a mixed-integer linear programming model that respects FPL's budget, positional, and transfer constraints, we constructed a decision-support tool capable of generating weekly squad selections. This two-stage approach—forecast then optimize—is both modular and extensible, meaning future work could incorporate richer feature sets (e.g. weather or player fatigue metrics) or alternative optimisation objectives (e.g. minimizing variance, maximizing differential points in head-to-head formats).

Fantasy sports is not without caveats, however, with research associating it to gambling-related problems, internet addiction and mental health concerns. Unfortunately, these aspects are not recognized by the English Premier League or EA Sports themselves, the or-

ganizers of the Fantasy Premier League. In this regard, their room for improvement is significant, and as a minimum they need to acknowledge the negative impacts of their own game. This may be optimistic, however, as they stand to gain nothing from encouraging their player-base to spend less time on their game. Nonetheless, a clear stance should be taken, as is their ethical duty. Built-in tools should be implemented to encourage responsible participation and reduced commitment with regards to time and financial involvement. Furthermore, the English Premier League should distance themselves from betting platforms, which are currently marketed to all viewers of their elite football league. With the monetary incentives these platforms bring, it would undoubtedly represent a costly process for the powers that be, but it would send a clear signal that the English Premier League and its clubs do not stand for it.

In light of the negative aspects of fantasy sports, we recognize how such a thesis may further encourage problematic time commitment and internet addiction with regards to FPL. However, by automating the process, we believe that our contribution is largely positive, freeing up time for participants who adopt our approach. The model collects data, processes it and generates a team. Thus, all the adopter needs to do is to run a script and select the team as proposed by the model. At the same time, there is a risk that increased confidence in model-driven advice might embolden some users to place heavier and more frequent bets in private cash-prize leagues. We do not encourage this type of behaviour, but are aware that it is out of our control once the model is publicly available.

# References


Columb, D., Griffiths, M. D., & O'Gara, C. (2022). Fantasy football (soccer) playing and internet addiction among online fantasy football participants: A descriptive survey study. *International Journal of Mental Health and Addiction*, *20*(2), 1200–1211. https://doi.org/10.1007/s11469-020-00436-4

Eilertsen, W., Gupta, A., & Kristiansen, B. K. (2018). *Developing a Forecast-Based Optimization Model for Fantasy Premier League* [Master's thesis, NTNU]. Retrieved February 3, 2025, from https://ntnuopen.ntnu.no/ntnu-xmlui/handle/11250/2577003

Griffiths, M. D., & Parke, J. (2017). Gambling addiction and its connection to machine learning in sports analytics: A growing concern. *International Journal of Mental Health and Addiction*, *16*(5), 1055–1070. https://doi.org/10.1007/s11469-017-9788-1

Martin, R. J., & Nelson, S. (2014). Fantasy sports, real money: Exploration of the relationship between fantasy sports participation and gambling-related problems. *Addictive Behaviors*, *39*(10), 1377–1382. https://doi.org/10.1016/j.addbeh.2014.05.017

OpenAI. (2025). Chatgpt [(May 15th version) [Large language model].].

Wilkins, L., Churchyard, J., Dowsett, R., & Britton, G. (2024). Exploring fantasy football involvement and mental health through player experience, engagement levels, social comparisons, and financial incentives. *Simulation & Gaming*, *55*(6), 1032–1056. https://doi.org/10.1177/10468781241261663

Wilkins, L., Dowsett, R., Zaborski, Z., Scoles, L., & Allen, P. M. (2021). Exploring the mental health of individuals who play fantasy football. *Human Behavior and Emerging Technologies*, *3*(5), 1004–1022. https://doi.org/10.1002/hbe2.316

# J Discussion Paper International - Per Anders Molberg

This discussion paper addresses the concept of "international" as one of the three key pillars—international, innovative, and responsible—that define the UiA School of Business and Law's mission statement and strategic direction. The paper aims to demonstrate the critical thinking and broader perspective expected of master's students by examining how international dimensions interlock with contemporary business and finance research.

The discussion paper will attempt to touch upon several trends related to the research question from our thesis, extending to not only Fantasy Premier League. But additionally drawing lines to the English Premier League and other men's elite football Leagues around the world. This paper will first introduce the topic of the thesis, the Fantasy Premier League. Then an introduction to the master thesis will follow. Following the introduction to the master thesis , a brief definition of internation, then the discussion of the topic "international" in context of Premier League, and Fantasy Sports. And lastly a summarising conclusion will end the discussion paper.

In alignment with the University of Agder's commitment to digital literacy, academic integrity and technological integration in learning, this paper acknowledges the use of advanced digital tools, including large language models such as Gemini 2.5 Flash/Pro(Free versions(Google, 2025)) and Chat GPT (Free version, (OpenAI, 2025)), to enhance academic writing quality, assist with technical formatting in the typesetting environment LaTeX(Overleaf Premium), and support in English language refinement.

## J.1 About the thesis

The master thesis that will be discussed in terms of the broad concept of "International", is titled "Fantasy Premier League: A Data-Driven Approach - Exploring LSTM-networks for performance forecasting" . The thesis is the culmination of experiences and knowledge gathered from two-years at the Analytical Finance specialisation of the master's programme for Business Administration. The research question attempts to "address the question, of whether or not a purely data driven approach, relying exclusively on historical in-game data and Long-Short Term Memory driven optimisation can replicate or beat human performance at a complex and information driven game like the Fantasy Premier League."(Andersen & Molberg, 2025).

### J.1.1 Fantasy Premier League

To understand the point of the thesis and its research questions. The research subject needs to be explained: The Fantasy Premier League is game made the by the English Premier

League, and is a free online game that follows the real English Premier League across the whole season. The game gives football fans to act as managers for their own team with real life Premier League players, called fantasy squads. From here on out, managers refer to Fantasy Premier League participants, squads refer to manager created fantasy teams, and players refer to real life footballers. The main event of the game is to gather as many points as possible with a chosen squad, where the point comes from choosing players that perform well in the Premier League. There are rules imposed to make the game more interesting and difficult, the main ones are limitations on budget, transfers(swapping out players in your squad), and in-game temporary point boosters. In real football, teams play against each other to win a league based on wins, in Fantasy premier league, squads do not play against each other. They collect points from players, and the squads are ranked based on point collected across the season. In Fantasy Premier League managers can create their own mini-leagues (with friends, coworkers etc.), but also participate in global rankings. The (English) Premier league is globally the top elite men's football league if measured by money spent, an one of the most popular elite men's football leagues in the world based on popularity(Deloitte, 2024). So it should be no surprise the spin-off online game is immensely popular domestically and internationally, the current Fantasy Premier League seasons has over 11 million participants from across the globe. The Fantasy Premier League even has an official Podcast and makes Fantasy Premier Leauge media content available online which mimics post-match events from the real thing, and there have been written several books on the topic of performing well at the game in different languages for example Roche (2023), Rogers (2023), and Whelan (2021). Claiming that the Fantasy game itself is a cultural phenomenon would be an understatement, although it is relatively small compared to the actual English Premier League.

### J.1.2 The Master Thesis

Moving over to the main event of the thesis, it sought to employ machine learning to forecast player performance based in historical Fantasy Premier League game data, in an attempt to see if we could beat humans with a "machine". The nature of the game lends itself to be used as an interesting sandbox for utilising a quantitative finance data-driven decision-making approach. Therefore, in the thesis, Long-Short Term Memory(LSTM) Neural Network(NN) models were trained and used to create forecasts together with a Fantasy Premier League Squad Optimisation Mixed Integer Linear Program(MILP) model developed by Eilertsen et al. (2018) from NTNU. When doing the preliminary exploratory data analysis, we found the Fantasy Premier League data to be challenging to work with for because of characteristics. The data was non-linear, appeared to have a non-normal multimodal distribution, and

different player positions had different distributions from one another. From these initial findings, we decided to train four models, one for each position, in the hope that this would help in the models being better at generalising player performance for each position. These models would then forecast data for a period of thirty-one rounds of premier league football matches which already had been recorded. From these forecasts we would compare the models guesstimates vs actual data, to understand the predictive power of them. Then we used both forecasted and actual data to perform fantasy squad optimisation. From our results we conclude that we fail to beat humans, rather we replicate the performance of a low-level performing manager. However, the model has a few short-comings which is addressed in the thesis discussion.

A non-technical explanation will follow. The research question for the thesis wonders if it possible to train a machine to make predictions about how many fantasy points a given real-life football player will get each match they play. The machine gets to "watch" the transcripts of each football match for two years, then we ask it to guess how many points a player will get - a specific game he plays. We "ask" this for all players for each round of Premier League, the trick is we ask about matches that have already played, but the model does not know about that. When then compare the models guesses against the real points players achieved. After collecting all model guesses for the whole season so far(until 31st round), we use a mathematical program to optimise which players to have on our fantasy squad.

## J.2 Defining International

Before discussing international trends and the internationalisation of Fantasy Premier League. It is imperative that we discuss what is meant by international, trends and internationalisation. In the Cambridge dictionary the word international is defined as "involving more than one country", and it defines the word trend as "a general development or change in a situation or in the way that people are behaving", and lastly it defines internationalisation as "the action of becoming or making something become international" (Cambridge University Press, 2025)

From these definitions we can establish that international trends within our topic, that is Elite Men's Football Divisions, the English Premier League and Fantasy Premier League, can be tailored to be "the general developments towards the fantasy premier league, and the premier league being consumed and interacted with beyond the borders of England." This definition is broad in nature and would indeed be a true for fantasy premier league players in Scotland, Wales and other countries within the U.K.

## J.3 Discussion of International Trends and Internationalisation of Fantasy Premier League

In light of the established definition, our research topic is one which is international in nature, we are not English, yet the English Premier League interests us. By extension, its side game also interest us, and lets us "participate" even though we are a sea apart from the real game. In the editorial 'It's a whole new ball game': thirty years of the English Premier League by Parnell et al. (2022), the authors go through some history of the premier league, and how it developed over the last thirty years to become a globally consumed football elite men's league. For example, foreign ownership of clubs like Chelsea, Manchester City, Manchester United, Liverpool and others has lead to more money funnelling into the league increasing the level of the league but also it global consumption. The trend of foreign ownership of the premier league and the global marketing of the premier league, and also other European leagues is a trend which will most likely continue. Another international trend is the onset of online fantasy sport games. All major leagues in the US have them, and they are administered and provided by the leagues themselves, instead of being made by newspapers or other media outlets(like previously in Argentina (Bonomo et al., 2014)). As stated in Wilkins (2024) about 60 million individuals played these games in north America in 2020. Classical sports games as those made by Electronic Arts, are different in function. Where as those games like Fifa (now F.C) focus on simulation games for consoles or computer games, which are played with actively playing and controlling the game. Fantasy Sports take a more passive approach, and lets the player take on a manager role, for free online. Fantasy Sports games administered by the elite sport leagues are meant to enhance the fan experience, rather than to make a profit selling a game. The Online Fantasy Sports games are called "complementary digital experiences"(Yuksel et al., 2021), and are "touchpoints in the customer journey that affect the value of a core brand experience, transforming it into a technology-enhanced experience"(Yuksel et al., 2021). Furthermore to start playing you do not need any devices to entry than a phone, tablet or computer, opposed to a simulation game. Fantasy Sports games also reflect real world events rather than being based on the real world, but interacting with the players in a simulation for single player like Sega's Football Manager series. Fantasy Premier League made by the league as a customer enhancement add-on, and is complementary to the main event and drives user engagement and acts as free advertising. This is also done by other leagues. Fantasy Sports Research is also an emerging trend. From a bibliometric overview of the topic produced by (Wilkins, 2024), look into the different topics and themes of the emerging body of literature surrounding fantasy sports, they find that most research is published in the US, but also internationally. However, it seems researchers are more interested in performing qualitative research, rather than fully

quantitative ones. They find the top main themes for fantasy sports research is in order: Gambling, Sports Consumption, Law, Motivations for participation, Social Sciences and Fantasy Sports Strategy. That is, how stakeholders react and interact with the game, and the consequences of their interactions with the game. The most popular journals for publishing the research, which are mostly of novel and original character, are Business Economics, Sport, Law and Psychology. Furthermore, the application of statistics in sports is not new, and has been done in other sports successfully, a popularised successes example is Moneyball(book and movie), where quantitative measures are employed to do team selection in American Baseball. Sports analytics is a growing field and has been used since the mid-1990s in the EPL, naturally this trend have influenced the game of Fantasy Premier League. The onset of various machine learning techniques or other use cases of Artificial Intelligence has been observed in Football and other sports, and have been researched sparingly in Fantasy Sports Research specifically the Fantasy Premier League since the first major research paper by Matthews et al. (2012) with some growth in the past 5 years (Wilkins, 2024). Seeing analytical approaches being used by fans and individuals playing Fantasy Premier League but also other Fantasy Sports are not strange considering there are monetary rewards from winning the game, but also globally there have been introduction of gambling in fantasy sport games. Like in business, the digitisation and push towards "big-data" and the exploitation of data to drive decision-making is a trend which is growing globally.

## J.4 Conclusion

The discussion paper has sought to discuss the "international" dimension of the master thesis "Fantasy Premier League: A Data-Driven Approach - Exploring LSTM-networks for performance forecasting". The master thesis is the final course and produce of the master's programme, applying knowledge from the last 2 years at the University of Agder, and learning more. The thesis has been massively useful in testing the boundaries of our knowledge and pushing us to learn more and applying technical skills to make it. Internationally fantasy premier league and other fantasy sports games, which are meant as complementary digital engaging customer enhancing experiences are growing trends, which brings and retains a larger audience to the main sports events. The consequent research on fantasy sports is a reaction by academics to be able to understand the effects this digital phenomena have on individuals. Fantasy Sports are a global trend which is here to stay, and no doubt more research will be conducted on both how to become better the game(and the transfer value of this research for other related problems) and to better understand the individuals playing the game, and how to better regulate the growing industry to be safe and productive for the sports industry and society as a whole.

# References


Andersen, E. B., & Molberg, P. A. (2025). *Predicting fantasy premier league: A data-driven approach* [Master's Thesis]. University of Agder, Faculty of Business and Law, UiA Business School [Supervisor: Jochen Jungeilges].

Bonomo, F., Durán, G., & Marenco, J. (2014). Mathematical programming as a tool for virtual soccer coaches: A case study of a fantasy sport game. *International Transactions in Operational Research*, *21*(3), 399–414. https://doi.org/10.1111/itor.12068

Cambridge University Press. (2025). *Cambridge dictionary* [Accessed: May 27, 2025]. https://dictionary.cambridge.org/

Deloitte. (2024). Annual review of football finance 2024 [Accessed: 3 April 2025].

Eilertsen, W., Gupta, A., & Kristiansen, B. K. (2018). *Developing a Forecast-Based Optimization Model for Fantasy Premier League* [Master's thesis, NTNU]. Retrieved February 3, 2025, from https://ntnuopen.ntnu.no/ntnu-xmlui/handle/11250/2577003

Google. (2025). Gemini [(2.5 Pro/Flash version) [Large language model].].

Matthews, T., Ramchurn, S., & Chalkiadakis, G. (2012). Competing with Humans at Fantasy Football: Team Formation in Large Partially-Observable Domains [Number: 1]. *Proceedings of the AAAI Conference on Artificial Intelligence*, *26*(1), 1394–1400. https://doi.org/10.1609/aaai.v26i1.8259

OpenAI. (2025). Chatgpt [(May 15th version) [Large language model].].

Parnell, D., Rookwood, J., Bond, A., Widdop, P., & and, J. A. L. L. (2022). 'it's a whole new ball game': Thirty years of the english premier league. *Soccer & Society*, *23*(4-5), 329–333. https://doi.org/10.1080/14660970.2022.2059853

Roche, F. (2023). *Fantasyvettreglene - en fantasy premier league-håndbok*. Sitamoja.

Rogers, P. (2023, July). *Fantasy premier league*. Arrowcroft Press.

Whelan, M. K. (2021, May). *Fpl obsessed*. Arrowcroft Press.

Wilkins, L. (2024). A bibliometric analysis of fantasy sports research. *Entertainment Computing*, *48*, 100613. https://doi.org/10.1016/j.entcom.2023.100613

Yuksel, M., Smith, A. N., & Milne, G. R. (2021). Fantasy sports and beyond: Complementary digital experiences (cdxs) as innovations for enhancing fan experience. *Journal of Business Research*, *134*, 143–155. https://doi.org/10.1016/j.jbusres.2021.05.037